

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Advanced Big Data Analytics

Project 2

Mobile Money Investigations

with PostgreSQL and Neo4j

Capture, relational and graph models, investigation queries, correctness, performance, concurrency and recommendations for 20,003,300 transfers of September 2026

Group members

No.	Full name	Student ID
1	Victor DUSENGIMANA	101372
2	Odile UMWIZA	101365
3	Ildephonse IGIRUKWAYO	101404
4	Alain Christian MUKIZA	101370

Source code and README on GitHub

GitHub Repository

Lecturer: **Temitope Ogutande**

Table of Contents

Introduction **3**
- Aim and questions 3
- Approach and tools 3
- Main findings 3
- How the report is organised 4

1 Capture the data **5**
- 1.1 Prepare the workspace 5
- 1.2 Build capture.py 8

2 Build PostgreSQL **12**
- 2.1 Define the five tables 12
- 2.2 Bulk-load and check 14

3 Build Neo4j **18**
- 3.1 Map the same facts 18
- 3.2 Prepare and import the CSVs 19
- 3.3 Inspect one account in both models 24

4 Answer routine business questions **26**
- 4.1 Account activity 26
- 4.2 Daily activity 27
- 4.3 Shared devices 29

5 Investigate connected accounts **30**
- 5.1 Ownership and device network 30
- 5.2 Rapid onward payments 33
- 5.3 Circular payments 35
- 5.4 Shortest transfer connection 37

6 Check correctness and performance **40**
- 6.1 Build one comparison program 40
- 6.2 Time a fixed selection 42
- 6.3 Try one SQL improvement 48

7 Test concurrent requests **51**
- 7.1 Build a small load test 51
- 7.2 Explain scaling 54

8 Make and defend decisions **57**
- 8.1 Prioritize investigation 57
- 8.2 Recommend a model 59
- 8.3 Explain individual judgement 60

9 Submit reproducible work **62**
- 9.1 Prepare the report 62
- 9.2 Provide code and verification 62
- Generator, rerun and stopped services 62

Introduction

A mobile money provider in Rwanda wants its analysts to explain payment activity and to find accounts that need closer review. Investigating an account means following transfers between wallets and examining who owns those wallets or shares their devices. This report documents how our group built the same reports and investigations in **PostgreSQL** and **Neo4j** from the September 2026 records, compared how each model expresses the problem, checked that both give the same answers, measured their performance, and made recommendations.

The data come from the supplied generator `mobile_money_generator.py`, which is used unchanged. It yields 20,993,500 records for 1 to 30 September 2026 (CAT): 228,000 customers, 248,000 accounts (240,000 person, 6,000 merchant and 2,000 agent), 228,600 devices, 285,600 account-device observations and 20,003,300 transfers. Every result was produced by our own programs, SQL and Cypher.

Aim and questions

The project answers five practical questions:

- How can the records be captured and stored in a relational and in a graph model so that both match the source exactly? (Sections 1 to 3)
- What do routine reports show about account activity, daily payment totals and shared devices? (Section 4, Q1 to Q3)
- Which accounts are connected through ownership, devices and transfer sequences, and how closely? (Section 5, Q4 to Q7)
- Do both databases return identical answers, and how do they compare in speed, after one SQL improvement, and under concurrent requests? (Sections 6 and 7)
- Which accounts should be examined first, which model suits each task, and how reproducible is the work? (Sections 8 and 9)

Approach and tools

PostgreSQL 18.6 and Neo4j Community 2026.10.0 (Java Temurin 21.0.12.1) run on one laptop (Intel Core i5-8250U, 8,035 MB RAM, Windows 11 Pro). Python 3.13.6 with psycopg 3.3.6 and the neo4j driver 6.4.1 runs the capture, the query pairs and the comparisons. One database service ran at a time during timing. Repeated evidence is referenced rather than repeated, and complete outputs are kept in the code submission.

Main findings

- **Same data, same answers.** Both databases hold the same records and return identical results in all 61 comparison cases.
- **Each model has its strength.** PostgreSQL is faster for reports and short lookups (Q2 in a median of 21.5 s, where Neo4j timed out); Neo4j is faster for long traversals (Q7 at bound 6 in 37.9 ms against 23.1 s for the first SQL).
- **Much of the gap was implementation.** A both-ends SQL rewrite made Q7 28 to 338 times faster and left PostgreSQL 1.8 to 3.7 times slower than Neo4j at bound 6.
- **Concurrency favours Neo4j here.** With 16 clients Neo4j completed 354.0 Q7 and 505.0 Q4 requests per second against 32.6 and 71.1 in PostgreSQL, which stopped gaining at 8 clients.
- **Investigation order.** A, B and C are reviewed before D, E and F; this is a screening order, not a finding of

fraud.

- **Recommendation.** PostgreSQL as system of record and for reporting, with a read-only Neo4j copy for multi-hop investigation.

How the report is organised

Section	Content
1	Workspace, versions and settings; capture of the five streams to CSV with audit totals
2	PostgreSQL tables, constraints, bulk load, indexes and checks
3	Neo4j graph model, CSV preparation and import, and one account traced in both models
4	Q1 account activity, Q2 daily activity, Q3 shared devices
5	Q4 ownership and device network, Q5 rapid onward payments, Q6 circular payments, Q7 shortest connection
6	61-case correctness comparison, timing of 21 cases per engine, one SQL improvement
7	Load test with 1, 4, 8 and 16 concurrent clients, and an explanation of scaling
8	Investigation priorities, model recommendation and individual judgement
9	Report, code, GitHub repository, verification and stopped services

1 Capture the data

1.1 Prepare the workspace

All work was done on one laptop. **PostgreSQL 18.6** runs natively as the Windows service `postgresql-x64-18`. The project database `mobile_money` was created in a tablespace `mm_space` located at `D:\MobileMoneyProject\pgdata`, so tables, indexes and the CSV files are stored on drive D:. The PostgreSQL time zone is `Africa/Kigali`. **Neo4j Community 2026.10.0** is a dedicated standalone instance in `D:\MobileMoneyProject\neo4j` that uses the default `neo4j` database and its own Java 21 runtime (Neo4j Desktop was not used because the Desktop build available is Enterprise only). The project folders are `code`, `csv`, `import_tmp`, `plans`, `results` and `screenshots`.

The supplied generator was copied unchanged into `code`, and its SHA256 hash `2ea0e59d2b7b987d84a3d151bd9d1ad7b3052bfbc25abf1466c8e94467e9455a` was saved in `results\generator_hash.txt` so that it can be verified again at submission (Section 9).

Recorded environment

Item	Value
CPU	Intel Core i5-8250U @ 1.60 GHz
Cores	4 physical cores, 8 logical processors
RAM	8,035 MB (about 7.8 GB)
Operating system	Microsoft Windows 11 Pro, version 10.0.26300, build 26300
Python and packages	Python 3.13.6 (virtual environment); psycopg 3.3.6 (psycopg-binary 3.3.6), neo4j driver 6.4.1, numpy 2.5.4, pandas 3.0.6, matplotlib 3.11.2
PostgreSQL	18.6 on x86_64-windows, compiled by msvc-19.44.35228, 64-bit; client psql 18.6
PostgreSQL settings	`shared_buffers` 1GB, `work_mem` 32MB, `maintenance_work_mem` 512MB, `effective_cache_size` 3GB, `max_wal_size` 1GB, `max_parallel_workers_per_gather` 2, `TimeZone` Africa/Kigali
PostgreSQL storage	database `mobile_money` in tablespace `mm_space` at `D:\MobileMoneyProject\pgdata`
Neo4j	Community 2026.10.0; default database `neo4j` (type standard, online); Java OpenJDK Temurin 21.0.12.1 (LTS)
Neo4j memory settings	`server.memory.heap.initial_size` 2g, `server.memory.heap.max_size` 2g, `server.memory.pagecache.size` 2g

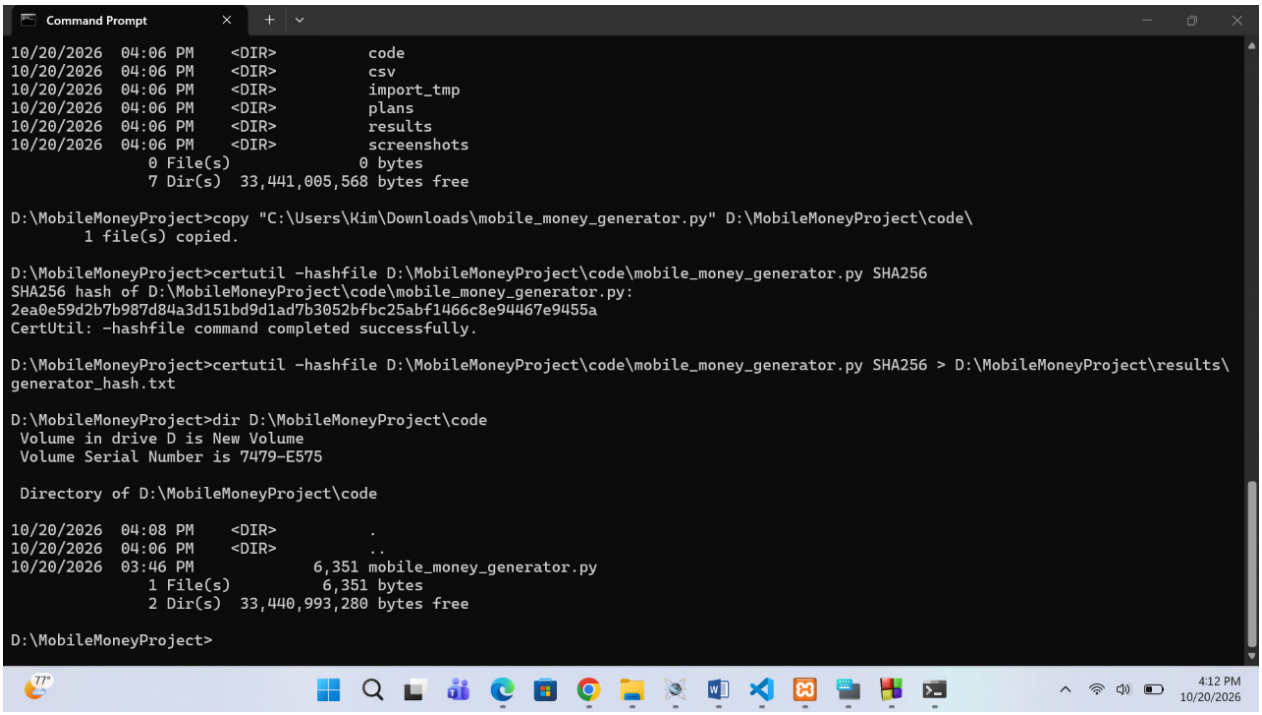


Figure 1.1-1: Project folders, the unchanged generator copied into code, and its SHA256 hash saved to results\generator_hash.txt

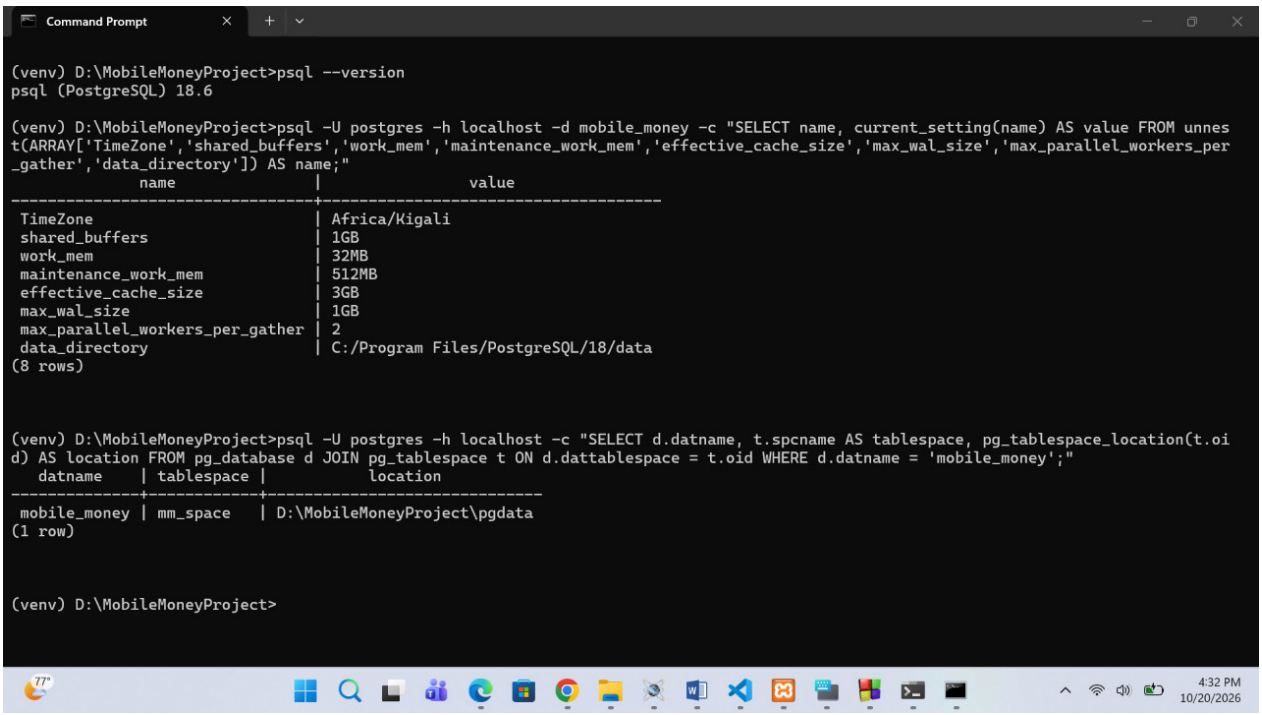


Figure 1.1-2: PostgreSQL 18.6: time zone Africa/Kigali, memory settings, and the tablespace mm_space at D:\MobileMoneyProject\pgdata

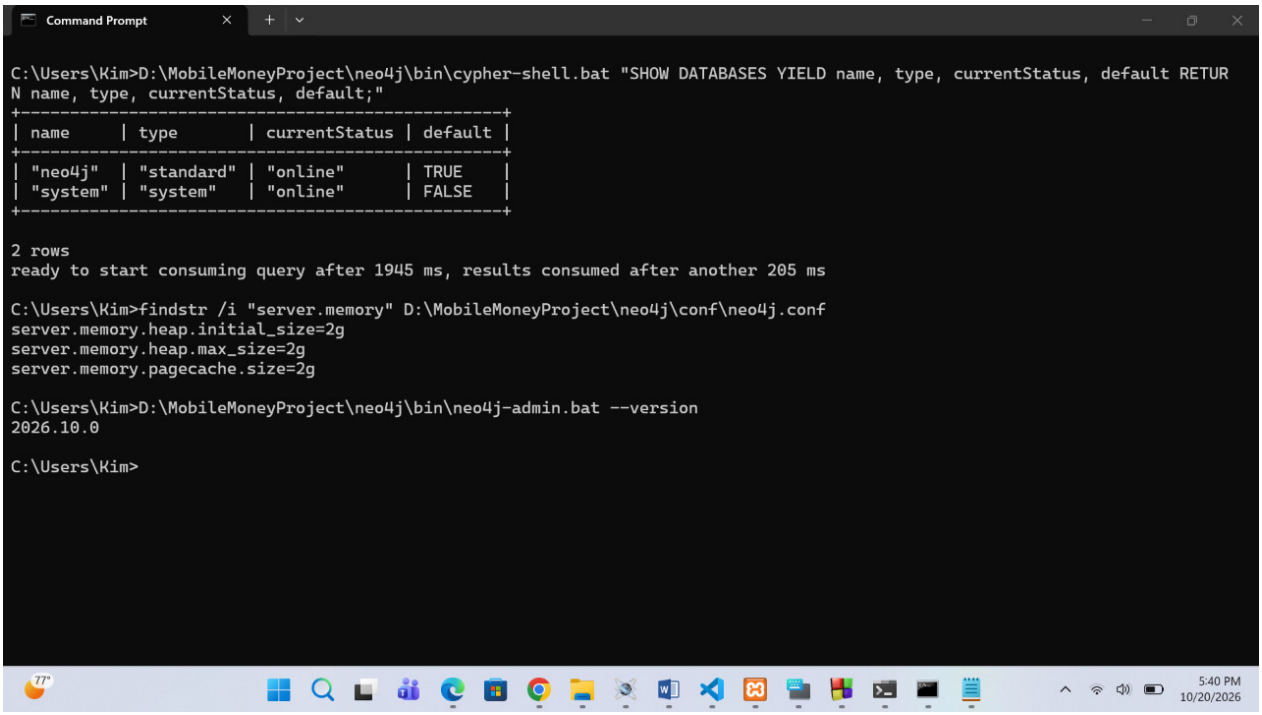


Figure 1.1-3: Neo4j: default database neo4j is online, memory settings of 2 GB heap and 2 GB page cache, and version 2026.10.0

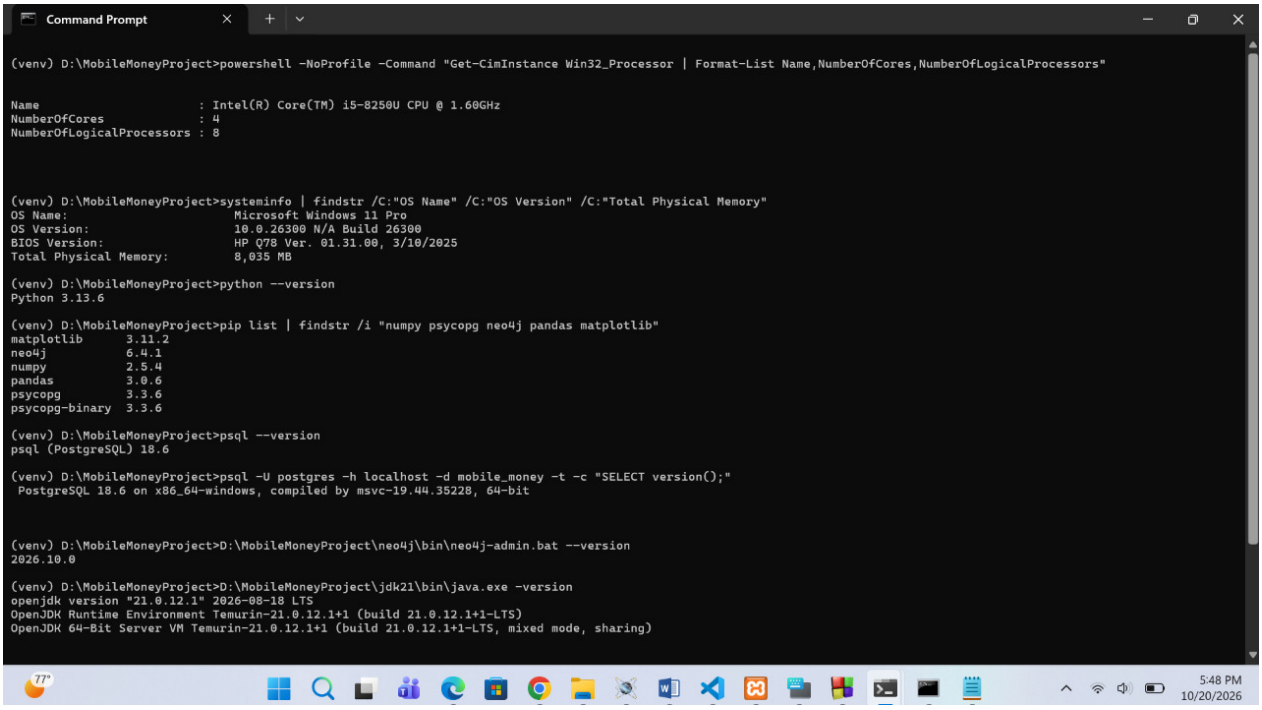


Figure 1.1-4: CPU (4 cores, 8 logical processors), Windows 11 Pro build 26300, 8,035 MB RAM, Python and package versions, PostgreSQL 18.6, Neo4j 2026.10.0 and Java 21.0.12.1

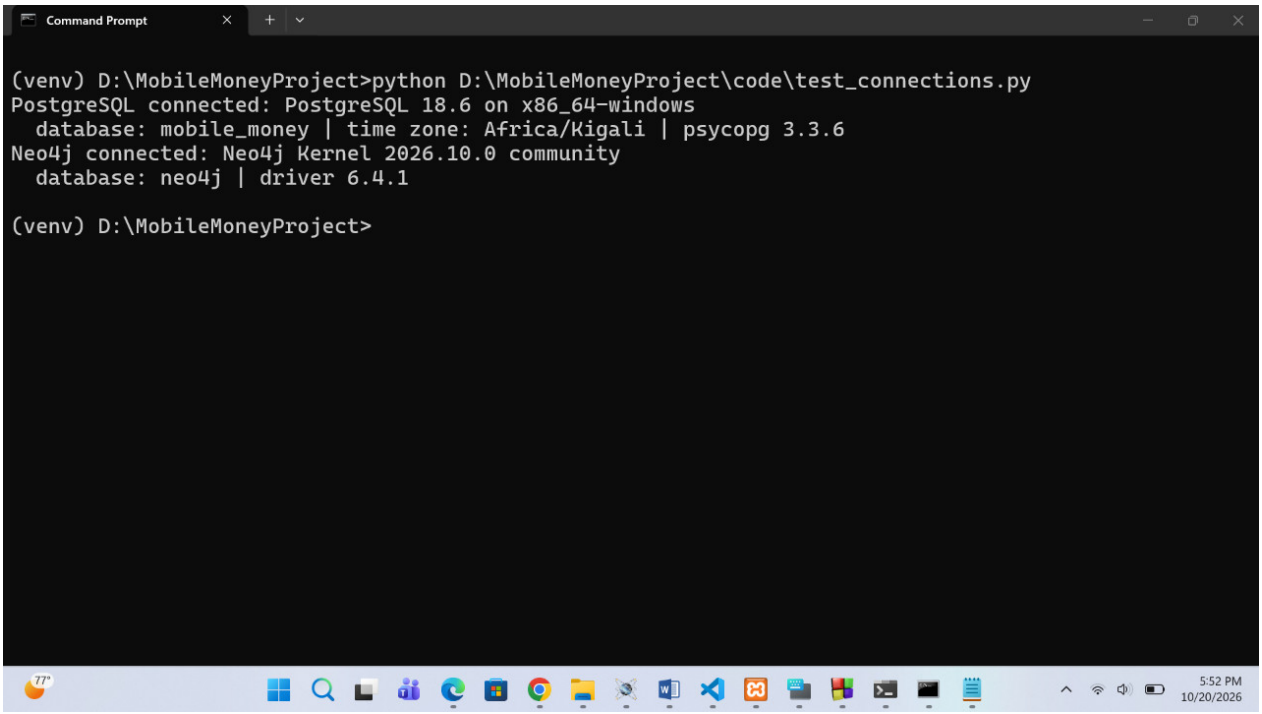


Figure 1.1-5: Successful Python connections to PostgreSQL 18.6 (mobile_money, Africa/Kigali, psycopg 3.3.6) and Neo4j 2026.10.0 community (database neo4j, driver 6.4.1)

1.2 Build capture.py

`capture.py` imports the generator as `g`, reads `g.dataset_info()` and `g.FIELDS`, and previews ten records of every stream with `itertools.islice`. It then writes each of the five streams to its own CSV file progressively with `csv.DictWriter`, so that no stream is ever held in memory. For every stream it counts the rows consumed and written and compares them with the expected table counts. For transfers it also accumulates the count, the earliest and latest `occurred_at` and the sum of `amount_rwf`, logs every million rows, and saves the audit in `results\capture_audit.json`. Fresh iterator calls restart a stream, so the preview does not affect the capture.

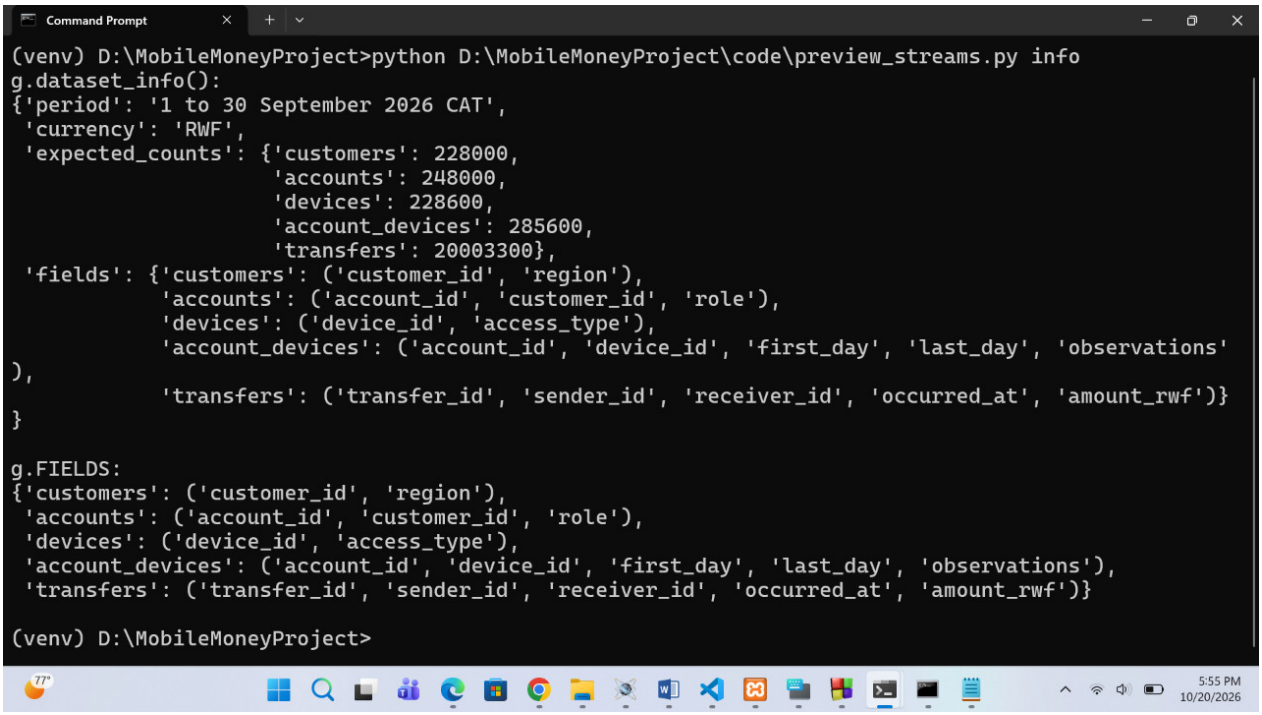


Figure 1.2-1: g.dataset_info() (period, currency, expected counts, fields) and g.FIELDS

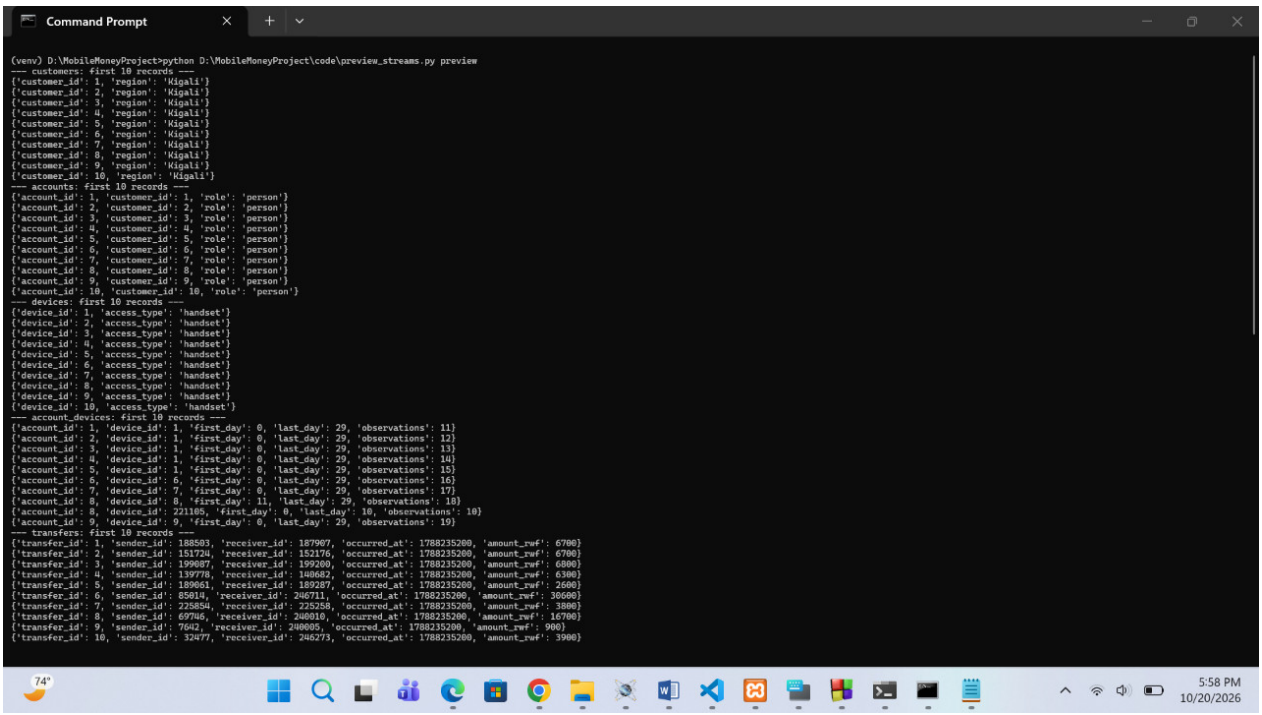


Figure 1.2-2: The first ten records of each of the five streams, obtained with itertools.islice

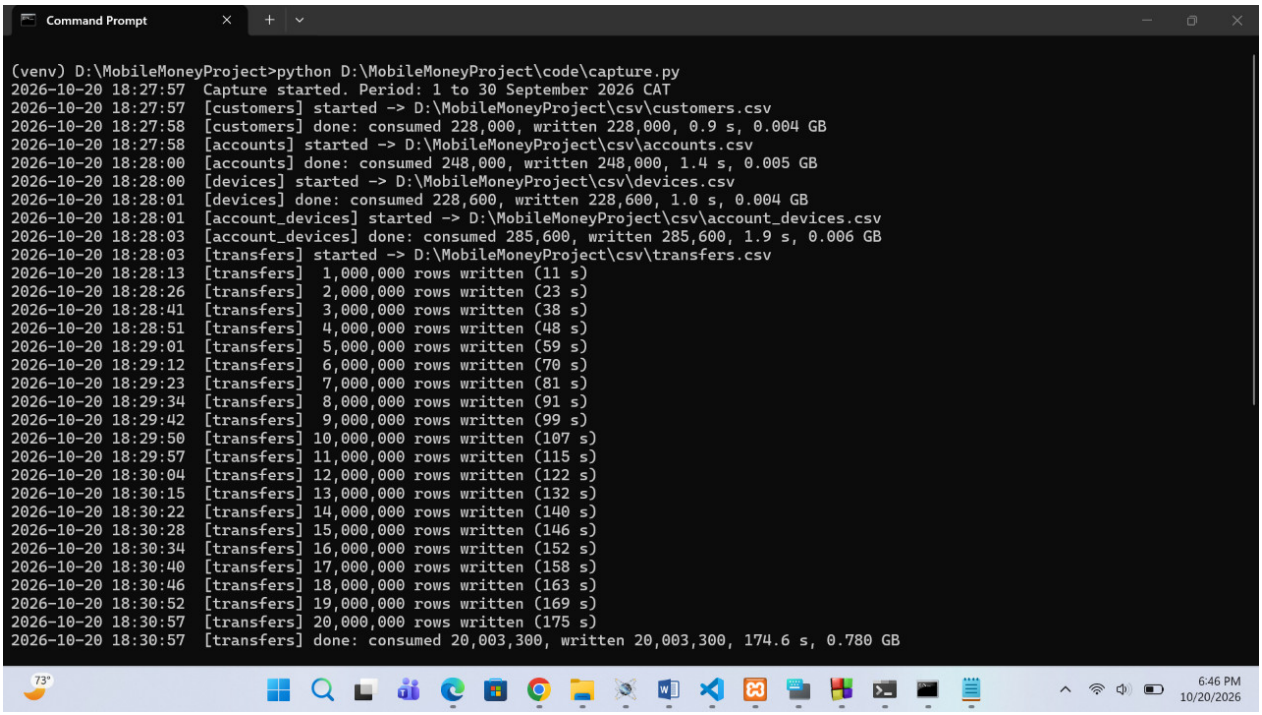


Figure 1.2-3: Capture progress: each small stream finishes in 0.9 to 1.9 s and transfers are logged every million rows up to 20,003,300 rows in 174.6 s

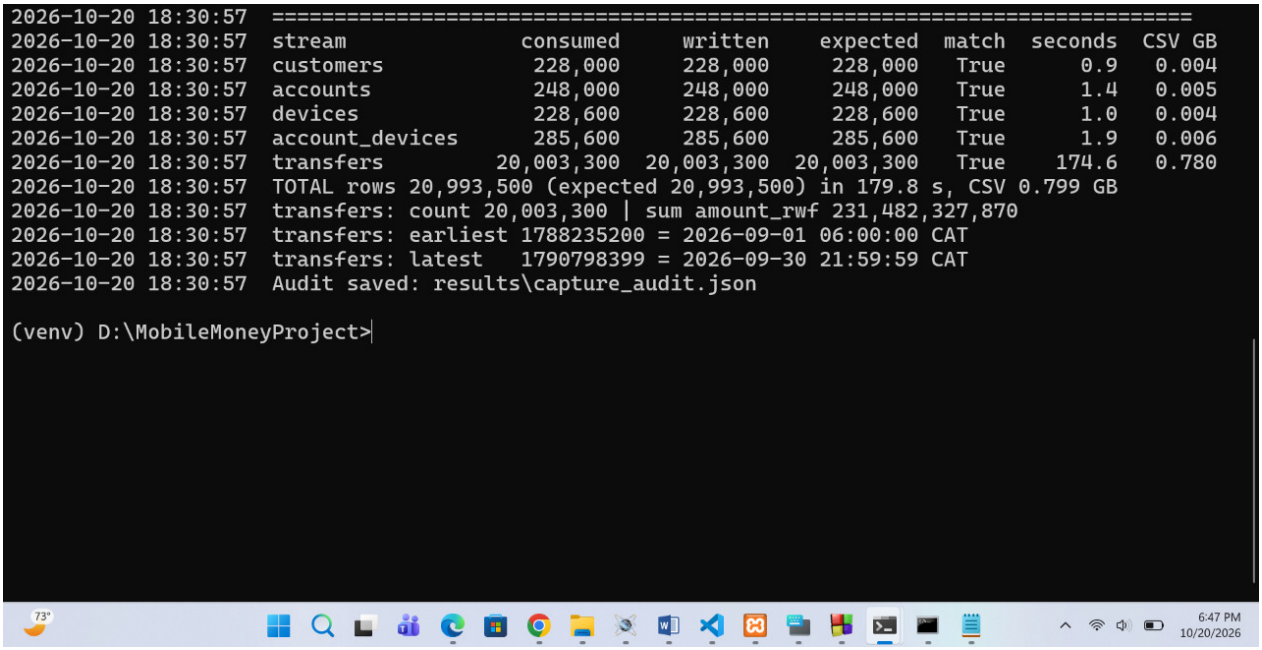


Figure 1.2-4: Five-stream totals (consumed, written and expected counts all match) and the transfer audit totals

Capture results

Stream	Consumed	Written	Expected	Match	Seconds	CSV bytes
customers	228,000	228,000	228,000	True	0.9	3,581,915
accounts	248,000	248,000	248,000	True	1.4	5,212,713
devices	228,600	228,600	228,600	True	1.0	3,554,518
account_devices	285,600	285,600	285,600	True	1.9	6,284,489
transfers	20,003,300	20,003,300	20,003,300	True	174.6	780,197,845
Total	**20,993,500**	**20,993,500**	**20,993,500**	**True**	**179.8**	**798,831,480**

Capture took **179.8 seconds** for **20,993,500 records**, and the five CSV files occupy 798,831,480 bytes (**0.799 GB**, using 1 GB = 1,000,000,000 bytes). The transfer audit recorded **20,003,300 transfers** with a total of **231,482,327,870 RWF**, an earliest `occurred_at` of 1788235200 (2026-09-01 06:00:00 CAT) and a latest of 1790798399 (2026-09-30 21:59:59 CAT). These audit totals are the reference against which the PostgreSQL load (Section 2.2) and the Neo4j import (Section 3.2) are verified.

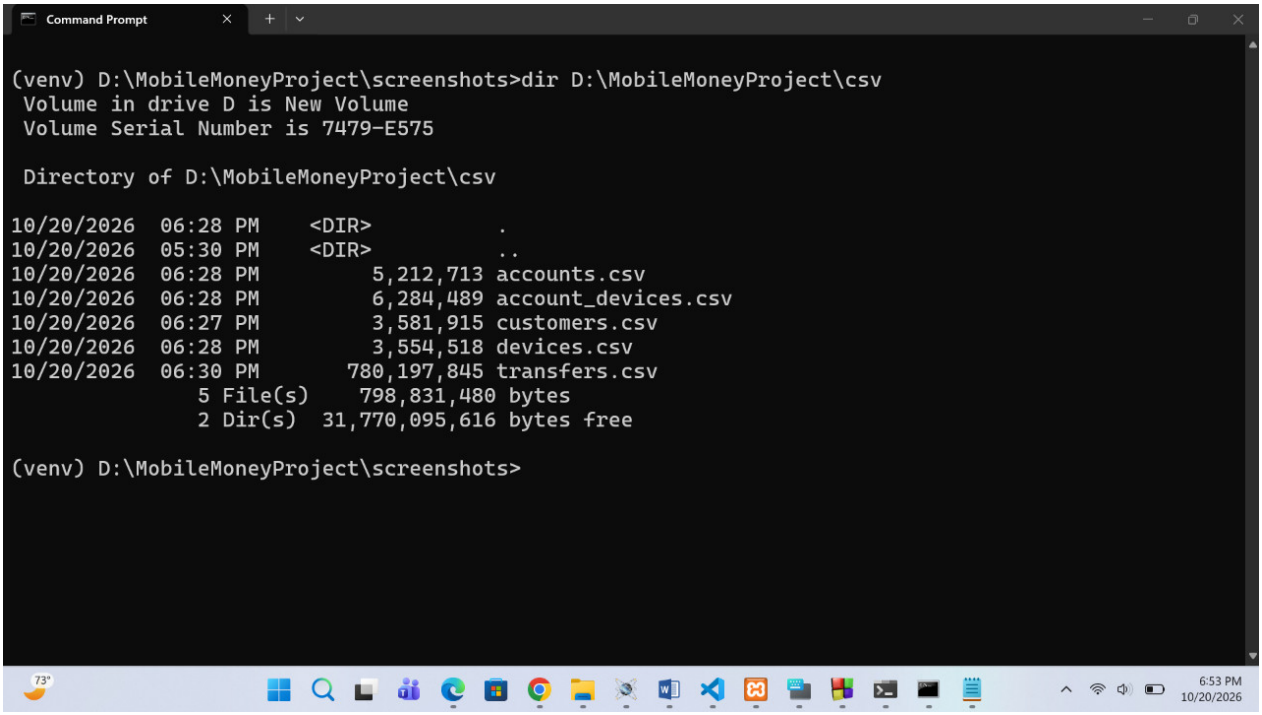


Figure 1.2-5: Sizes of the five source CSV files on drive D: (798,831,480 bytes in total)

2 Build PostgreSQL

2.1 Define the five tables

The captured records are represented as five tables. Entity IDs, amounts and observation counts are `INTEGER`; `transfer_id` and `occurred_at` are `BIGINT`; the day numbers are `SMALLINT`; categories are `TEXT`. Every column is `NOT NULL`. Each entity or event ID is the primary key and `(account_id, device_id)` is the key of `account_devices`. Five foreign keys protect ownership (`accounts` to `customers`), device use (`account_devices` to `accounts` and to `devices`) and both transfer endpoints (`sender_id` and `receiver_id` to `accounts`). Three checks require positive amounts and observations and `0 <= first_day <= last_day <= 29`.

Field meanings and units

Table	Field	Type	Meaning and unit
customers	customer_id	INTEGER, PK	Customer identifier
customers	region	TEXT	Region of the customer; used to group Q4 results by owner region
accounts	account_id	INTEGER, PK	Wallet identifier
accounts	customer_id	INTEGER, FK	Owner of the wallet (a customer holds one or more accounts)
accounts	role	TEXT	Account role: person, merchant or agent
devices	device_id	INTEGER, PK	Device identifier
devices	access_type	TEXT	Kind of device, such as a handset or a public terminal
account_devices	account_id, device_id	INTEGER, composite PK, FKs	An observation of an account using a device (it does not show which device started a payment)
account_devices	first_day, last_day	SMALLINT	Inclusive day numbers 0 to 29 of the observed interval; day 0 is 1 September 2026
account_devices	observations	INTEGER	Number of observations inside the interval (positive count)
transfers	transfer_id	BIGINT, PK	Identifier of one dated payment
transfers	sender_id, receiver_id	INTEGER, FKs	Paying and receiving account
transfers	occurred_at	BIGINT	Time of the payment in Unix seconds (UTC); converted to CAT, UTC+2, for reports
transfers	amount_rwf	INTEGER	Amount in whole Rwandan francs (RWF), always positive

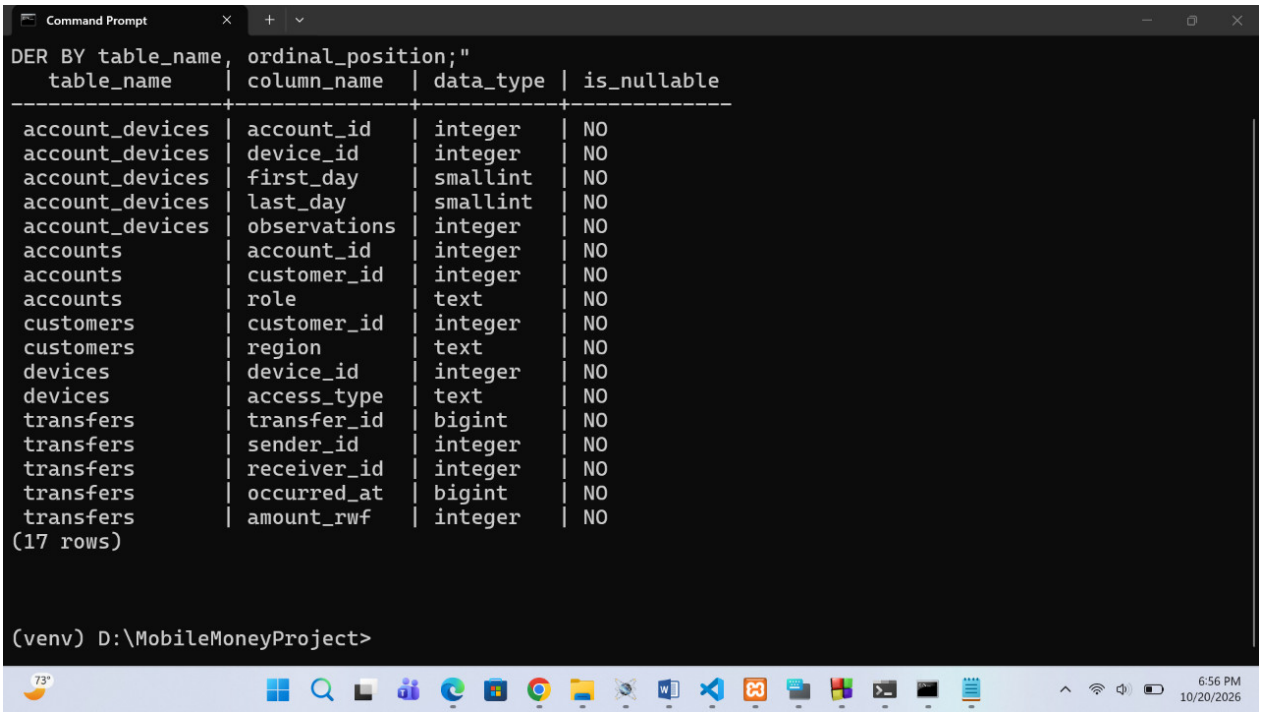


Figure 2.1-1: Column names and types of the five tables: 17 columns, all NOT NULL

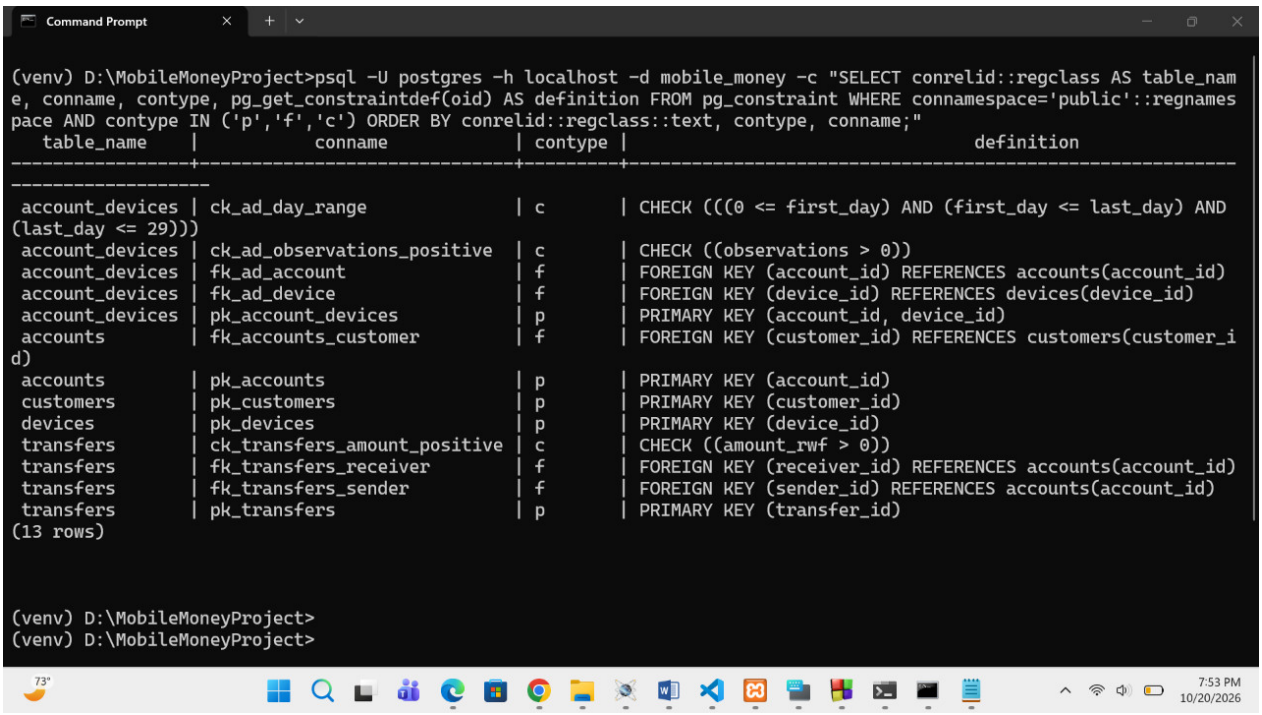


Figure 2.1-2: Constraints: 5 primary keys, 5 foreign keys and 3 checks (13 in total)

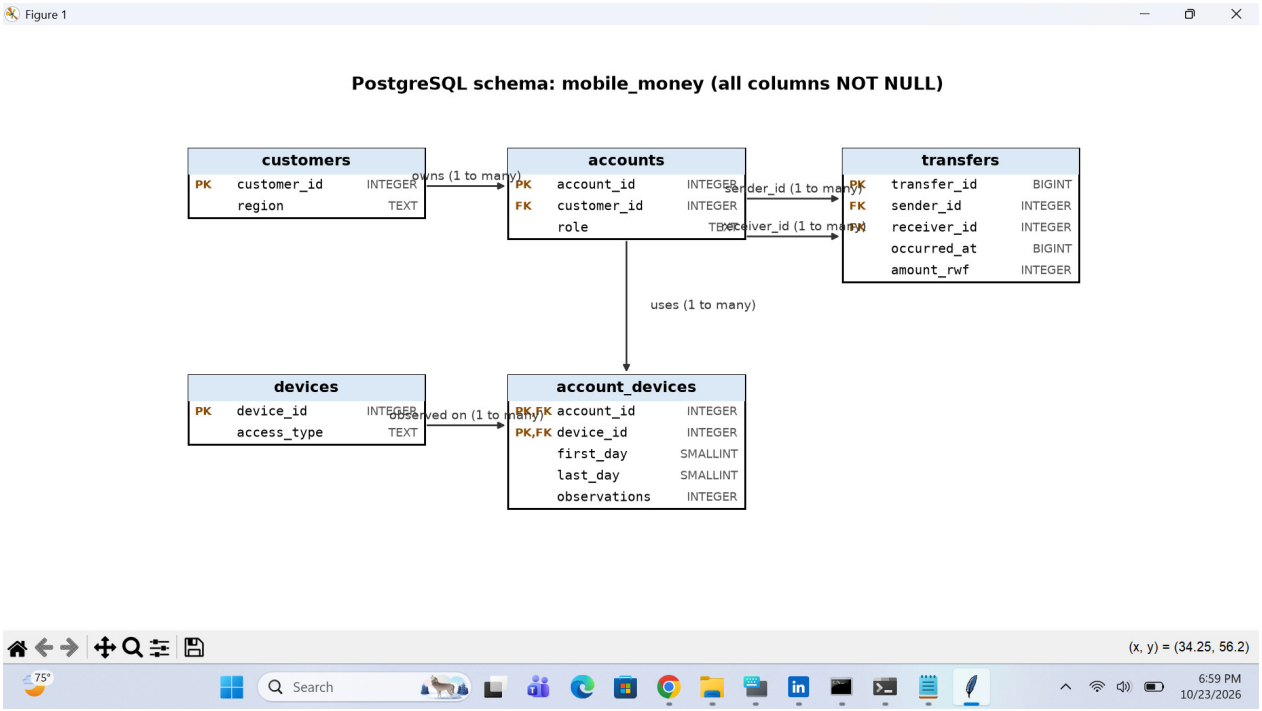


Figure 2.1-3: PostgreSQL schema diagram: customers own accounts; accounts use devices through account_devices and send and receive transfers

2.2 Bulk-load and check

The five CSV files were loaded with `\copy` in dependency order: customers, devices, accounts, account_devices and finally transfers. Loading time and the index and constraint setup time were recorded separately. After the load, `post_load.sql` created the two foreign keys on `transfers`, the five required indexes and ran `ANALYZE`. The two covering indexes on `transfers` are `(sender_id, occurred_at) INCLUDE (receiver_id, amount_rwf, transfer_id)` and `(receiver_id, occurred_at) INCLUDE (sender_id, amount_rwf, transfer_id)`; a BRIN index covers `occurred_at`.

Loading and setup times

Step	Rows	Seconds
\copy customers	228,000	0.929
\copy devices	228,600	0.819
\copy accounts	248,000	4.937
\copy account_devices	285,600	11.067
\copy transfers	20,003,300	102.857
Loading total	**20,993,500**	**120.6**
Foreign keys on transfers (sender and receiver)		34.068
Index on accounts (customer_id, account_id)		0.559
Index on account_devices (device_id, account_id)		0.479
Covering index on transfers (sender_id, occurred_at)		44.845
Covering index on transfers (receiver_id, occurred_at)		48.761
BRIN index on transfers (occurred_at)		4.090
ANALYZE		3.134
Constraint and index setup total		**135.9**

Loading took **120.6 s** and constraint, index and statistics setup took a further **135.9 s**. The transfer foreign keys and all indexes were built after the bulk load, so that the load itself did not maintain them row by row. All five row counts match capture.py, the transfer count, amount sum, earliest and latest time match the capture audit, and the roles are **240,000 person, 6,000 merchant and 2,000 agent** accounts. All 13 constraints are validated.

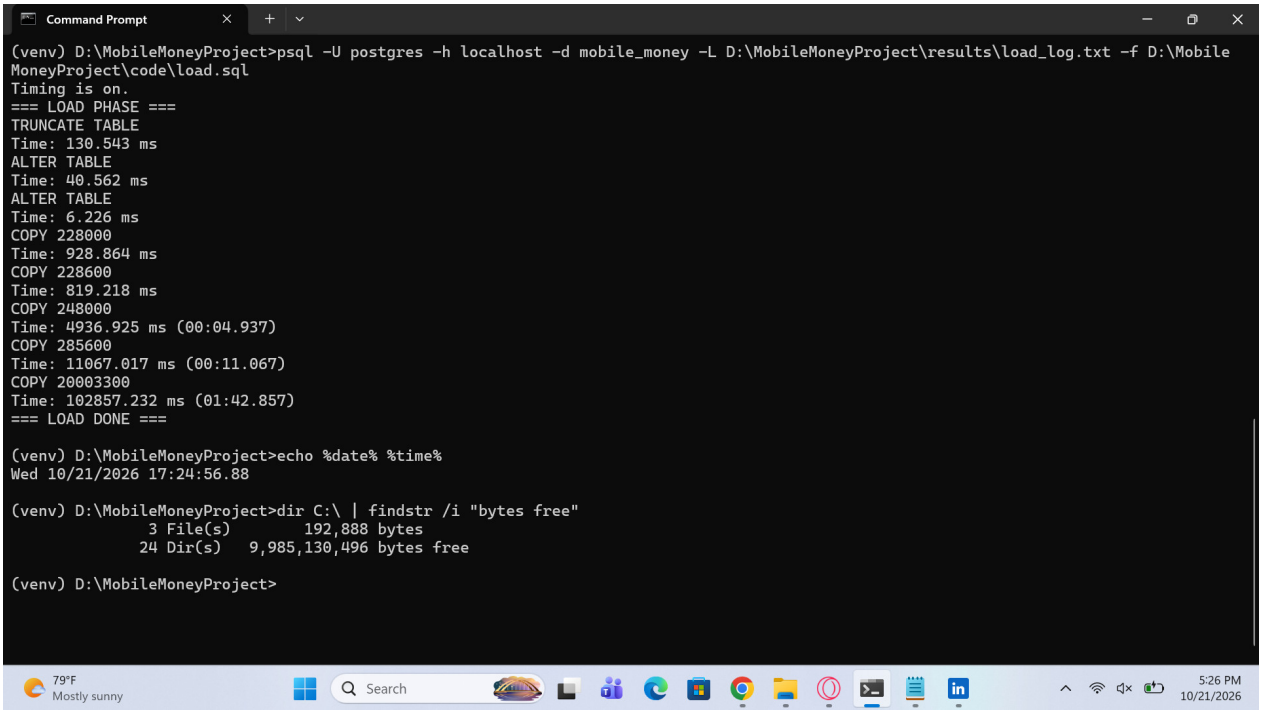


Figure 2.2-1: Bulk load with \copy: row counts and time of each file (load.sql), and free space on drive C:

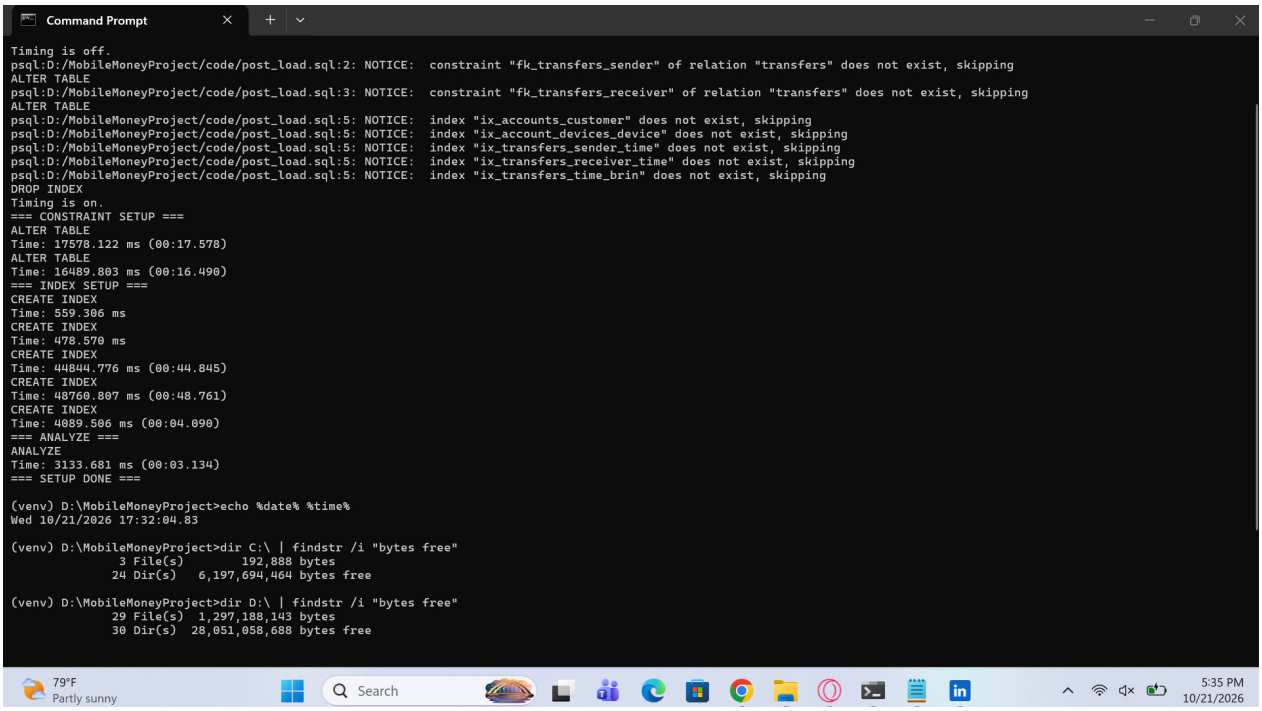


Figure 2.2-2: Constraint setup, creation of the five indexes and ANALYZE with their times (post_load.sql), and free space on drives C: and D:

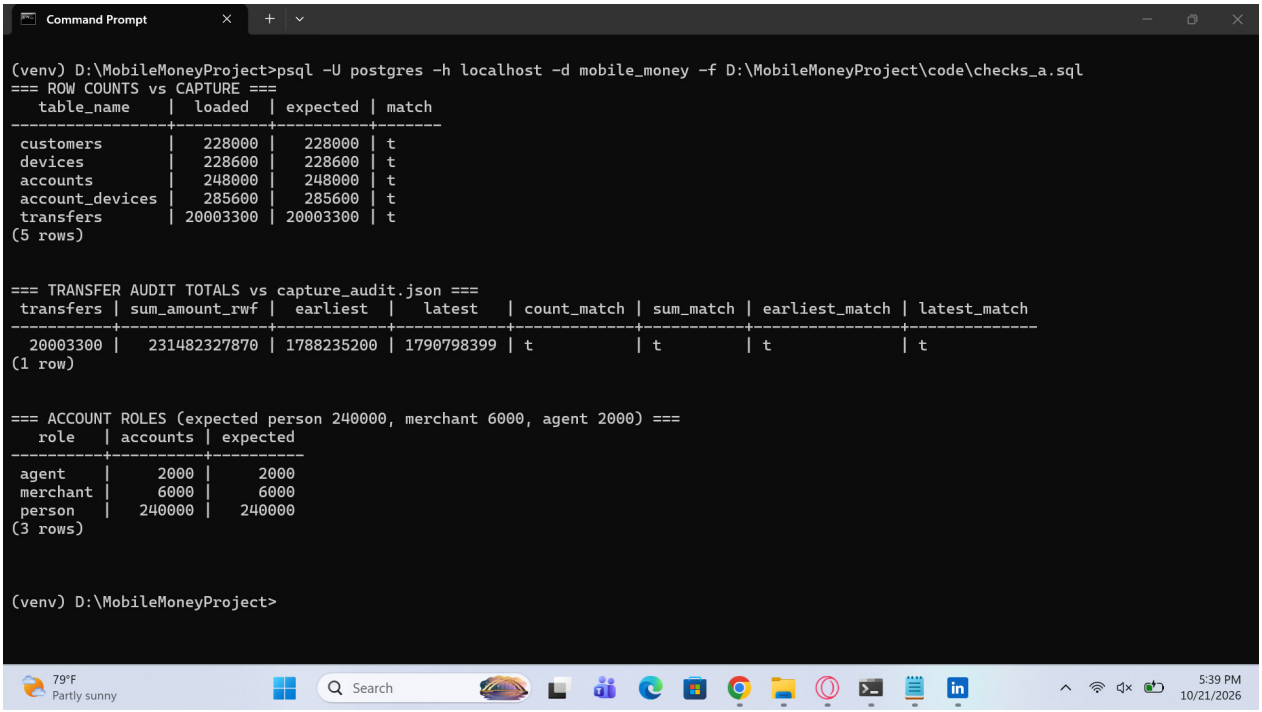


Figure 2.2-3: Row counts and transfer audit totals match the capture; account roles are 240,000 person, 6,000 merchant and 2,000 agent

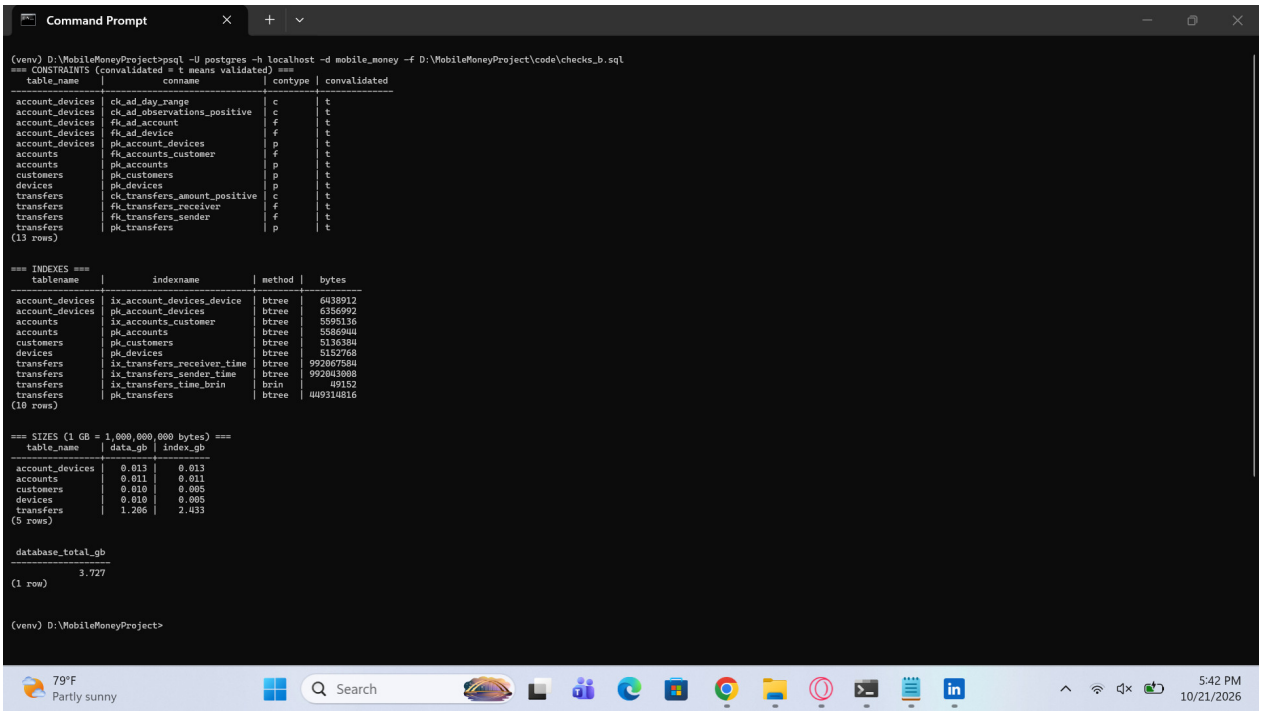


Figure 2.2-4: All 13 constraints validated, the ten indexes with their sizes, table data and index sizes in GB, and the database total of 3.727 GB

Data and index sizes (1 GB = 1,000,000,000 bytes)

Table	Data (GB)	Indexes (GB)
customers	0.010	0.005
devices	0.010	0.005
accounts	0.011	0.011
account_devices	0.013	0.013
transfers	1.206	2.433
Database total	**3.727**	

The `transfers` indexes (2.433 GB) are twice the size of the table data (1.206 GB) because each of the two covering indexes (about 0.992 GB each) repeats three further columns beside its search keys, and the primary key adds 0.449 GB. The BRIN index needs only 49,152 bytes, since it stores one summary per block range of the physical order. These sizes are the storage cost of fast lookups by sender and receiver, and they are compared with the Neo4j store in Section 8.2.

3 Build Neo4j

3.1 Map the same facts

The same records are represented as a graph with three node labels and three relationship types. `Customer(customer_id, region)`, `Account(account_id, role)` and `Device(device_id, access_type)` are nodes with integer IDs. Ownership becomes `(Customer)-[:OWNS]->(Account)` and each device observation becomes `(Account)-[:USED_DEVICE]->(Device)` with its three observation properties. **Every transfer is a separate** `(Account)-[:TRANSFER]->(Account)` relationship with `transfer_id`, `occurred_at` and `amount_rwf`, so repeated payments between the same two accounts are all preserved. The loaded graph is expected to have 704,600 nodes (228,000 + 248,000 + 228,600) and 20,536,900 relationships (248,000 + 285,600 + 20,003,300).

Mapping of every source field to the graph

Source field	Graph element	Type and notes
customers.customer_id	Customer.customer_id	int; unique business ID
customers.region	Customer.region	string
accounts.account_id	Account.account_id	int; unique business ID
accounts.role	Account.role	string: person, merchant or agent
accounts.customer_id	OWNS relationship (Customer to Account)	The foreign key becomes the relationship; it is not stored on the Account node
devices.device_id	Device.device_id	int; unique business ID
devices.access_type	Device.access_type	string
account_devices.account_id, device_id	USED_DEVICE relationship (Account to Device)	The composite key becomes the two end nodes
account_devices.first_day, last_day, observations	USED_DEVICE properties	int; inclusive day numbers 0 to 29 and observation count
transfers.sender_id, receiver_id	Start and end Account of a TRANSFER	Direction is sender to receiver
transfers.transfer_id	TRANSFER.transfer_id	long; unique per relationship
transfers.occurred_at	TRANSFER.occurred_at	long; Unix seconds
transfers.amount_rwf	TRANSFER.amount_rwf	int; RWF

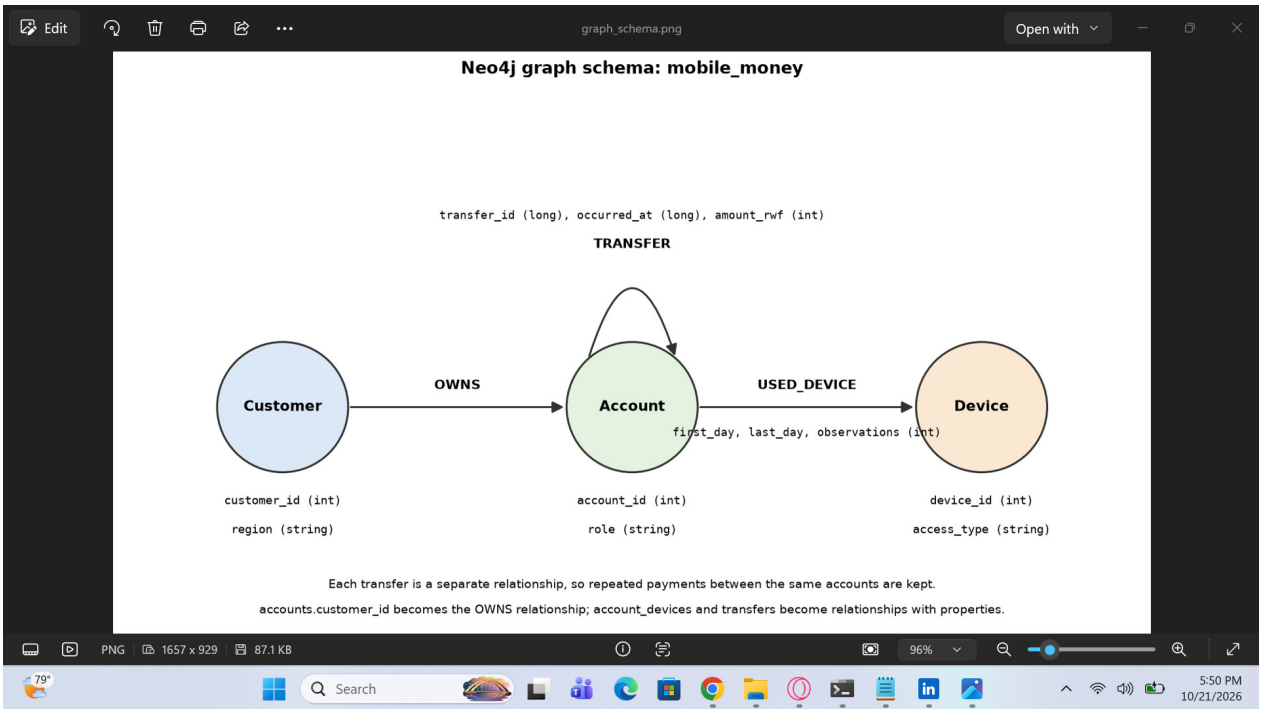


Figure 3.1-1: Neo4j graph schema: node labels with their properties, the OWNS, USED_DEVICE and TRANSFER relationships, and the field mapping notes

3.2 Prepare and import the CSVs

`prepare_import.py` leaves the source CSV files unchanged and writes temporary import copies in `import_tmp` with Neo4j typed headers. Customer, Account and Device IDs form separate ID groups (`ID(Customer)`, `ID(Account)`, `ID(Device)`), numeric properties are typed `:int` or `:long`, and `owns.csv` is derived from the accounts in `customer_id, account_id` order. The ignored column in the Account header stays in the file, while its ownership fact goes to `owns.csv`. Every copy has the expected row count and the check **Source CSVs unchanged: True** passed. Neo4j was stopped during the import (nothing was listening on port 7687).

Typed headers and import copies

File	Typed header	Rows	Bytes	Sec
customers.csv	`customer_id:ID(Customer), region:string`	228,000	3,581,935	0.1
accounts.csv	`account_id:ID(Account),:IGNORE, role:string`	248,000	5,212,728	0.1
devices.csv	`device_id:ID(Device),access_ type:string`	228,600	3,554,536	0.1
account_ devices.csv	`:START_ID(Account),:END_ ID(Device),first_day:int,last_ day:int,observations:int`	285,600	6,284,515	0.1
transfers.csv	`transfer_id:long,:START_ ID(Account),:END_ID(Account), occurred_at:long,amount_rwf:int`	20,003,300	780,197,873	8.0
owns.csv	`:START_ID(Customer),:END_ ID(Account)`	248,000	3,466,722	2.1

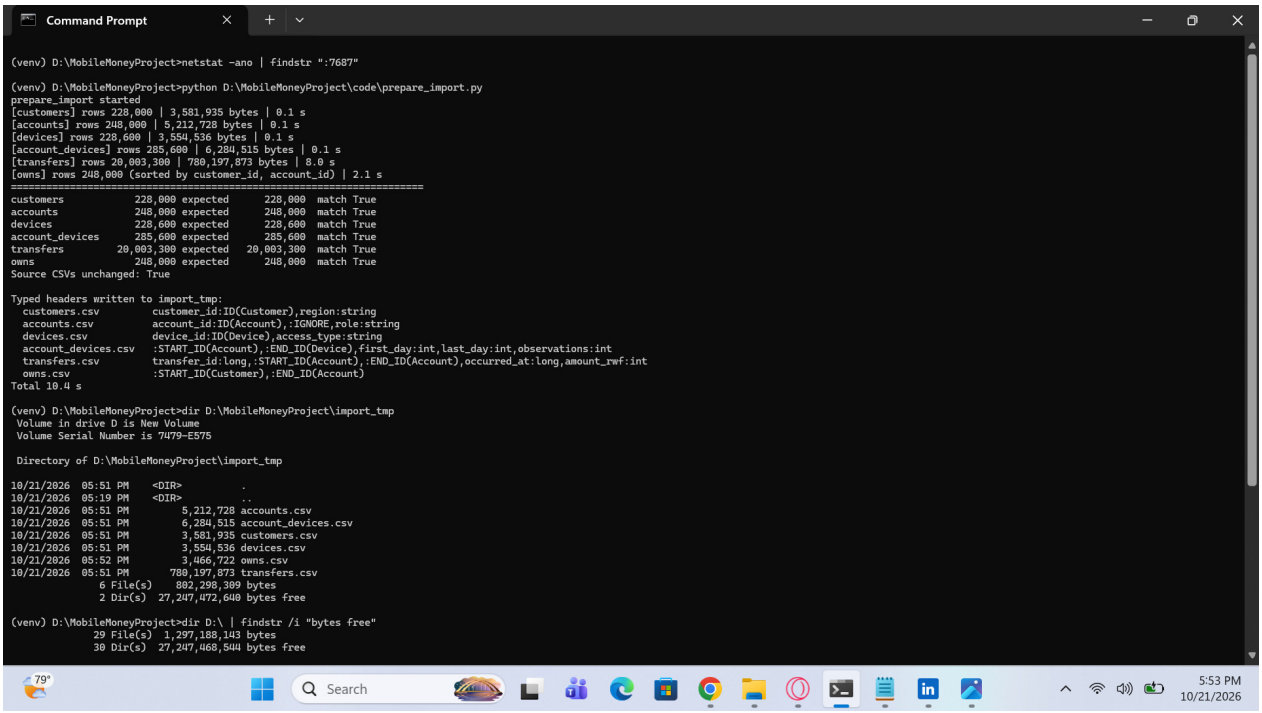


Figure 3.2-1: prepare_import.py: row counts match, source CSVs unchanged, typed headers written, and the six import copies in import_tmp (10.4 s in total)

The full import ran with `--id-type=integer`, three `--nodes=Label=file.csv` options (Customer, Account, Device) and three `--relationships=TYPE=file.csv` options (OWNS from `owns.csv`, USED_DEVICE from `account_devices.csv`, TRANSFER from `transfers.csv`), into the empty `neo4j` database. It finished in **1 min 42 s**, imported **704,600 nodes, 20,536,900 relationships and 62,275,900 properties**, and peaked at 1.033 GiB of memory. The property count reconciles exactly: 2 x 228,000 (customers) + 2 x 248,000 (accounts) + 2 x 228,600 (devices) + 3 x 285,600 (USED_DEVICE) + 3 x 20,003,300 (TRANSFER) = 62,275,900. The store was 1.607 GB after the import.

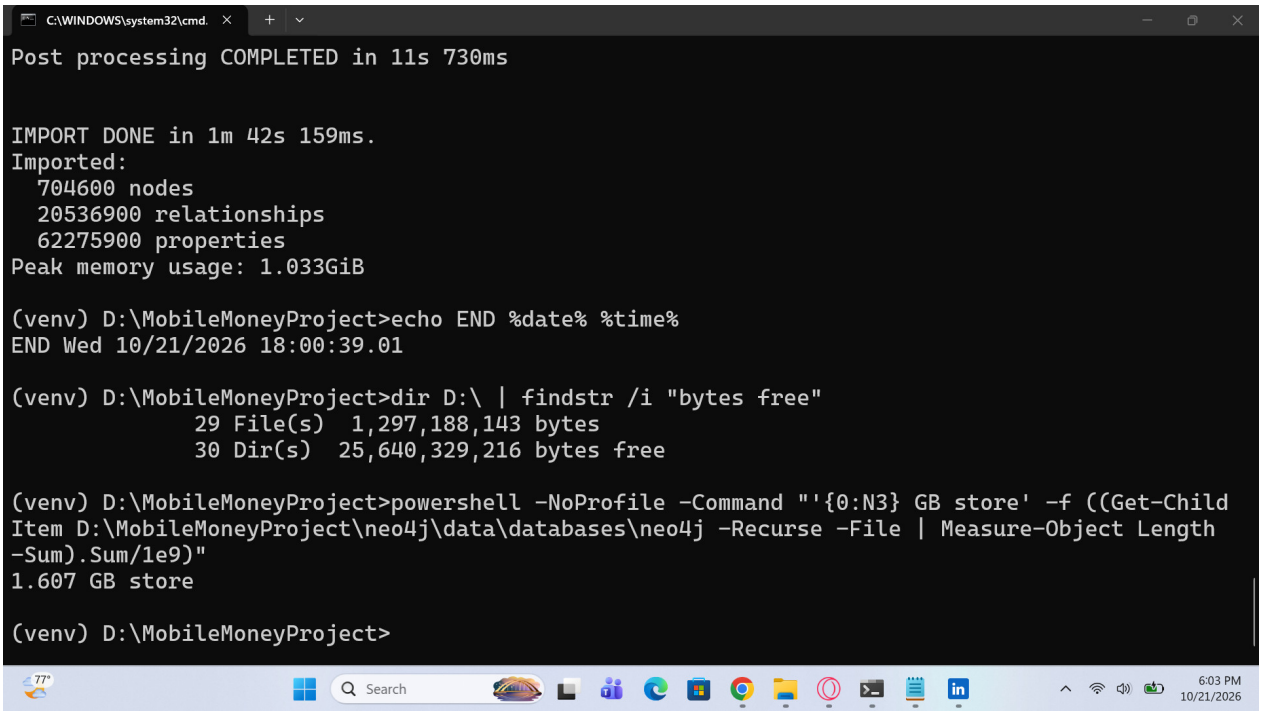


Figure 3.2-2: Import completed in 1 min 42 s: 704,600 nodes, 20,536,900 relationships and 62,275,900 properties; store size 1.607 GB

The import command line itself is described in words above and is not shown in a screenshot; Figure 3.2-2 shows the completed import.

After starting Neo4j, `neo4j_setup.cypher` created unique constraints on the three business IDs and a range index on `TRANSFER.occurred_at`, and `db.awaitIndexes(900)` waited until every index was online. The four statements returned after 5.0, 2.3, 1.9 and 0.2 s and the wait took 74.8 s, so setup needed about **84 s** of query time (about 2 minutes on the clock including each `cypher-shell` start-up). All six indexes are `ONLINE` at 100 %: the three constraint-backed range indexes, the `transfer_occurred_at` range index, and two lookup indexes that Neo4j creates by default.

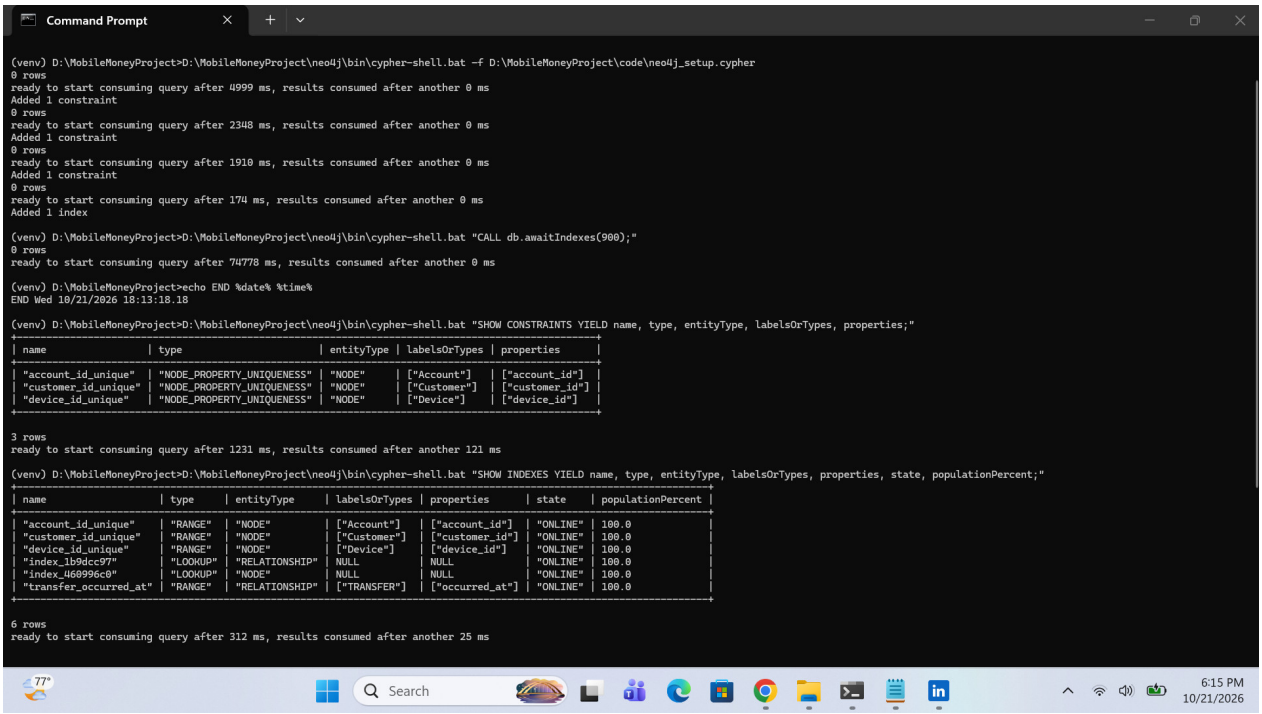


Figure 3.2-3: Setup script and waiting for indexes; three unique constraints on Account, Customer and Device IDs; six indexes online at 100 %, including transfer_occurred_at

The store grew from 1.607 GB to **2.049 GB** once the constraints and the index existed (+0.442 GB, mostly index data), and the transaction logs hold only **10,626 bytes** (0.000011 GB), because the bulk import writes the store directly instead of through the transaction log. Sizes use 1 GB = 1,000,000,000 bytes.

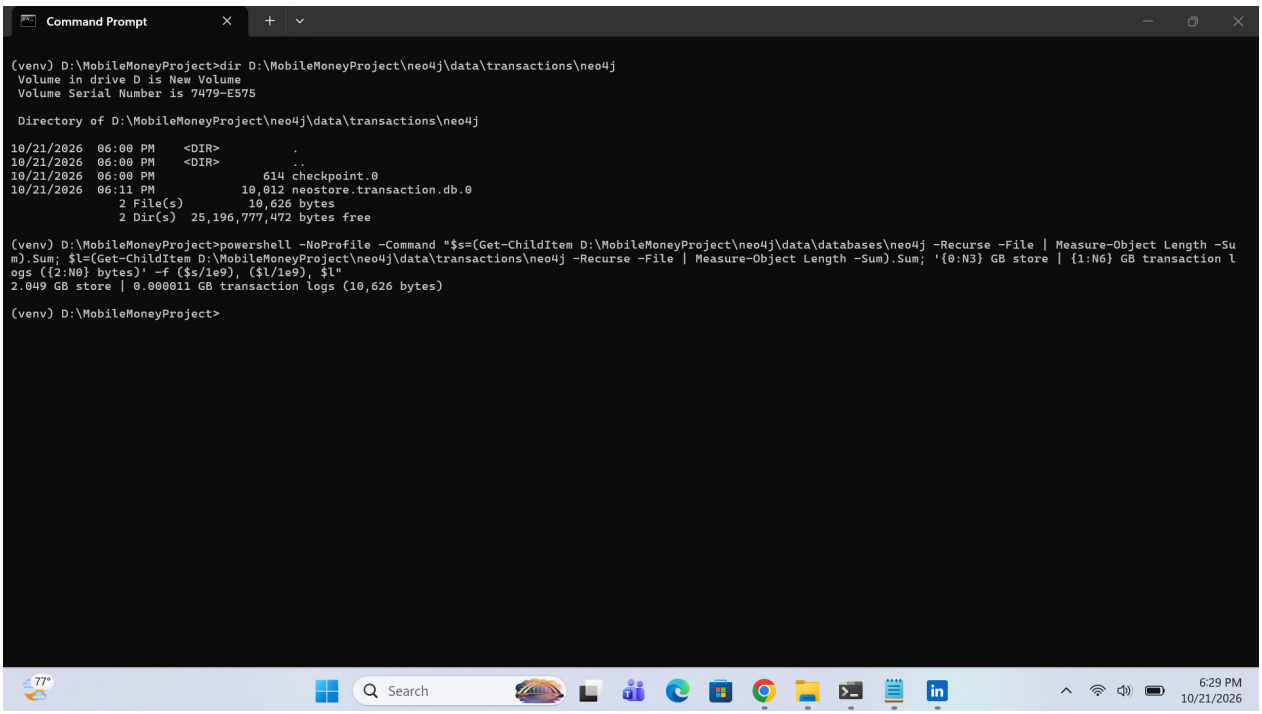


Figure 3.2-4: Neo4j store 2.049 GB and transaction logs 10,626 bytes after constraints and indexes

Graph verification against the source and the capture audit

Check	Neo4j result	Expected
Customer, Account and Device nodes	228,000 / 248,000 / 228,600	Same (all TRUE)
Total nodes	704,600	704,600
OWNS / USED_DEVICE / TRANSFER relationships	248,000 / 285,600 / 20,003,300	Same (all TRUE)
Total relationships	20,536,900	20,536,900
Account roles: agent / merchant / person	2,000 / 6,000 / 240,000	Same
Accounts with exactly one owner	248,000 (0 without one owner)	248,000
Transfer count and amount sum	20,003,300; 231,482,327,870 RWF	Capture audit
Earliest and latest occurred_at	1788235200; 1790798399	Capture audit
Distinct transfer_id	20,003,300	Unique per transfer (TRUE)

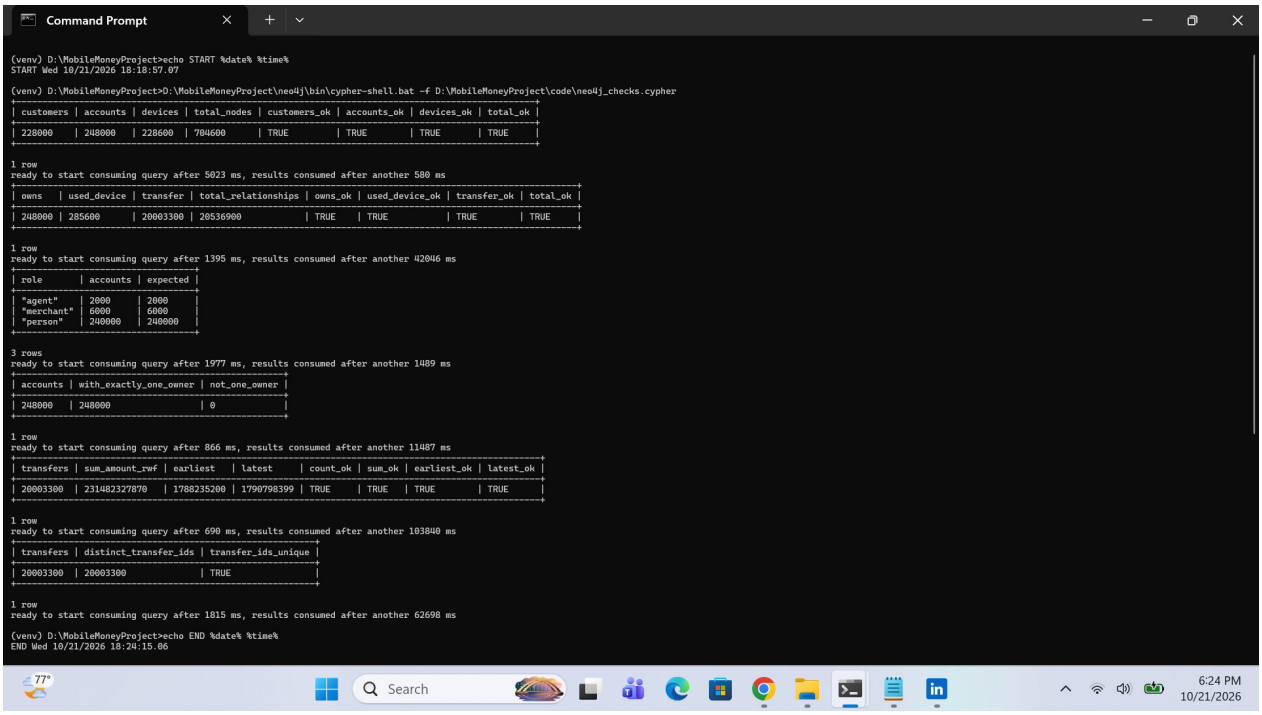


Figure 3.2-5: Neo4j checks: node and relationship counts, roles, one owner per account, audit totals and unique transfer_id, all TRUE

Finally the temporary import copies were deleted. `import_tmp` is empty, and the five source CSV files (798,831,480 bytes) in `csv` are untouched.

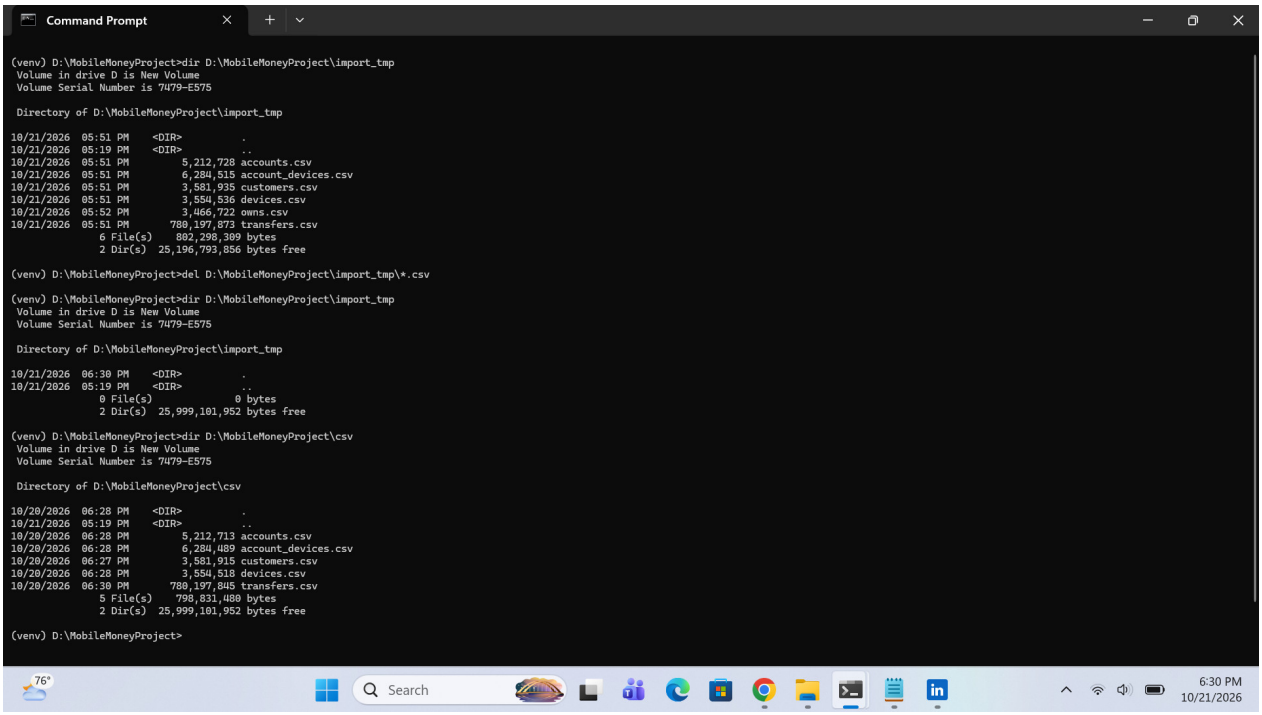


Figure 3.2-6: The six temporary import files deleted (import_tmp empty) while the five source CSVs remain

3.3 Inspect one account in both models

Account **998** was traced in both models to its owner, a device and an outgoing transfer. In SQL the trace joins `accounts`, `customers`, `account_devices`, `devices` and `transfers`; in Cypher it matches the path from the Customer through the Account to its Device and to a TRANSFER relationship. Both return the same two rows: the person account 998, owned by customer 998 in region Kigali, used handset **998** (days 0 to 29, 39 observations) and handset **228001** (days 0 to 29, 16 observations), and sent transfer **113540** of **2,100 RWF** to account 240004 at 1788245009 (2026-09-01 08:43:29 CAT). The transfer repeats on both rows because the account has two devices.

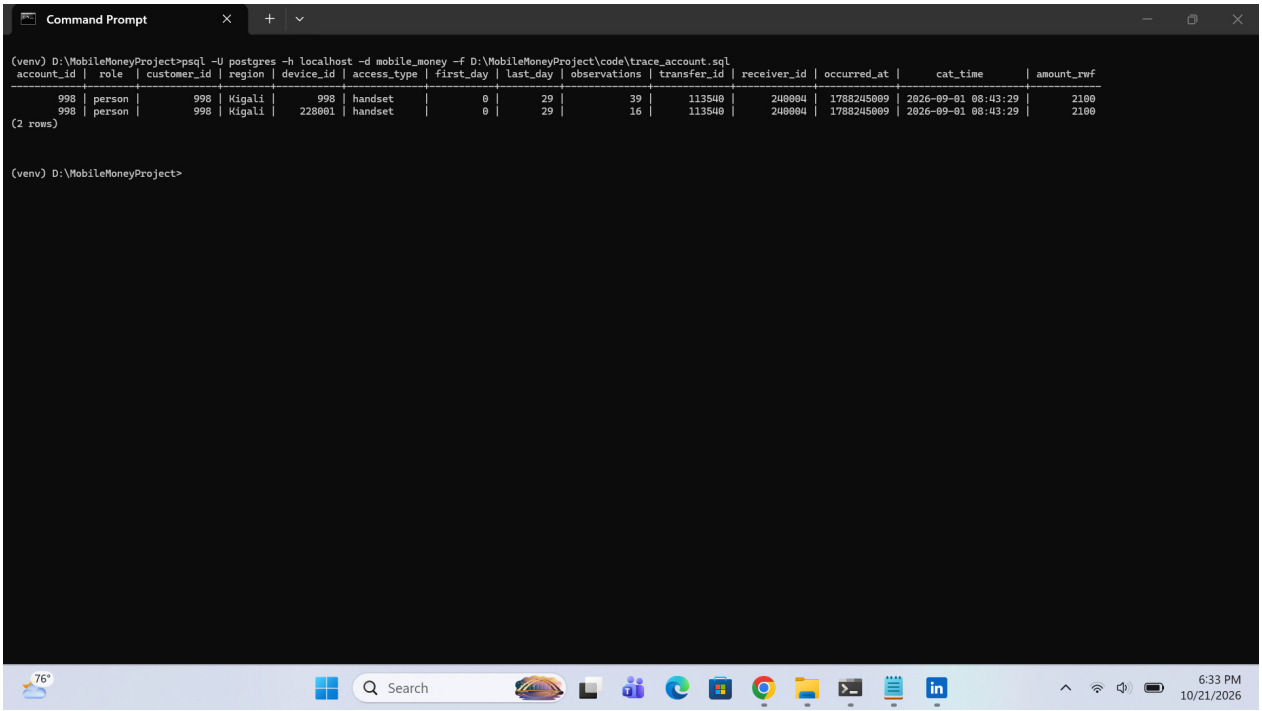


Figure 3.3-1: SQL: account 998 with its owner, its two devices and an outgoing transfer (trace_account.sql)

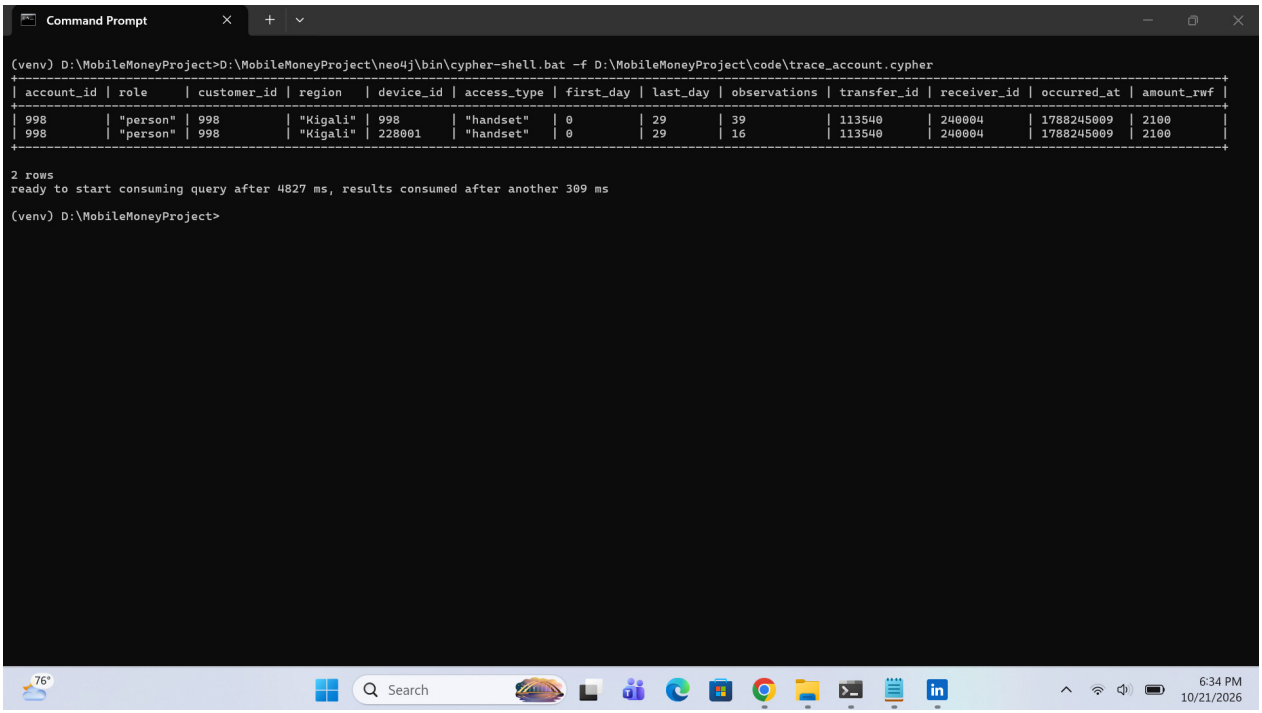


Figure 3.3-2: Cypher: the same two rows for account 998 (trace_account.cypher)

How foreign keys become relationships. `accounts.customer_id` becomes the `OWNS` relationship, the two keys of `account_devices` become the two ends of a `USED_DEVICE` relationship that carries the day interval and the observation count, and `sender_id` and `receiver_id` become the two ends of each `TRANSFER`. SQL finds the matching rows by comparing key values through indexes in every join, while Neo4j follows the stored relationship from one node to the next.

Evidence lost if repeated payments were collapsed into one edge. One edge per pair of accounts could keep only a count and a total. It would lose each `transfer_id`, the exact time of every payment, the amount of each payment and their order. Without them the hourly window of Q1, the daily totals of Q2, the ten-minute gaps and amount ratios of Q5, and the 30-minute return rule of Q6 could not be evaluated, and a payment repeated every few seconds could not be told from one repeated over a month. This is why every transfer is kept as its own relationship.

4 Answer routine business questions

Each question has one parameterised SQL and Cypher pair built on the stored data. Account-specific questions use the six sets below; each review account starts the search and only Q7 uses the destination. Where an hour is specified it runs from 09:45:00 inclusive to 10:45:00 exclusive CAT on the listed date, and the CAT boundaries are converted to epoch seconds in Python.

Set	Review account	Destination	September date
A	998	24755	6
B	147228	186823	13
C	206978	6573	8
D	237998	21755	6
E	144228	183823	13
F	203978	3573	8

4.1 Account activity

returns the number and total value (RWF) of the outgoing transfers of each review account in its hour, to receivers of every role, with zeros when nothing matches. All six pairs match. Each account sent exactly one transfer in its hour: A 200,000, B 1,050,000, C 300,000, D 45,000, E 40,000 and F 30,000 RWF.

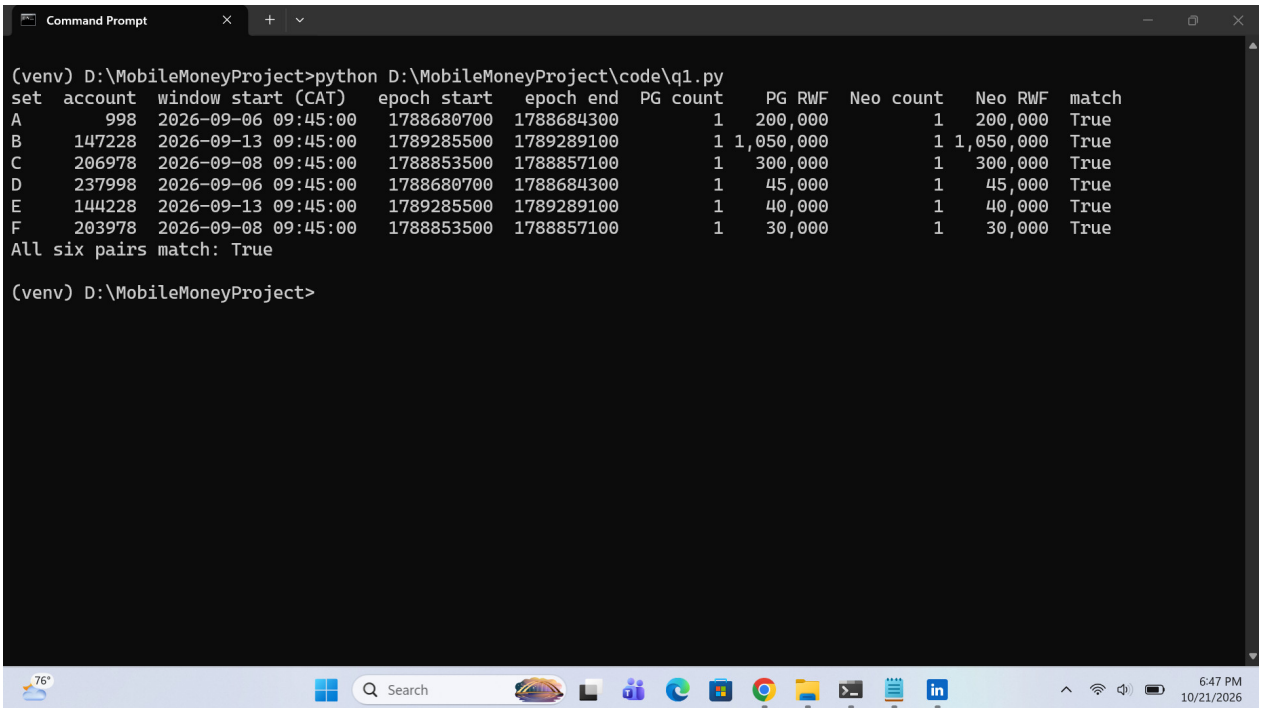


Figure 4.1-1: Q1: the six paired results with window start, epoch boundaries, PostgreSQL and Neo4j counts and totals; all six pairs match

Why outgoing value is not a balance. Q1 adds up only the money that left the account in one hour. A balance also depends on the opening balance, money received, cash-in and cash-out, and fees, none of which Q1 uses. A 1,050,000 RWF payment from account B does not tell how much B holds, whether B was left with anything, or whether the money came from B's own funds.

4.2 Daily activity

Groups the whole month into 30 CAT days and returns the date, the count and the total RWF in date order. Both engines return the same 30 rows. The daily counts add up to **20,003,300** and the daily totals to **231,482,327,870 RWF** in both engines, which reconcile to the capture audit. One run took 30.1 s in PostgreSQL and 341.4 s in Neo4j; this single run is not the benchmark, and the timing method of Section 6 is used for the comparison.

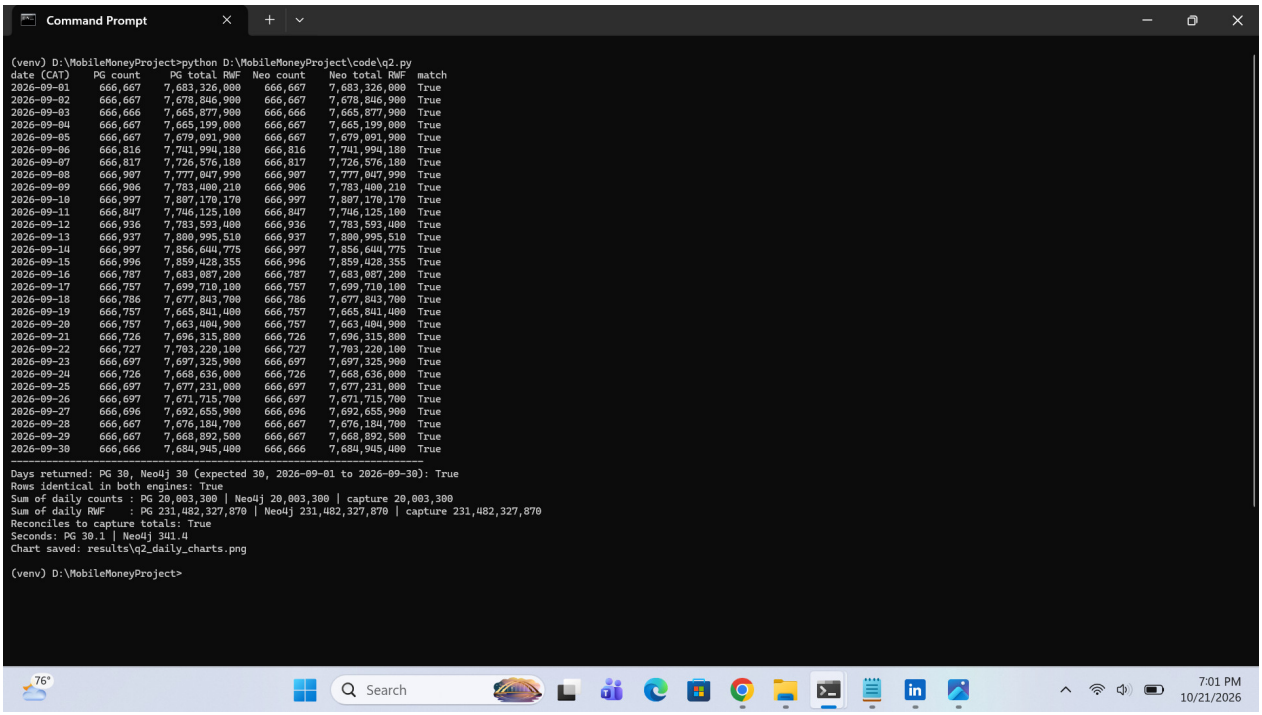


Figure 4.2-1: Q2: 30 daily rows identical in PostgreSQL and Neo4j, and reconciliation of the sums to the capture totals

summary

Measure	Value	Day
Lowest daily count	666,666 transfers	3 and 30 September
Highest daily count	666,997 transfers	10 and 14 September
Lowest daily value	7,663,404,900 RWF	20 September
Highest daily value	7,859,428,355 RWF	15 September
Average value per transfer	11,525 RWF (1 Sep); 11,783 RWF (15 Sep)	up 2.2 %

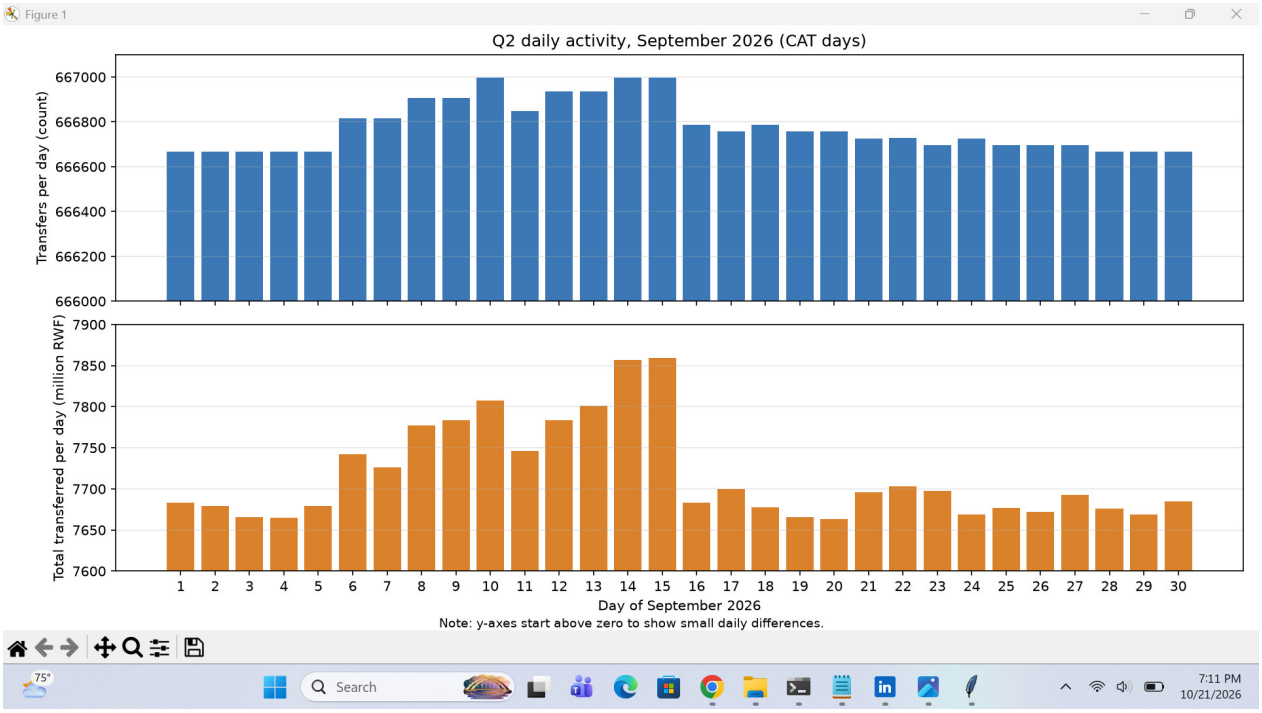


Figure 4.2-2: Daily transfers (count) and total transferred (million RWF) per CAT day. The y-axes start above zero to show small differences

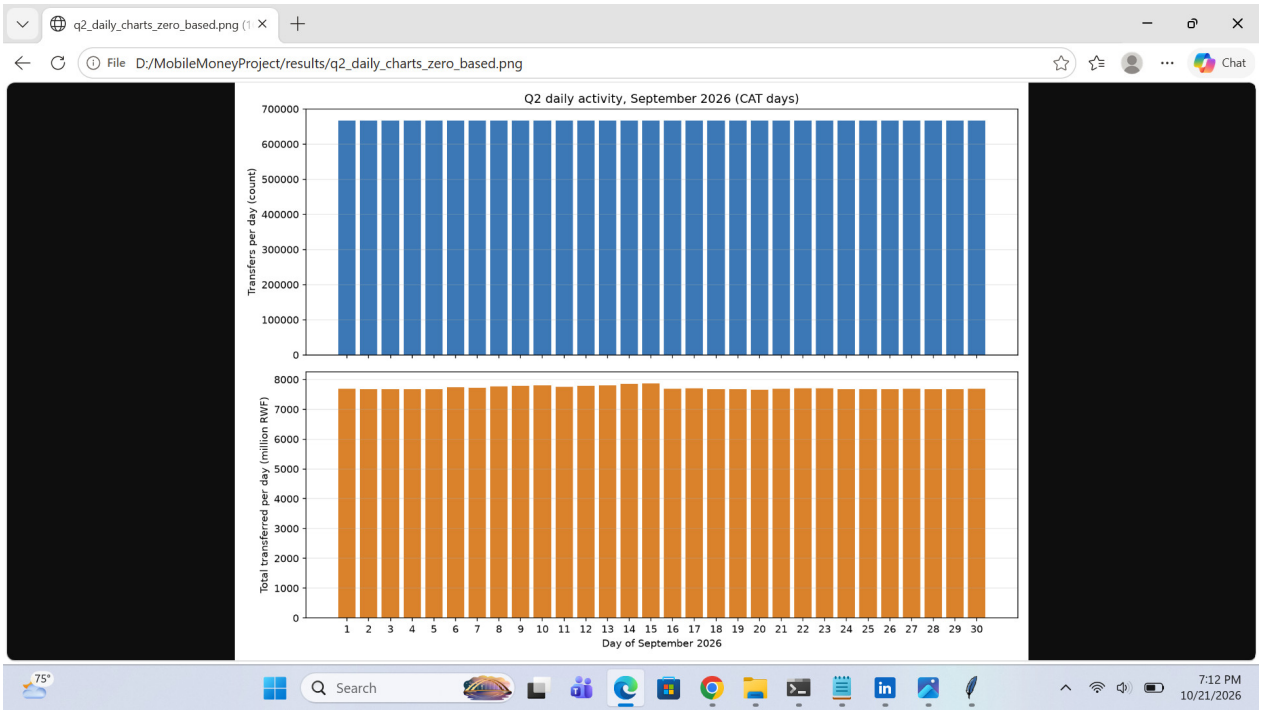


Figure 4.2-3: The same data with y-axes starting at zero: the daily count and value are almost constant

One pattern. The number of transfers is almost flat at 666,666 to 666,997 a day, while the daily value rises from about 7.68 billion RWF on 1 to 5 September to a peak of 7.86 billion RWF on 14 and 15 September and then falls back to about 7.67 billion. The average transfer therefore grows from about 11,525 to 11,783 RWF (2.2 %) while the count hardly changes, so the mid-month rise comes from larger payments, not more payments. The zero-based chart shows that the whole variation is small compared with the daily level, which is why the first chart truncates its axes.

Why transfer value does not establish revenue. Transfer value is the money customers move. A provider earns only what it charges on those transfers, such as fees and commissions, and the fee rates, free transfers and the split between person, merchant and agent payments are not in these records. Two days with the same value can therefore bring very different revenue.

4.3 Shared devices

Finds the other accounts observed using the same device as the review account during overlapping periods, keeping an overlap where the larger of the two first days is at most the smaller of the two last days. It returns the other account, the device, its access type and the overlap start and end days. Both engines return the same rows for all six sets: **4** other accounts for A and D and **9** for B, C, E and F. All of them share a **handset**; no public terminal appears among the shared devices of these six accounts.

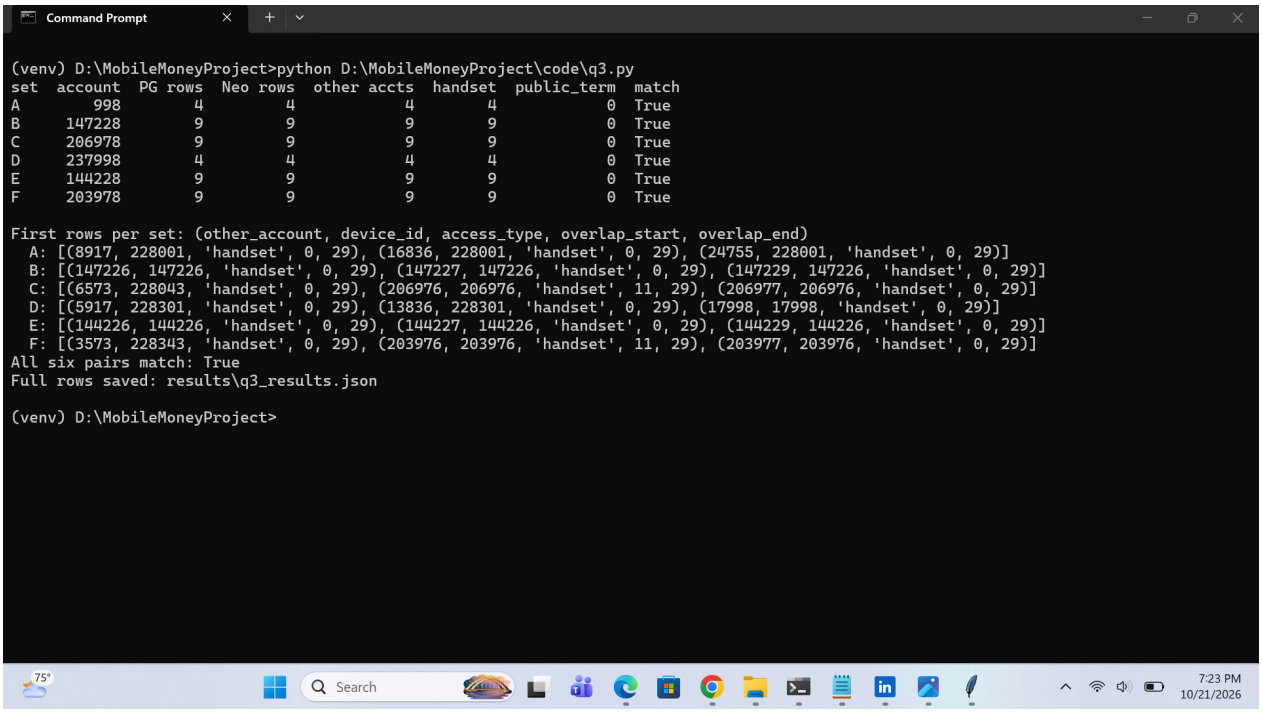


Figure 4.3-1: Q3 for all six sets: PostgreSQL and Neo4j row counts, other accounts, handset and public-terminal counts, and the first rows of each set

A legitimate sharing scenario. Members of one household, or the staff of a small shop, can all use the same handset, so several wallets are observed on one device for the whole month. **Overlap does not prove simultaneous use.** A device record is an interval of days with a count of observations, and it does not say which device started a payment. Intervals [2, 7] and [5, 11] overlap on days [5, 7], but the two accounts may have used the handset on different days or hours inside that overlap, or only one of them may have used it at all.

5 Investigate connected accounts

Device sharing gives a direct connection. This section extends the search to indirect connections through ownership and devices, and then follows transfer sequences. For each question the report shows the query pair, a completed case summary and an example from the loaded data.

5.1 Ownership and device network

Q4 finds the distinct other accounts connected to a review account through `OWNS` or `USED_DEVICE` links in either direction within 4, 6, 8 and 10 links. Transfers are excluded and no date or overlap filter applies. Each reached account is counted once and grouped by its owner region, sorted by region.

SQL. An `edges` CTE holds the four directions of the ownership and device links (Customer to Account, Account to Customer, Account to Device and Device to Account) and carries the node **kind** with the ID, so Customer 7 is never confused with Account 7. A recursive `walk` starts at the review account and joins on kind and ID, and its `UNION` removes repeated states. The final query takes the distinct accounts other than the start and groups them by the owner's region. **Cypher.** `MATCH (a)-[:OWNS|USED_DEVICE*1..DEPTH]-(b:Account)` is a bounded undirected path, `WHERE b <> a` excludes the start, `WITH DISTINCT b` counts each account once, and the owner region is then read through `OWNS`.

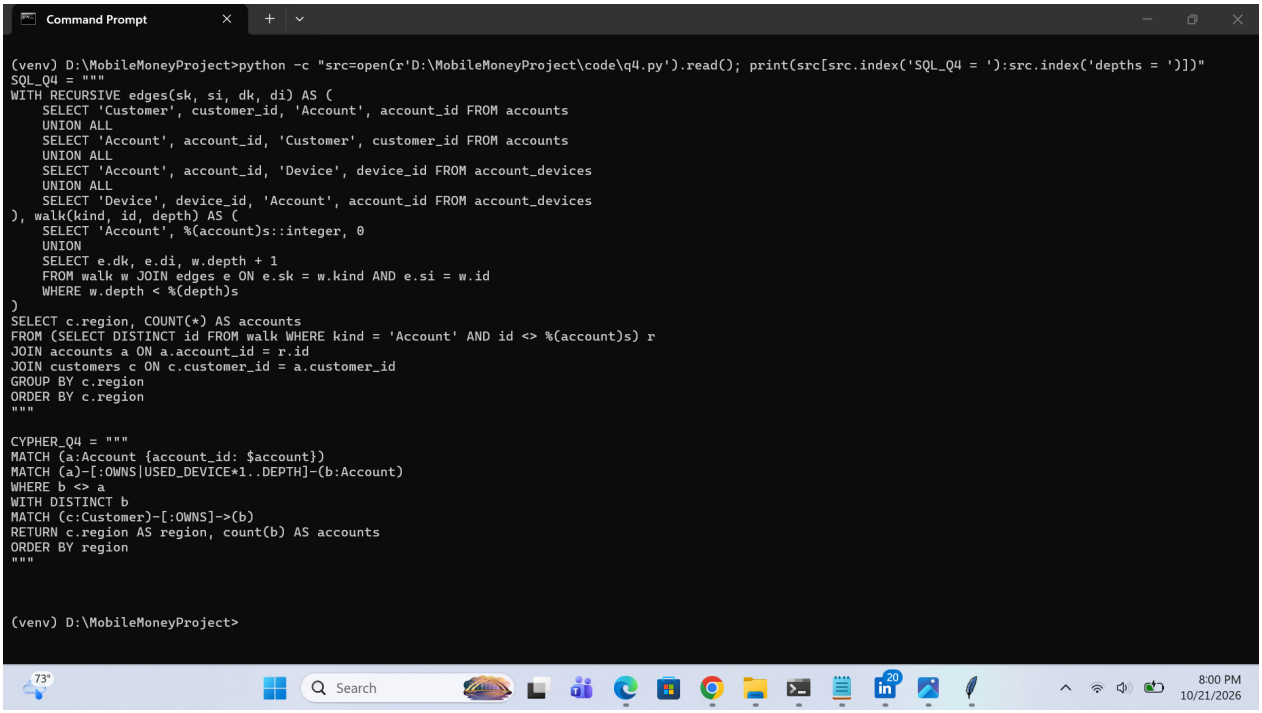


Figure 5.1-1: Q4 query pair: the recursive SQL with its edges CTE and the bounded undirected Cypher path

Prediction. The program q4.py was written to run depths 4 and 6 first and to run depths 8 and 10 only after predictions were written (see its usage line in the code submission). We did not keep a written record of our predicted counts for depths 8 and 10, so we cannot show one, and we do not present a prediction made after the results as if it had come before. The reasoning we can give is based on depths 6 and 10 and is stated afterwards: the counts rise sharply between 6 and 10 links (A 43 to 187, B 178 to 228, C 483 to 1,433), so each extra link reaches many accounts with a weaker association, and growth should slow once the connected network is used up, as it already has for B.

Accounts reached (all 24 PostgreSQL and Neo4j results match)

Set	Review account	4 links	6 links	8 links	10 links
A	998	10	43	175	187
B	147228	46	178	188	228
C	206978	146	483	832	1,433
D	237998	10	44	180	192
E	144228	47	183	193	230
F	203978	147	488	837	1,426

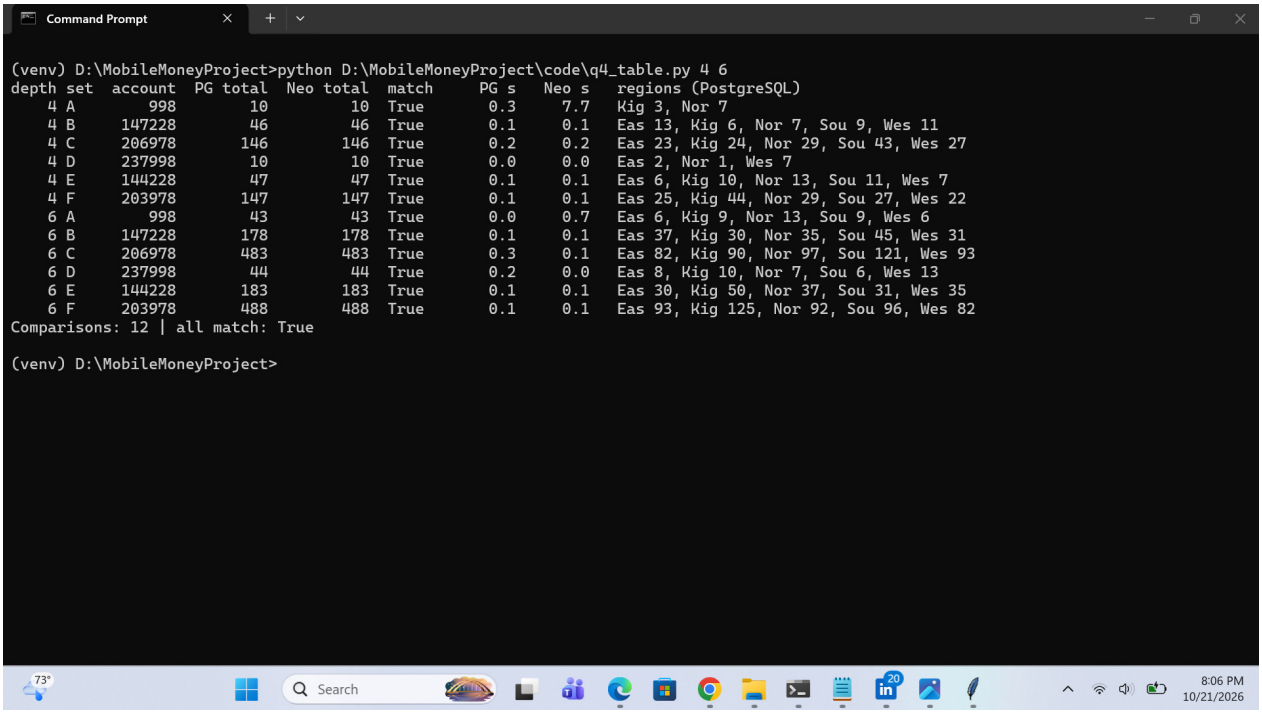


Figure 5.1-2: Q4 comparisons at depths 4 and 6 (12 cases): totals, match flag and accounts per owner region; all match

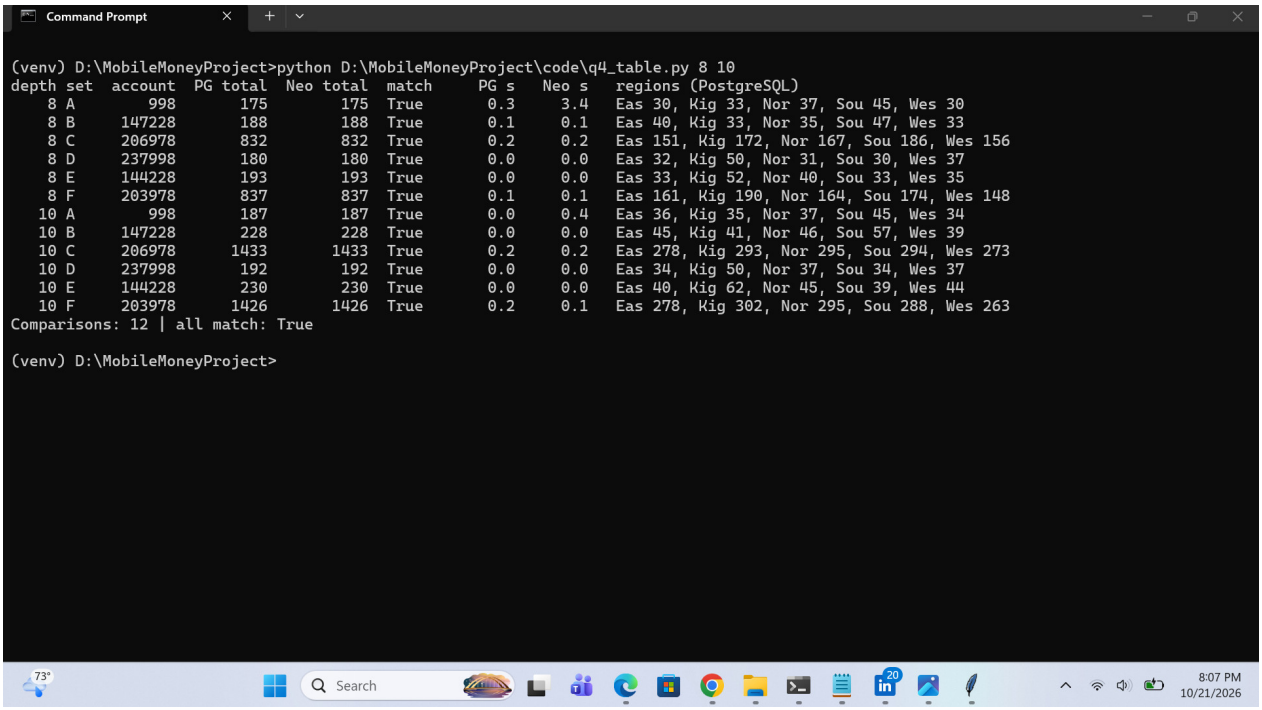


Figure 5.1-3: Q4 comparisons at depths 8 and 10 (12 cases): totals, match flag and accounts per owner region; all match

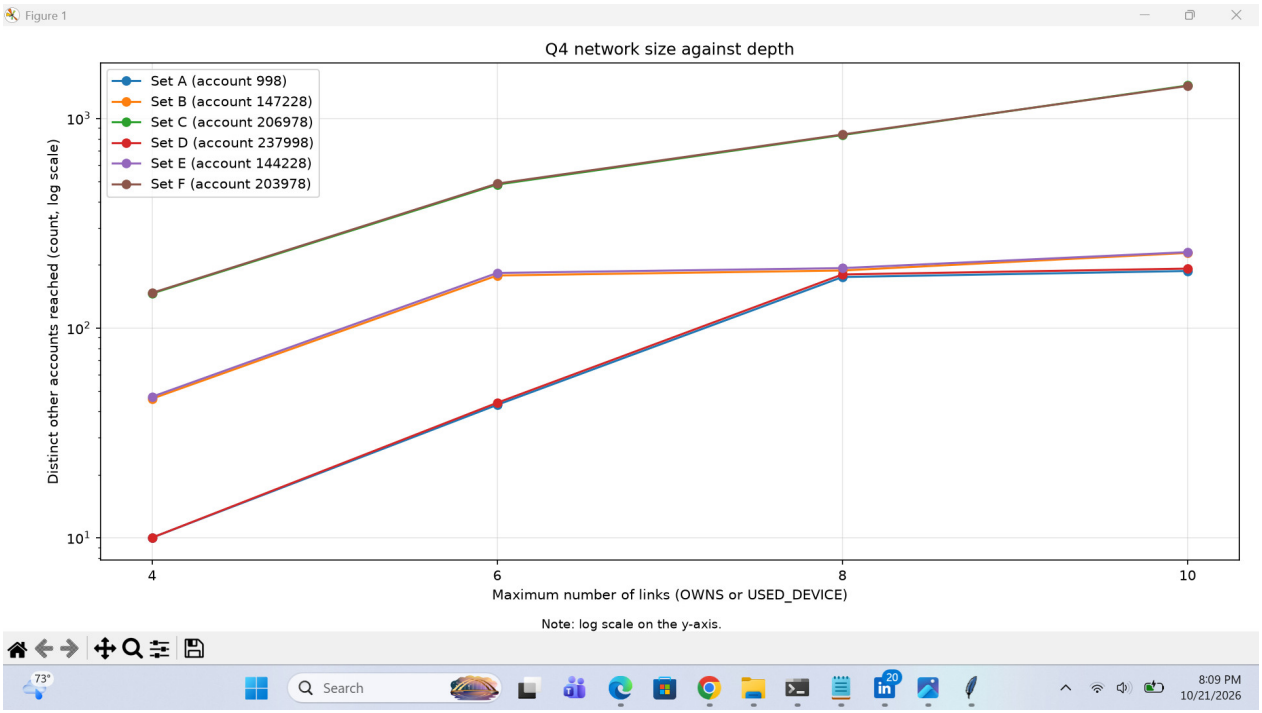


Figure 5.1-4: Q4 accounts reached against the maximum number of links (log scale). Sets with nearly equal results overlap: A with D, B with E, and C with F

One supporting path. For account 998 within four links, the path is `Account 998 - Device 228001 - Account 8917 - Device 8917 - Account 228917`, with four `USED_DEVICE` links: account 998 and account 8917 used handset 228001, and account 8917 also used device 8917, which account 228917 used.

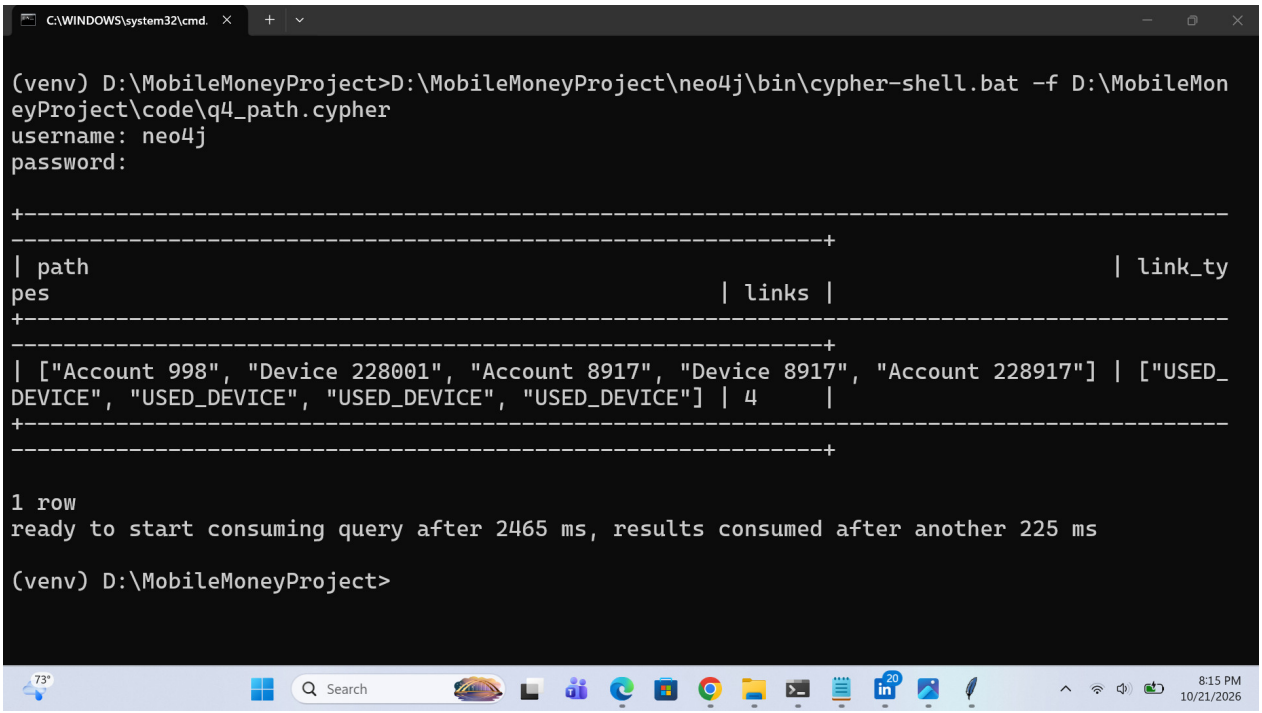


Figure 5.1-5: Cypher path from account 998 to account 228917: Account, Device, Account, Device, Account (four USED_DEVICE links)

How deeper links introduce weak associations. Every extra link adds a shared handset or a shared owner, so each step lets the search jump to people who have less to do with the review account. Sets A and D grow from 10 accounts at 4 links to about 190 at 10 links. Account 998 reaches 10 accounts in two regions (Kig 3, Nor 7) at 4 links, but 187 accounts spread over all five regions (Eas 36, Kig 35, Nor 37, Sou 45, Wes 34) at 10 links, which is hardly a meaningful relationship. Sets A, B, D and E level off between 175 and 230 accounts, while C and F keep growing to about 1,430, so they sit in a much larger connected cluster. A link count alone is therefore a poor measure of how strongly two accounts are related, and the shorter links deserve more weight.

5.2 Rapid onward payments

Looks for exactly three directed transfers through four distinct accounts. The first transfer starts at the review account inside its hour. Each later transfer is strictly later than the one before it and at most 600 seconds after it (and may fall after the hour), all receiver roles are included, and each onward amount is 90 % to 105 % of the preceding amount, written as `90 * previous <= 100 * next` and `100 * next <= 105 * previous` with BIGINT arithmetic in SQL. SQL joins three aliases of `transfers` receiver-to-sender and applies the rules, with six inequalities to keep the four accounts distinct. Cypher matches three directed `TRANSFER` relationships and applies the same rules. Both return the ordered transfer IDs and the final receiver, sorted by those fields.

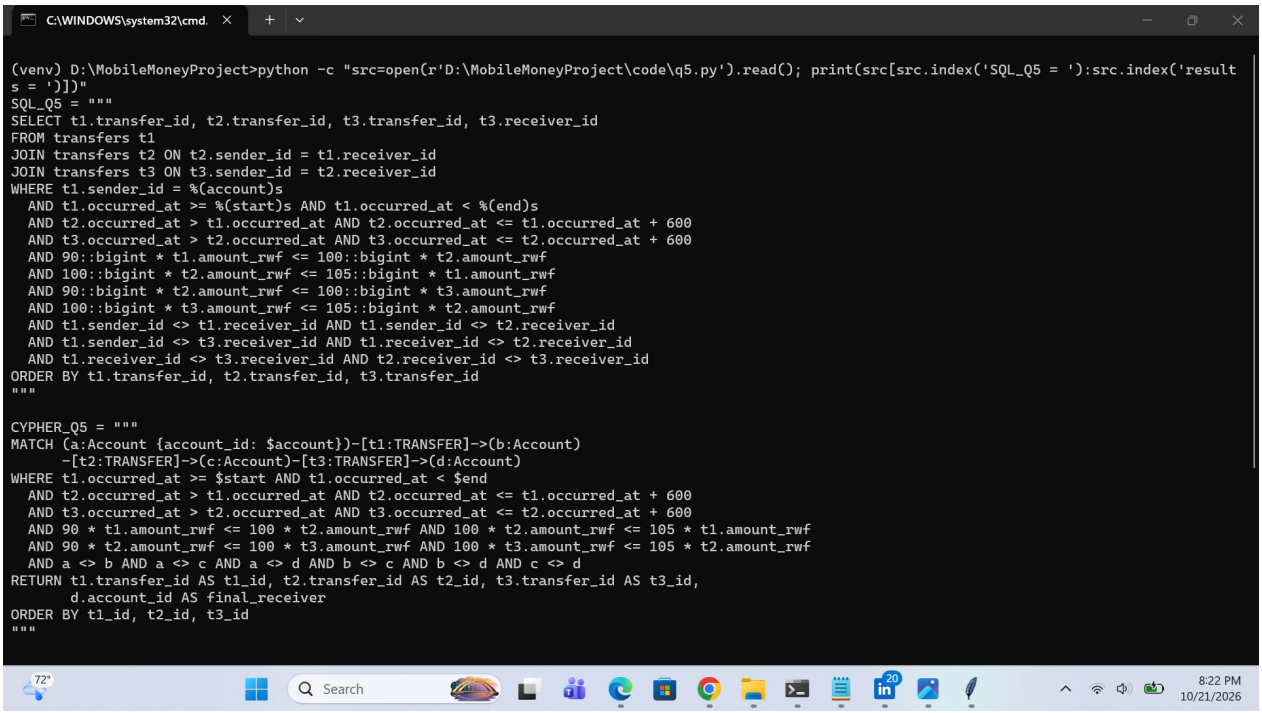


Figure 5.2-1: Q5 query pair: three joined transfer aliases in SQL and three directed relationships in Cypher, with the time-gap and amount rules

All six pairs match. Sets A, B and C each have **one** rapid onward chain and sets D, E and F have none. The first chain per set is: A (20000001, 20000002, 20000003) ending at account 24755; B (20000092, 20000093, 20000094) ending at 170985; C (20000229, 20000230, 20000231) ending at 230735.

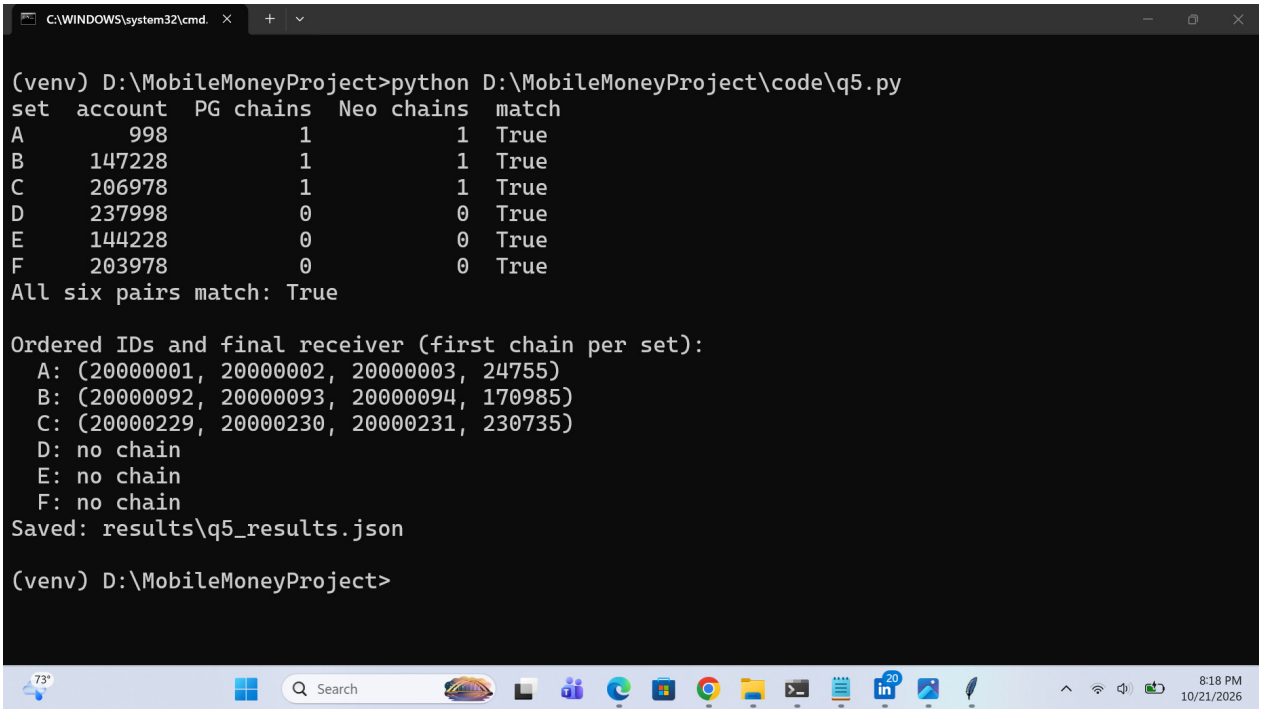


Figure 5.2-2: Q5: PostgreSQL and Neo4j chain counts for the six sets (all match) and the ordered transfer IDs with final receiver of the first chain per set

Chains with IDs, CAT times and amounts

Set	Transfer	Accounts	CAT time	Amount (RWF)	Of previous
A	20000001	998 to 8917	06 Sep 10:00:00	200,000	first
A	20000002	8917 to 16836	10:00:40	196,000	98.00 %
A	20000003	16836 to 24755	10:01:20	192,080	98.00 %
B	20000092	147228 to 155147	13 Sep 10:00:17	1,050,000	first
B	20000093	155147 to 163066	10:00:57	1,029,000	98.00 %
B	20000094	163066 to 170985	10:01:37	1,008,420	98.00 %
C	20000229	206978 to 214897	08 Sep 10:00:42	300,000	first
C	20000230	214897 to 222816	10:01:22	294,000	98.00 %
C	20000231	222816 to 230735	10:02:02	288,120	98.00 %

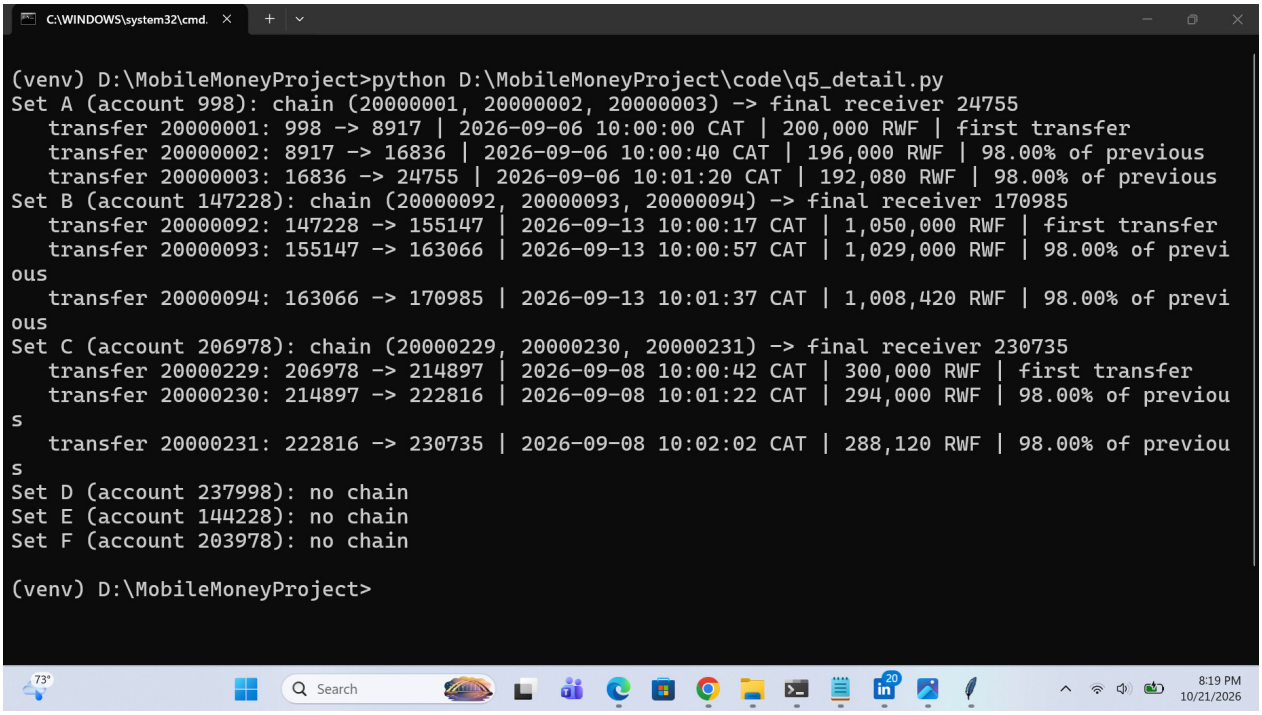


Figure 5.2-3: The chains of sets A, B and C with transfer IDs, CAT times, amounts and the percentage of the previous amount; no chain in D, E and F

Why a chain does not prove that the same funds moved. Q5 tests only timing and amounts: each payment follows the previous one within ten minutes and is 90 % to 105 % of it. Account 8917 receives 200,000 RWF at 10:00:00 and sends 196,000 RWF at 10:00:40, but nothing in the records says that the second payment was paid from the first. A wallet can hold other money, and the amounts may match by coincidence, because of fees, or because the accounts belong to one household or business that settles in rounds. Balances, fees and the source of each payment would be needed to follow money itself.

5.3 Circular payments

Asks whether a sequence of 3 to 8 transfers starts and ends at the review account. All events must lie inside its hour in strictly increasing time order and span at most 1,800 seconds, the Q5 amount rule applies, all roles are included, the search stops at the first return, and other accounts may repeat. The recursive SQL carries the current account, the ordered transfer IDs, the first and last time and the last amount, extends only eligible transfers (`len < 8`) and stops extending once the path is back at the review account. The Cypher uses a bounded path `TRANSFER*3..8` back to the start with `all(...)` conditions for the hour, the increasing

time, the span, the amount rule, and none(. . .) to reject an earlier return. Both return the ordered transfer IDs and the length, sorted by sequence.

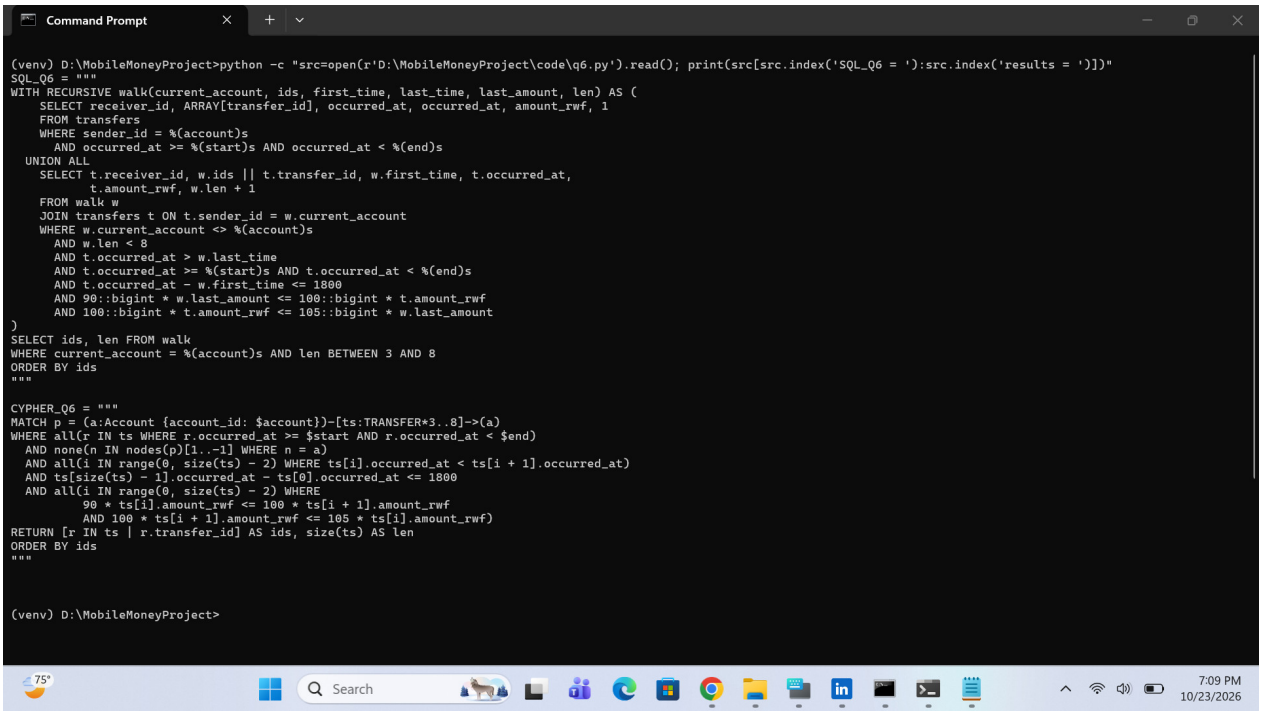


Figure 5.3-1: Q6 query pair: recursive SQL carrying the ordered IDs, times and last amount, and the bounded Cypher path with equivalent checks

All six pairs match. Set A has one cycle of length 4 and set C has one cycle of length 6; sets B, D, E and F have none.

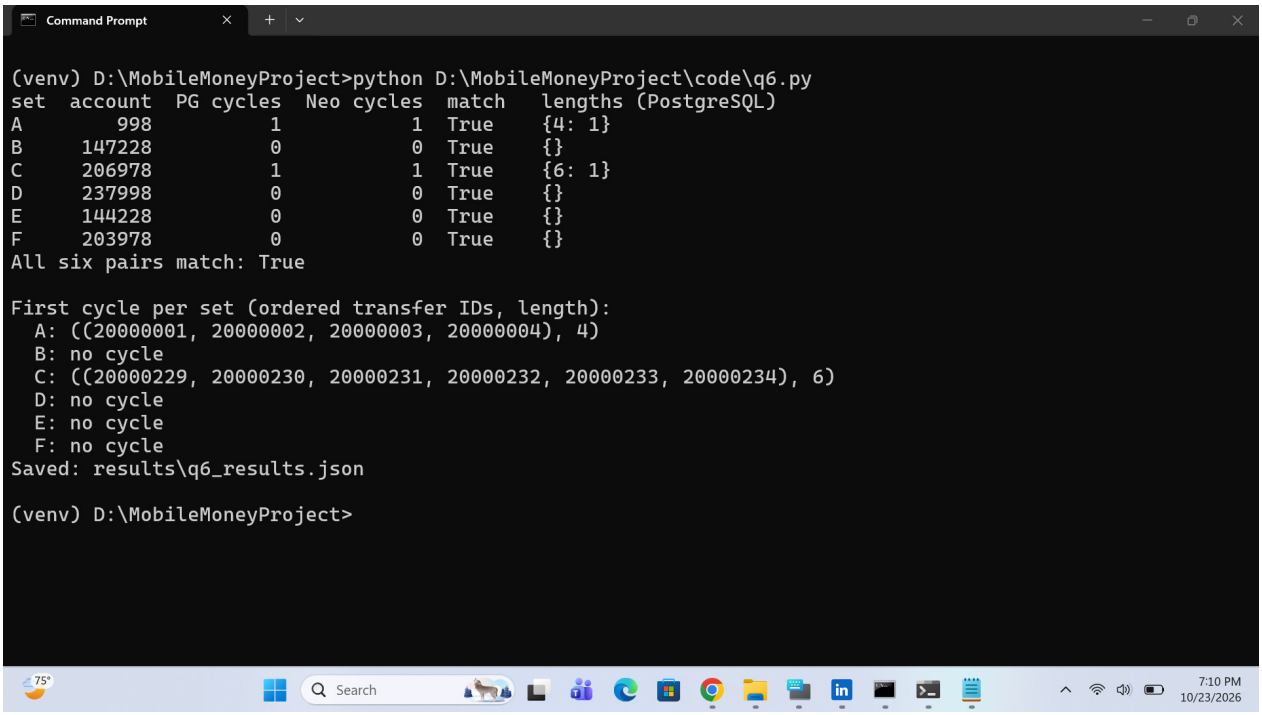


Figure 5.3-2: Q6: cycle counts and lengths for the six sets (all match) and the first cycle per set

Cycles

Set	Transfer IDs	Length	Time span (CAT)	Amounts (RWF), each 98.00 % of the previous
A	20000001 to 20000004	4	10:00:00 to 10:02:00 (120 s)	200,000; 196,000; 192,080; 188,238
C	20000229 to 20000234	6	10:00:42 to 10:04:02 (200 s)	300,000; 294,000; 288,120; 282,357; 276,710; 271,176

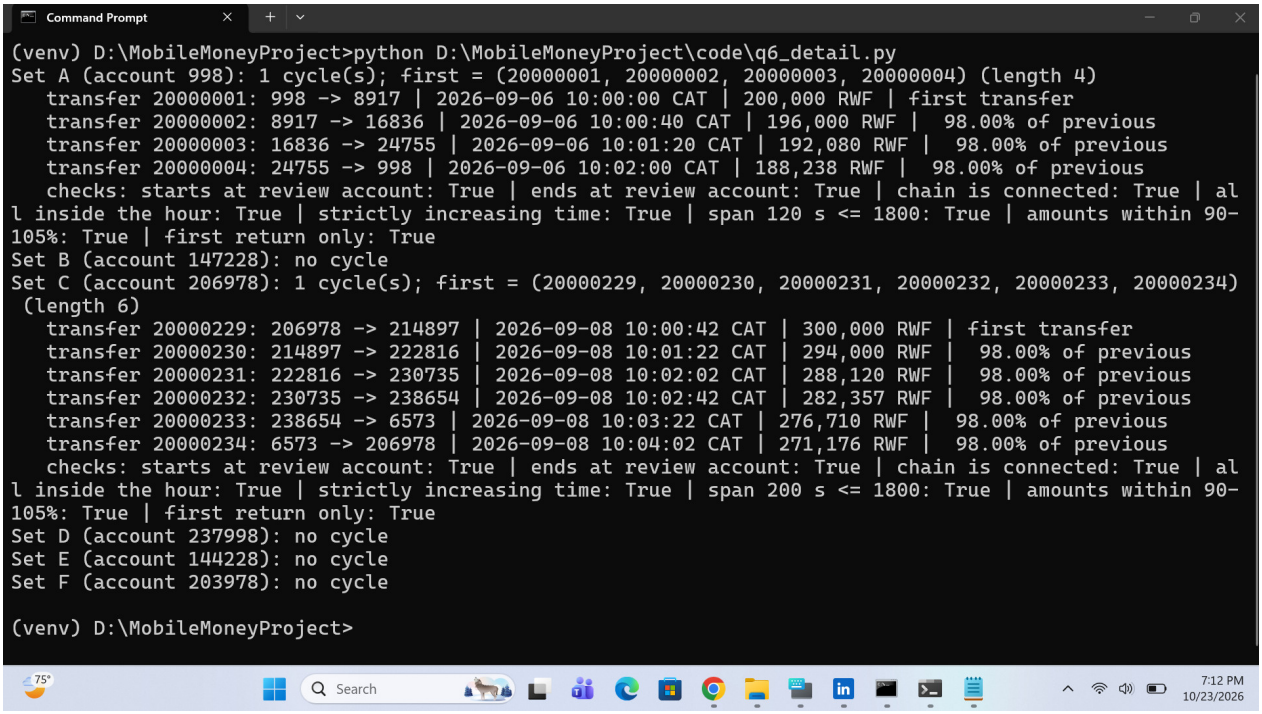


Figure 5.3-3: The cycles of sets A and C with every transfer, CAT time and amount, and the checks (starts and ends at the review account, inside the hour, strictly increasing, span within 1,800 s, amounts within 90 to 105 %, first return only), all True

In set A the money returns to account 998 as 188,238 RWF, 94.1 % of the 200,000 RWF sent, and in set C as 271,176 RWF, 90.4 % of 300,000 RWF. **Further evidence I would request:** the balance history of every account in the cycle, the fees charged on each transfer, the cash-in and cash-out records, which device and session started each payment, whether the accounts belong to one household, group or business, and the account holders' explanation. I would also look at a longer period to see whether the pattern repeats. A rotating savings group, a refund or a test of the service could explain a payment that returns in minutes, while a repeated circle with no other activity would raise concern.

5.4 Shortest transfer connection

Finds the fewest directed transfers from each review account to its destination over the whole month, using bounds of 4 and 6, where every account on the route must have the role person and amounts and time order are ignored. The SQL adapts the recursive walk to directed transfer neighbours: it starts at the source if it is a person, expands the distinct receivers of each account, keeps only person receivers at every step, removes repeated (account, depth) states with `UNION`, and returns the minimum depth at which the destination appears. Neo4j uses the native `SHORTEST 1` search with the person condition written inside the path pattern, so the restriction is applied **during** the search and not after a path has been chosen. A separate query retrieves one path for every successful case.

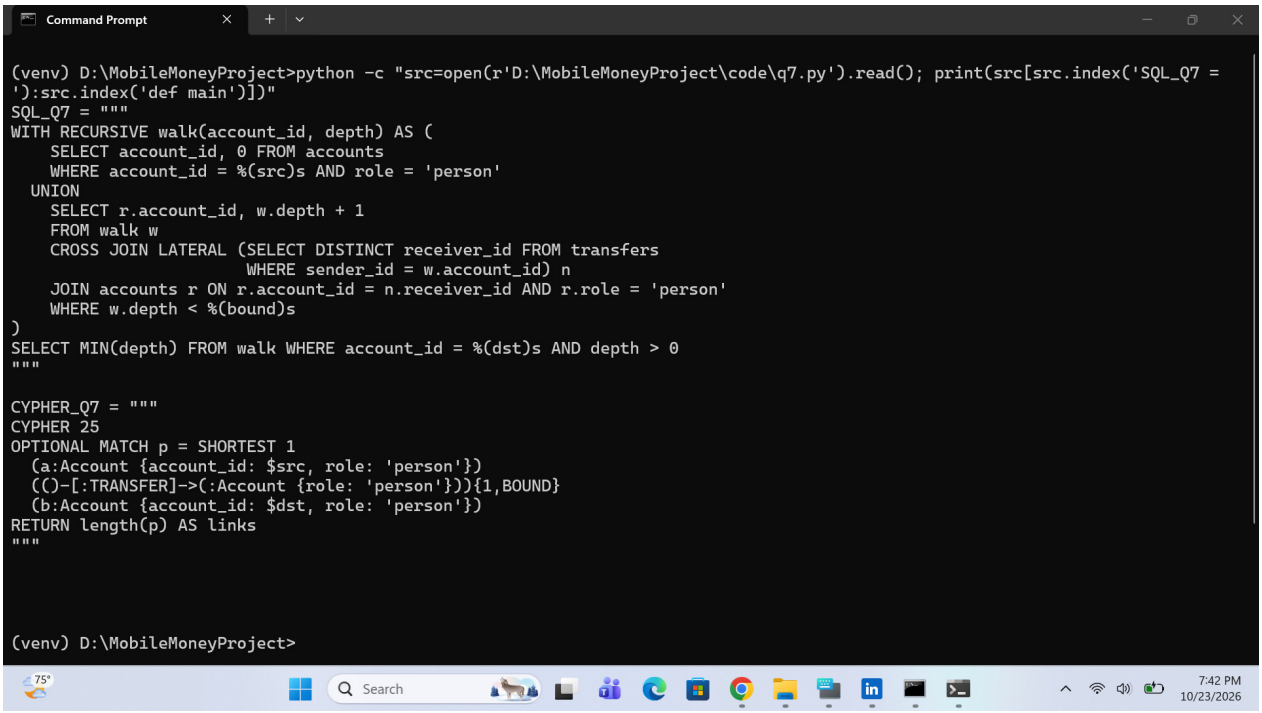


Figure 5.4-1: Q7 query pair: recursive SQL over distinct person receivers and the Cypher SHORTEST 1 pattern with the person role inside the path

All 12 length comparisons match. Within four transfers only sets A and D have a route (3 transfers); B, C, E and F have none. Within six transfers every set has one: A and D need 3 transfers and B, C, E and F need 5.

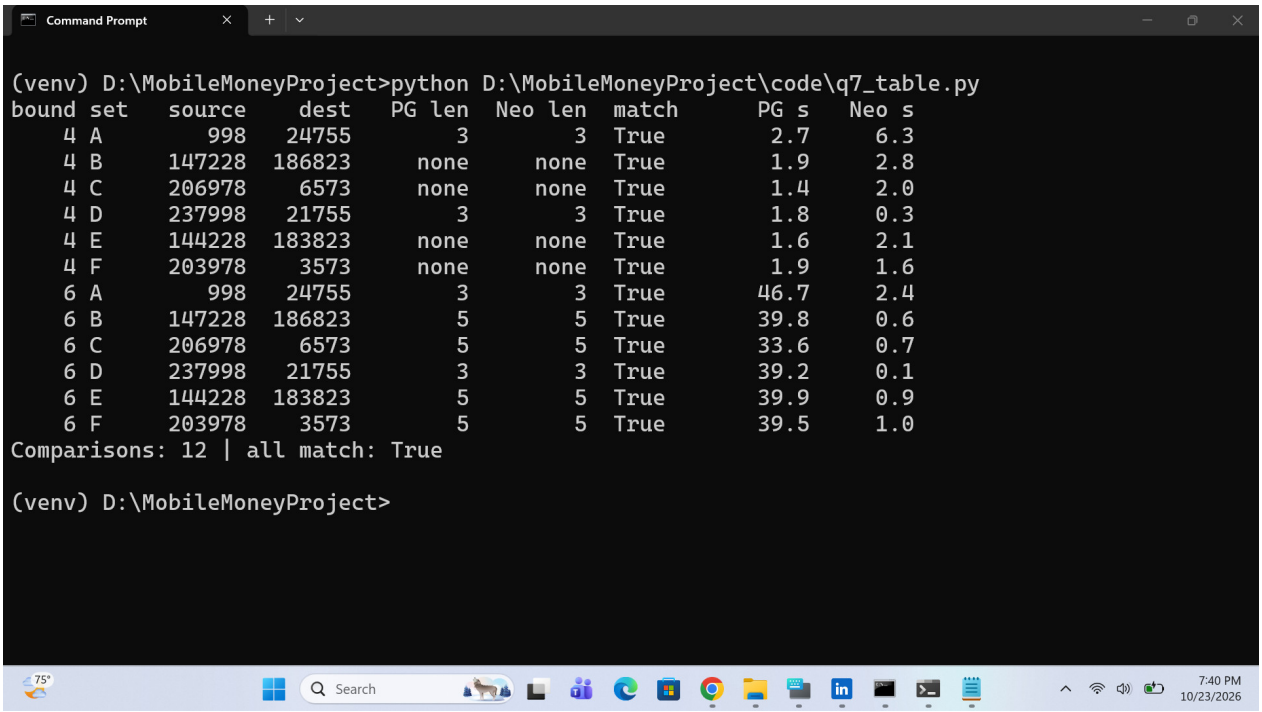


Figure 5.4-2: Q7: the 12 length comparisons (bounds 4 and 6, six sets) between PostgreSQL and Neo4j; all match

Shortest routes and the verified path of each successful case

Set	Route (account IDs)	Length	Transfer used for each hop
A	998, 8917, 16836, 24755	3	20000001, 20000002, 20000003
B	147228, 147906, 147310, 148101, 178904, 186823	5	5725461, 7330340, 4769221, 8125262, 20000096
C	206978, 214897, 222816, 230735, 238654, 6573	5	20000229 to 20000233
D	237998, 5917, 13836, 21755	3	20001651, 20001652, 20001653
E	144228, 152147, 160066, 167985, 175904, 183823	5	20001742 to 20001746
F	203978, 211897, 219816, 227735, 235654, 3573	5	20001879 to 20001883

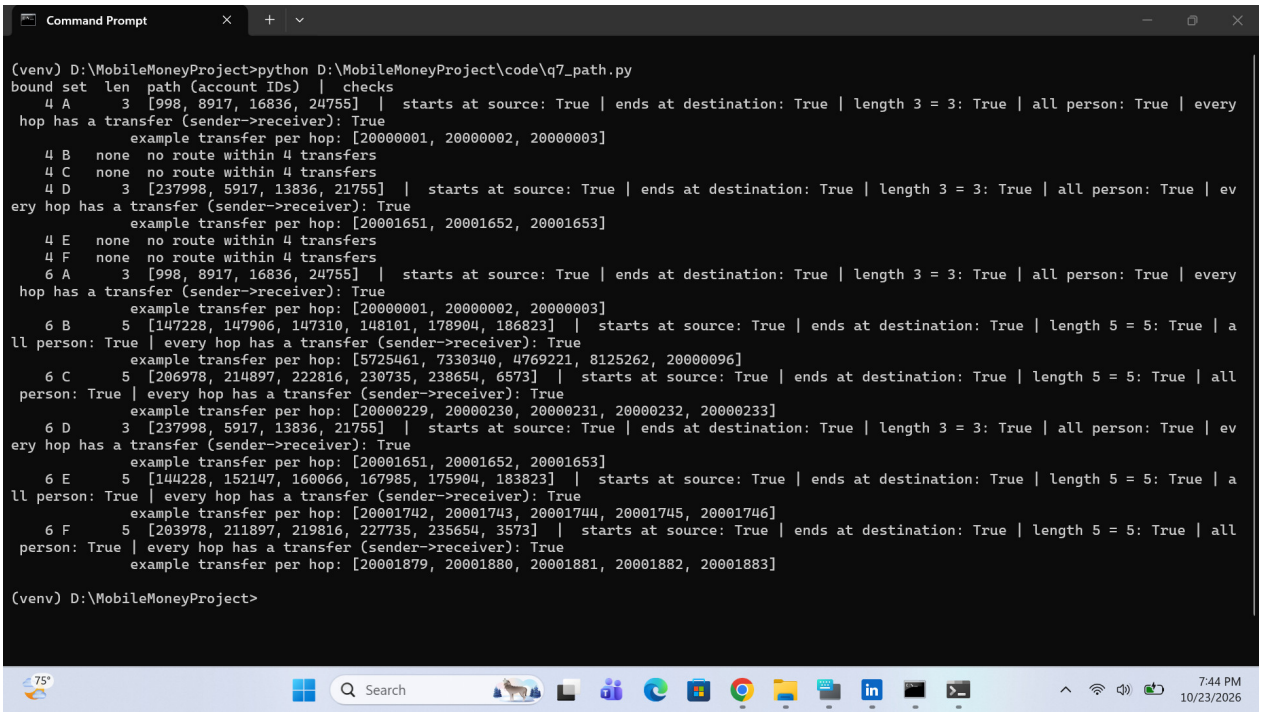


Figure 5.4-3: Q7 path checks for the eight successful cases: starts at source, ends at destination, length equals the minimum, all accounts are person, and every hop has a transfer from sender to receiver; four cases show no route within 4 transfers

Why no route within four transfers does not mean no connection. B, C, E and F have no person-only route within four transfers, but each has one of five transfers. The search stops at its bound, it is limited to person accounts, and the records cover one month, so a missing short route says only that no route was found inside these limits. **Why a structural route need not show onward movement of funds.** Q7 ignores amounts and time order, so the hops of a route can happen on different days and for unrelated amounts. Set B shows this: its first four hops are ordinary background transfers (IDs in the millions) and only the last one, 20000096, is part of the investigated pattern. Equally short routes can differ between engines, so the lengths are compared and one path per success is verified separately.

6 Check correctness and performance

6.1 Build one comparison program

`compare_all.py` defines `run_pg(query, params)` and `run_neo(query, params)`, which return every result row. It normalises integers, sorts the rows and compares them exactly. It covers **61 cases**: Q1 6, Q2 1, Q3 6, Q4 24, Q5 6, Q6 6 and Q7 12. For Q7 the lengths are compared separately from the supporting paths, which are verified in Section 5.4. Every case matches, and no mismatch had to be resolved.

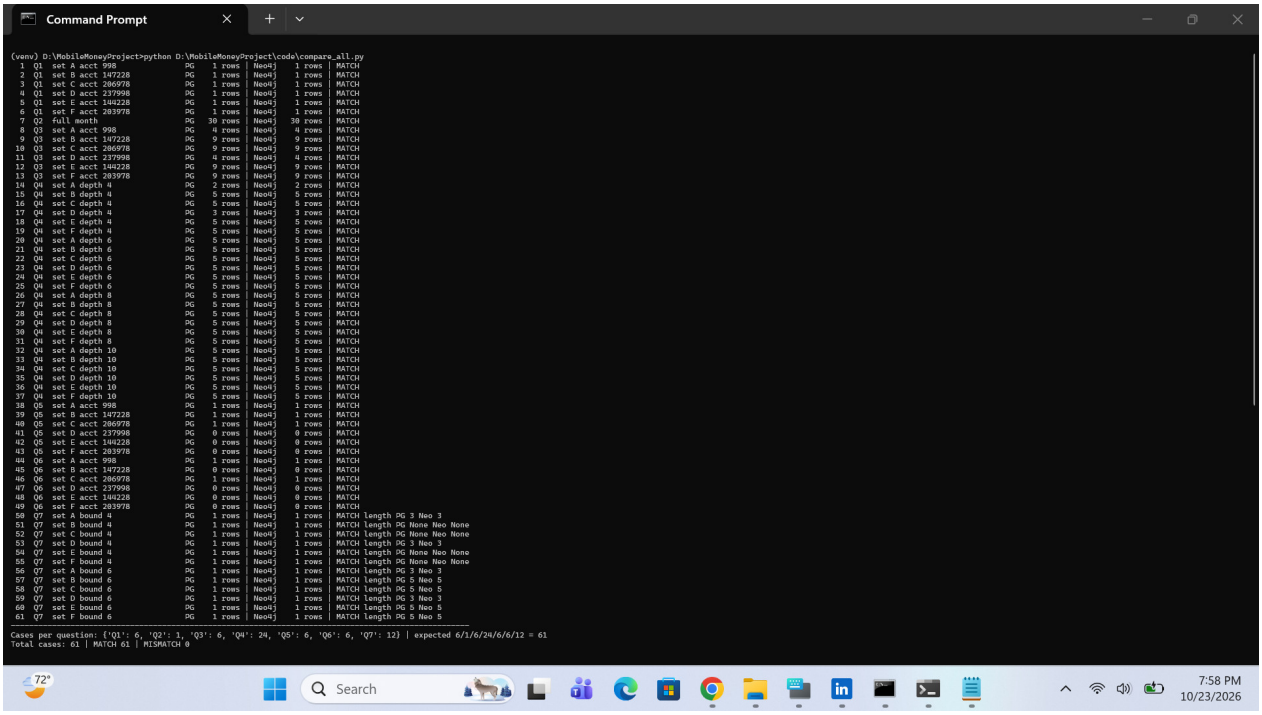


Figure 6.1-1: The 61-case status table: every case MATCH, with the row counts of both engines and the Q7 lengths; MATCH 61, MISMATCH 0

Cases per question

Question	Cases	Expected	Match	Mismatch	Status
Q1	6	6	6	0	ALL MATCH
Q2	1	1	1	0	ALL MATCH
Q3	6	6	6	0	ALL MATCH
Q4	24	24	24	0	ALL MATCH
Q5	6	6	6	0	ALL MATCH
Q6	6	6	6	0	ALL MATCH
Q7	12	12	12	0	ALL MATCH
Total	**61**	**61**	**61**	**0**	

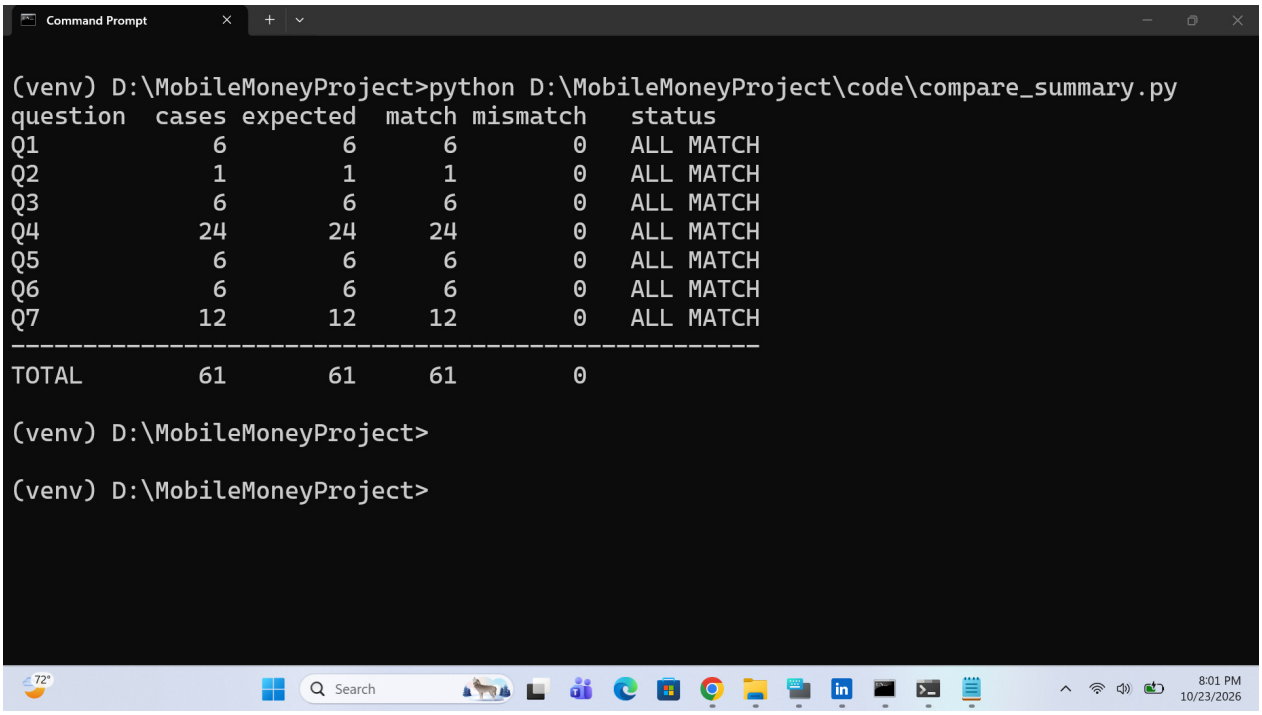


Figure 6.1-2: Summary of the comparison by question: 61 cases, 61 matches, 0 mismatches

Two results were also checked independently in Python against the captured CSV rows. **Check 1** recomputed the Q3 device overlap of set A from `account_devices.csv`: account 998 uses devices 998 and 228001 (days 0 to 29), and Python finds account 220998 on device 998 and accounts 8917, 16836 and 24755 on device 228001, each overlapping on days 0 to 29, the same rows both databases returned. **Check 2** rebuilt the Q6 cycle of set A from `transfers.csv`: transfers 20000001 to 20000004 start and end at 998, are connected, lie inside the hour, span 120 s, keep every amount within 90 to 105 %, and match the database result.

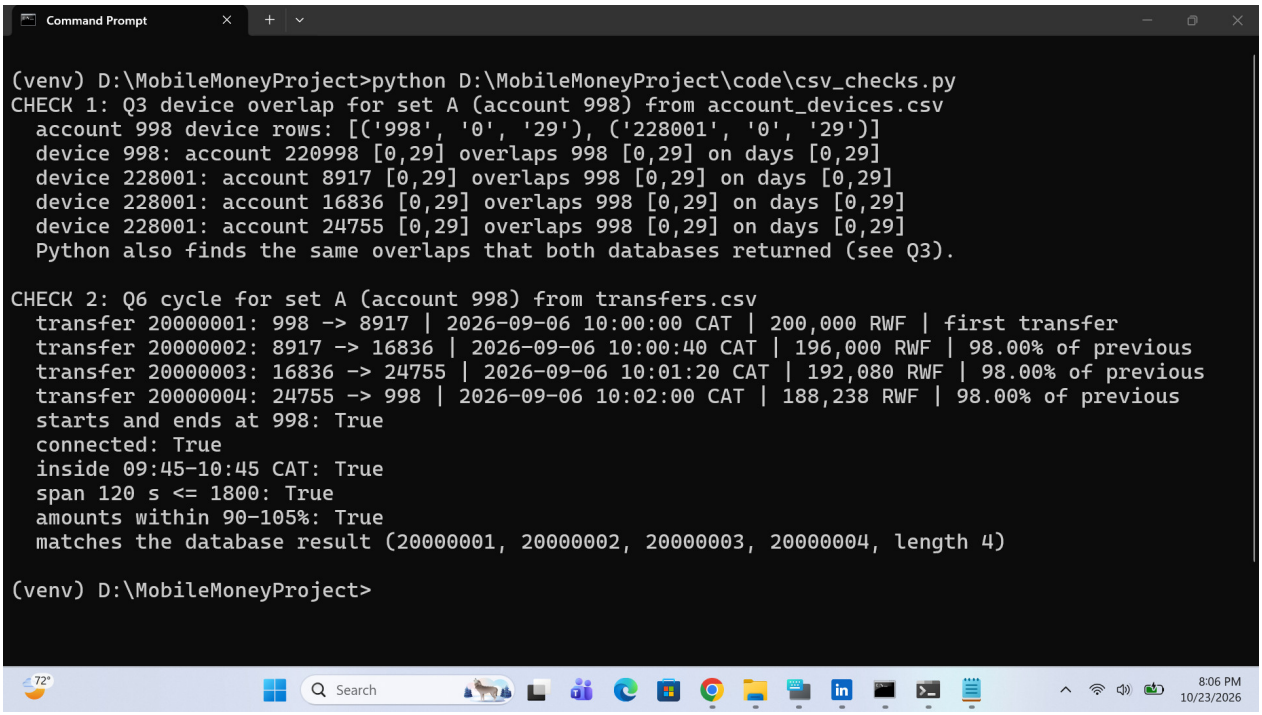


Figure 6.1-3: Independent CSV checks: the Q3 device overlap of set A and the Q6 cycle of set A, both matching the database results

6.2 Time a fixed selection

The timing used the same machine, **one database service at a time**, persistent connections and fixed resource settings (PostgreSQL `shared_buffers` 1 GB; Neo4j heap 2 GB and page cache 2 GB). The 21 cases per engine are Q1, Q3, Q5 and Q6 for sets B and E, Q2 for the full month, Q4 for B and E at the four depths, and Q7 for B and E at both bounds. Each case had **three untimed warm-ups and five measured runs**. A run was timed from submitting the query to receiving the final row with `perf_counter`, with limits of 180 s per warm-up and 60 s per measured query, and errors and timeouts were recorded separately and never as completed timings. The PostgreSQL run finished at 8:29 PM on 23 October; the PostgreSQL service was then stopped (8:31 PM) and the Neo4j run finished at 8:44 PM.

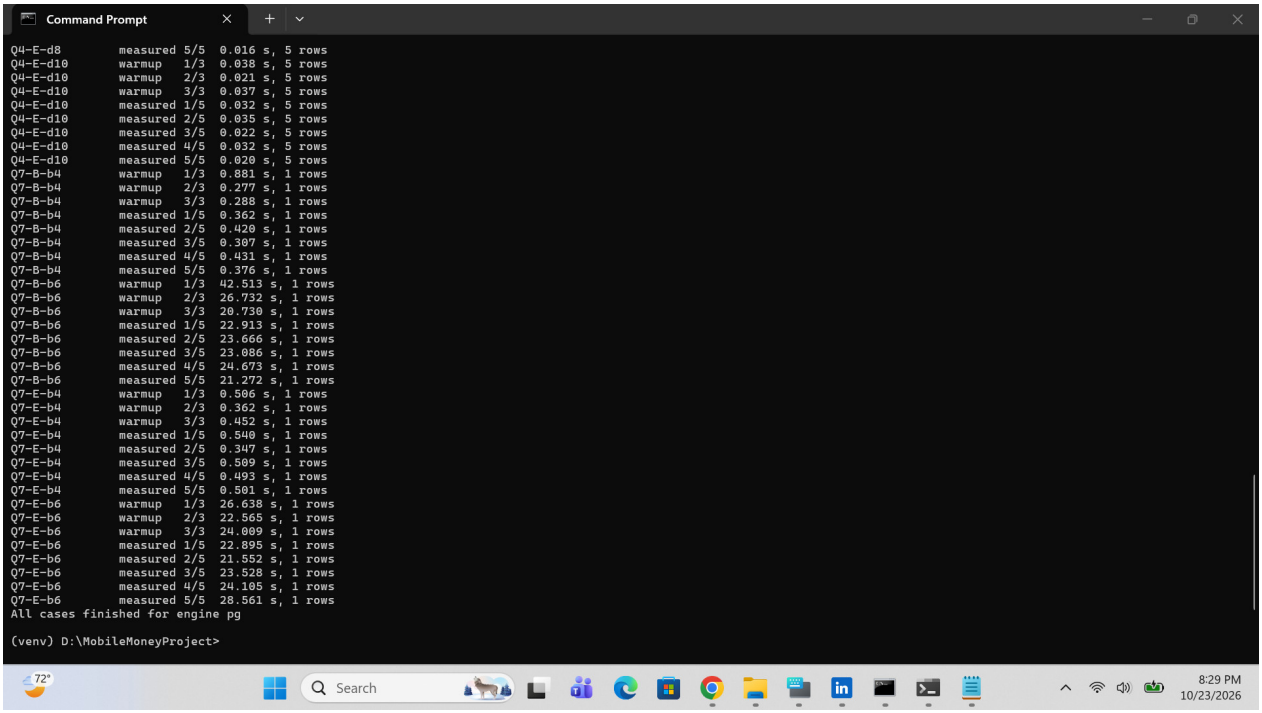


Figure 6.2-1: PostgreSQL timing run: three warm-ups and five measured runs per case, with row counts; all cases finished for engine pg

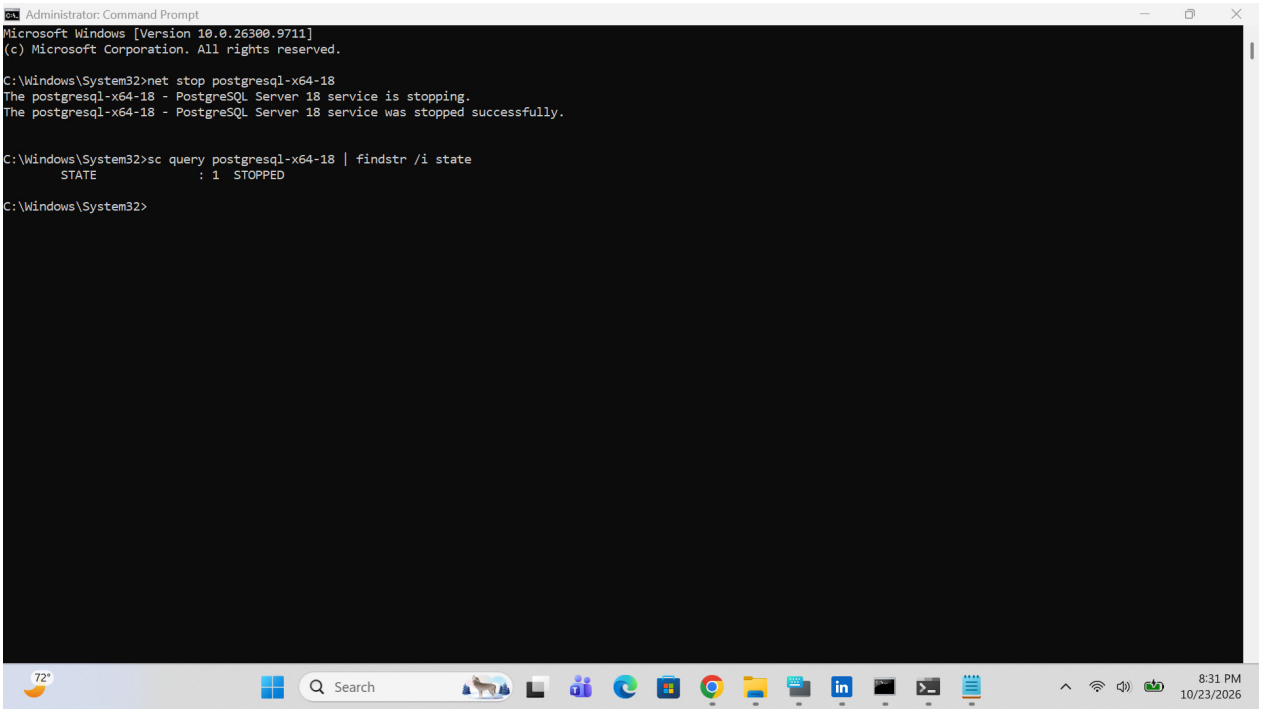


Figure 6.2-2: PostgreSQL service stopped (STATE: STOPPED) before the Neo4j run, so that one database service ran at a time

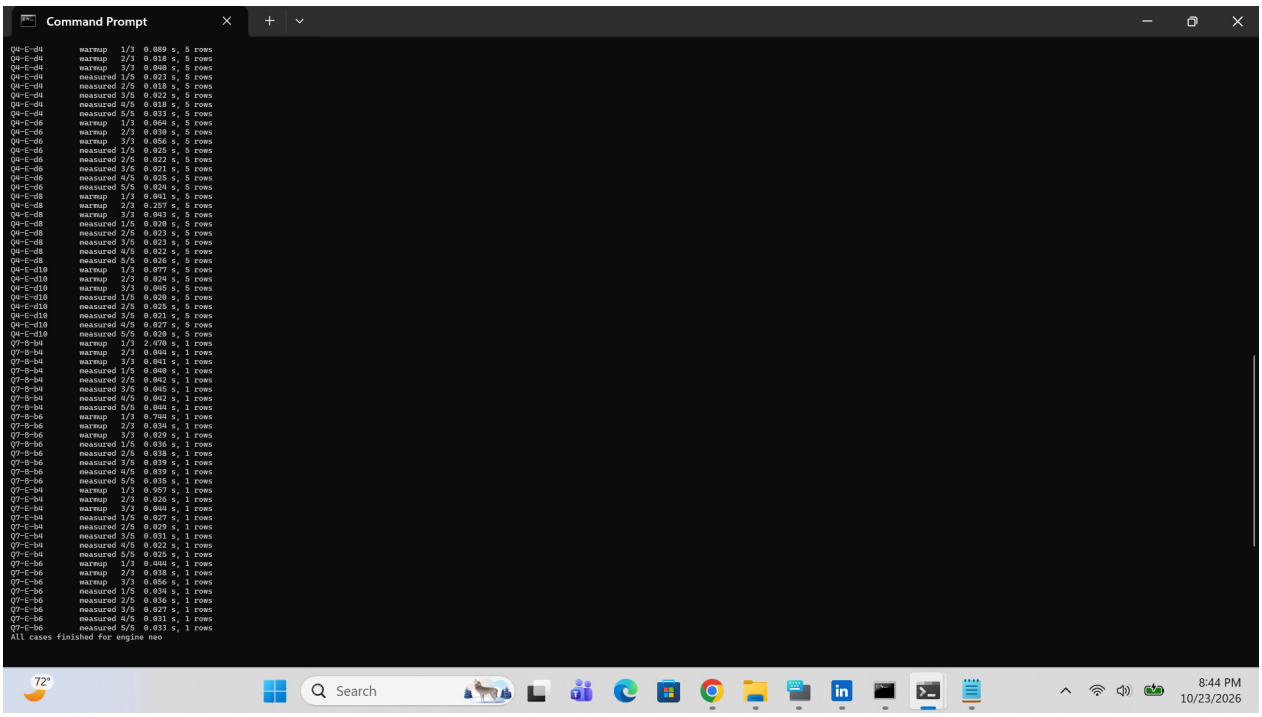


Figure 6.2-3: Neo4j timing run: all cases finished for engine neo

Timing of 21 cases per engine (milliseconds)

Case	PG median	PG min	PG max	Neo median	Neo min	Neo max	PG / Neo
Q1-B	1.1	0.9	3.0	24.9	20.3	26.7	0.0
Q1-E	0.5	0.4	0.8	18.5	16.7	21.6	0.0
Q3-B	1.6	1.6	2.7	21.1	19.2	22.1	0.1
Q3-E	0.8	0.6	2.0	18.3	15.8	22.7	0.0
Q5-B	4.6	3.1	5.6	23.9	20.8	25.1	0.2
Q5-E	0.5	0.4	0.7	17.7	13.6	22.1	0.0
Q6-B	2.7	1.9	4.2	22.9	18.9	26.2	0.1
Q6-E	0.6	0.5	0.7	17.7	15.9	19.2	0.0
Q2 full month	21,457.9	20,870.4	22,522.3	timeout in warm-up	-	-	-
Q4-B depth 4	5.0	3.1	5.8	26.8	22.5	32.7	0.2
Q4-B depth 6	9.2	4.5	11.6	29.8	23.1	32.0	0.3
Q4-B depth 8	22.7	22.1	24.7	28.5	27.2	117.6	0.8
Q4-B depth 10	31.9	21.2	40.6	30.5	22.7	31.2	1.0
Q4-E depth 4	2.7	1.9	3.9	21.9	17.5	32.5	0.1
Q4-E depth 6	10.1	4.8	10.5	24.0	21.0	24.9	0.4
Q4-E depth 8	16.3	16.0	24.1	22.6	19.8	25.6	0.7
Q4-E depth 10	32.2	20.2	35.1	21.4	20.0	26.7	1.5
Q7-B bound 4	375.5	306.9	431.3	42.0	40.4	44.8	8.9
Q7-B bound 6	23,086.1	21,271.7	24,673.2	37.9	35.5	39.2	609.1
Q7-E bound 4	501.1	347.0	540.5	27.4	21.7	31.0	18.3
Q7-E bound 6	23,527.8	21,552.1	28,560.6	33.2	27.4	35.8	708.7

The answers of both engines were identical in every completed case. **Neo4j could not complete Q2:** its first warm-up exceeded the 180 s limit, so Q2 has no Neo4j timing and is recorded as a timeout and never as a completed timing. PostgreSQL completed Q2 in a median of 21.5 s. For short lookups (Q1, Q3, Q5, Q6) PostgreSQL is much faster, at 0.5 to 4.6 ms against 17.7 to 24.9 ms. In Q4 the two engines are close at depth 10 (31.9 against 30.5 ms for B, 32.2 against 21.4 ms for E). With the first SQL version, Q7 at bound 6 took a median of 23.1 s in PostgreSQL against 37.9 ms in Neo4j for set B (a ratio of 609), which Section 6.3 examines.

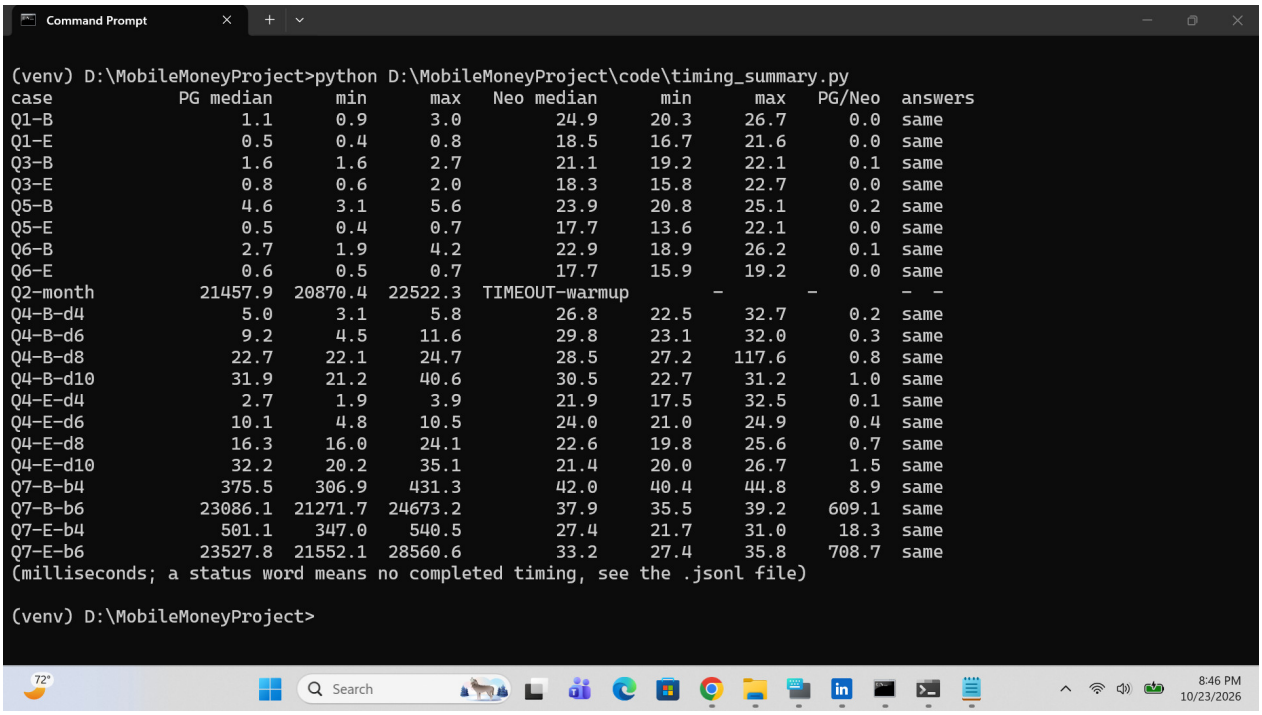


Figure 6.2-4: Timing summary of the 21 cases: median, minimum and maximum milliseconds per engine, the PostgreSQL to Neo4j median ratio, and whether the answers were the same

Outside the timed runs, `EXPLAIN (ANALYZE, BUFFERS)` was collected in PostgreSQL for Q2, for Q4 set B at depth 10 and for Q7 set B at bound 6. Q2 is a parallel sequential scan of `transfers` (146,272 buffers read from disk) followed by a partial and a final hash aggregate and took 32.8 s under instrumentation. Q4 builds its 1,187 rows in about 91 ms through a recursive union that runs 11 times and uses 14,285 buffer hits. Q7 took 58.8 s under instrumentation with **2,775,123 buffer hits**: the recursive union re-expands accounts at every depth (its hash aggregate runs 187,957 times over the `ix_transfers_sender_time` index and the `accounts` primary key is probed 24,799 times) and only afterwards takes the minimum depth. EXPLAIN ANALYZE adds measurement overhead, so its times are longer than the 23 s of the timed runs.

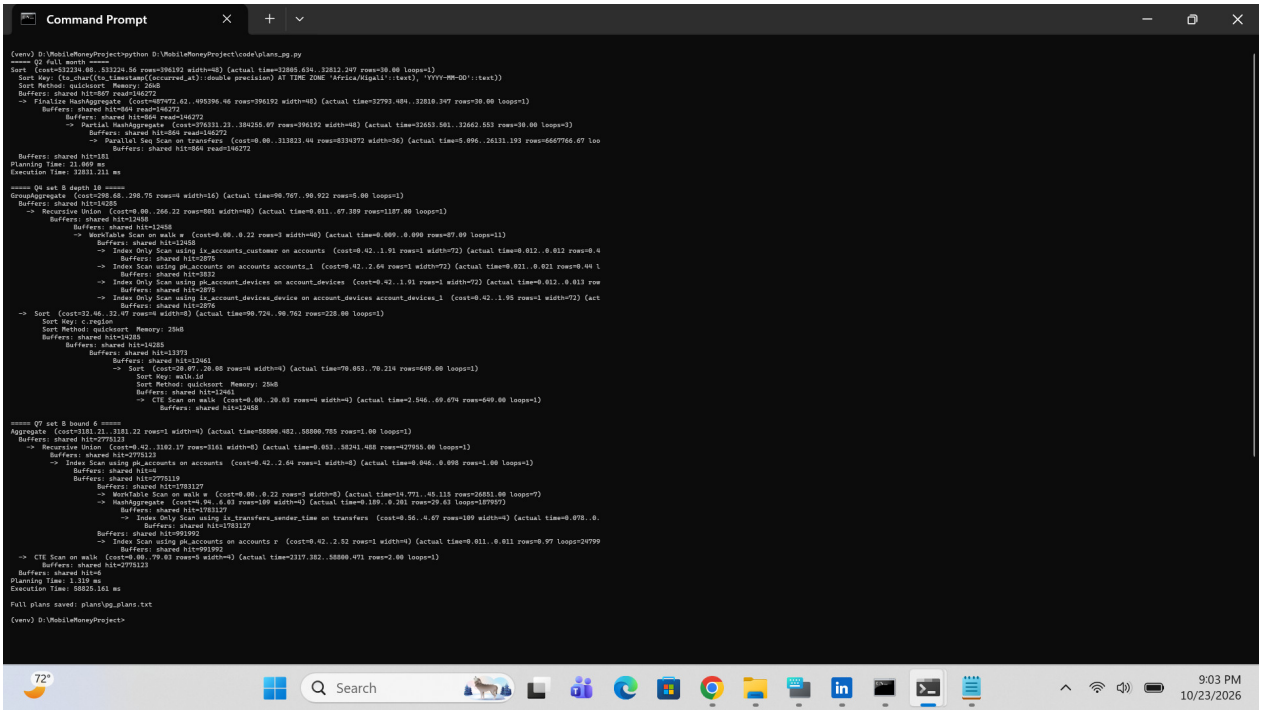


Figure 6.2-5: PostgreSQL EXPLAIN (ANALYZE, BUFFERS) for Q2 (full month), Q4 set B depth 10 and Q7 set B bound 6

Neo4j plans. PROFILE was run in Neo4j for Q4 set B at depth 10 and for Q7 set B at bound 6 (plans_neo.py). For Q2 the PROFILE run reached its 170 s limit and the transaction was terminated (TransactionTimedOutClientConfiguration), so Q2 has no profile; plans_neo_q2.py therefore collected an EXPLAIN-only plan, which is estimated and not executed (Figure 6.2-7). Q4 is one VarLengthExpand(Pruning, BFS, All) operator that needs 1,885 db hits to reach 453 nodes, 228 of them accounts; the 228 owner lookups add 912 db hits, and one EagerAggregation groups the result into the 5 regions, about 3,700 db hits in all. Q7 is one ShortestPath operator with 19,662 db hits, and the person-role test (all(… IN nodes(…))) sits inside that operator, so the restriction is applied during the search.

One processing difference. The Q7 plans show how differently the two engines search. Neo4j's ShortestPath is a breadth-first search that uses 19,662 db hits. The first PostgreSQL version builds the search from a recursive union that does not remember which accounts it has already expanded: the work table is read in 7 rounds totalling 187,957 rows, each probing the sender index once, and 427,955 rows are produced before the minimum depth is taken. This costs 2,775,123 buffer hits and 58.8 s under instrumentation (Figure 6.2-5). Db hits count store accesses and buffer hits count page accesses, so the two numbers are not the same unit, but the 141-fold difference points the same way as the 609-fold gap in time (Figure 6.2-4). Q2 shows a difference in the other direction. Neo4j has no cheaper route than reading all 20,003,300 TRANSFER relationships one by one (DirectedRelationshipTypeScan, a datetime() projection per row and a single EagerAggregation with no partial step), whereas PostgreSQL scans the table in parallel and aggregates partially in each worker. The estimate of 4,472.5 groups is far from the real 30 days, but it does not change the plan. Neo4j timed out where PostgreSQL finished in a median of 21.5 s.

Plan evidence

Query	Engine	Main operator	Work measure
Q2	PostgreSQL	Parallel sequential scan, hash aggregate	146,272 buffers read
Q2	Neo4j	DirectedRelationshipTypeScan, EagerAggregation	EXPLAIN only: 20,003,300 est. rows; PROFILE timed out
Q4 B depth 10	PostgreSQL	Recursive union, 11 rounds	14,285 buffer hits
Q4 B depth 10	Neo4j	VarLengthExpand(Pruning, BFS, All)	1,885 db hits (about 3,700 in all)
Q7 B bound 6	PostgreSQL	Recursive union, accounts re-expanded	2,775,123 buffer hits
Q7 B bound 6	Neo4j	ShortestPath	19,662 db hits

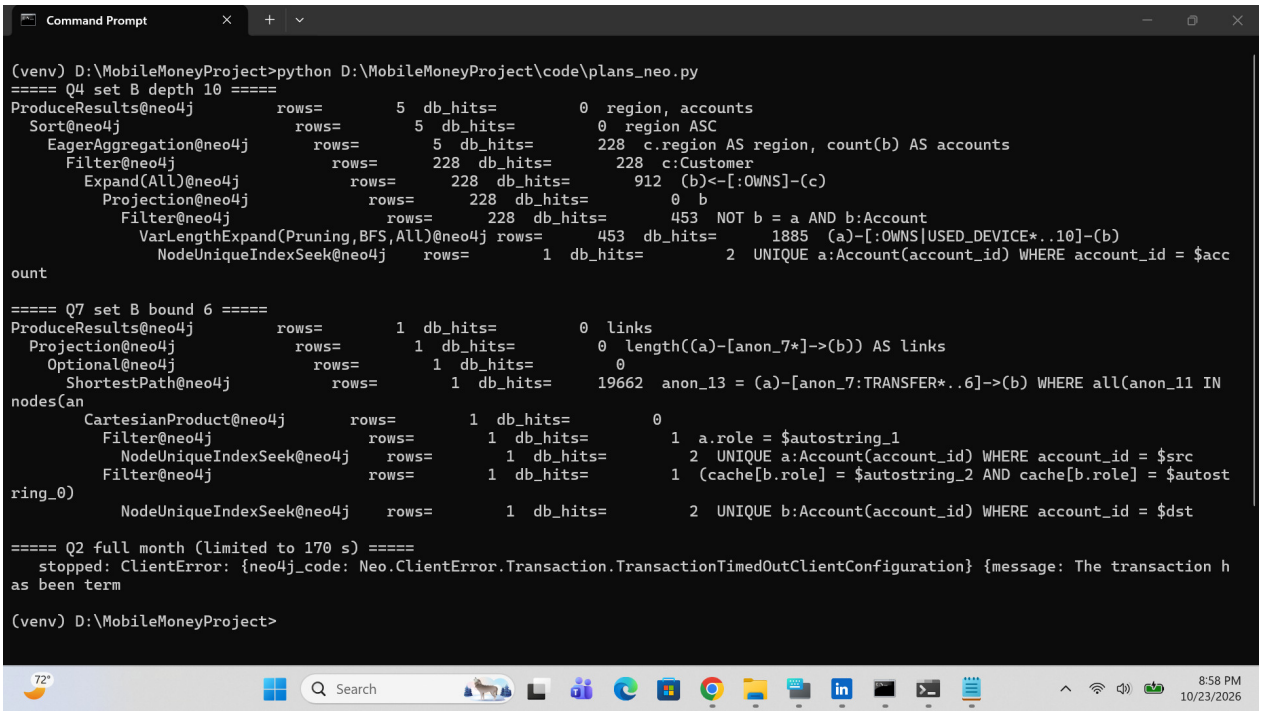


Figure 6.2-6: Neo4j PROFILE for Q4 set B depth 10 (VarLengthExpand, 1,885 db hits) and Q7 set B bound 6 (ShortestPath, 19,662 db hits); the Q2 PROFILE stopped at the 170 s limit

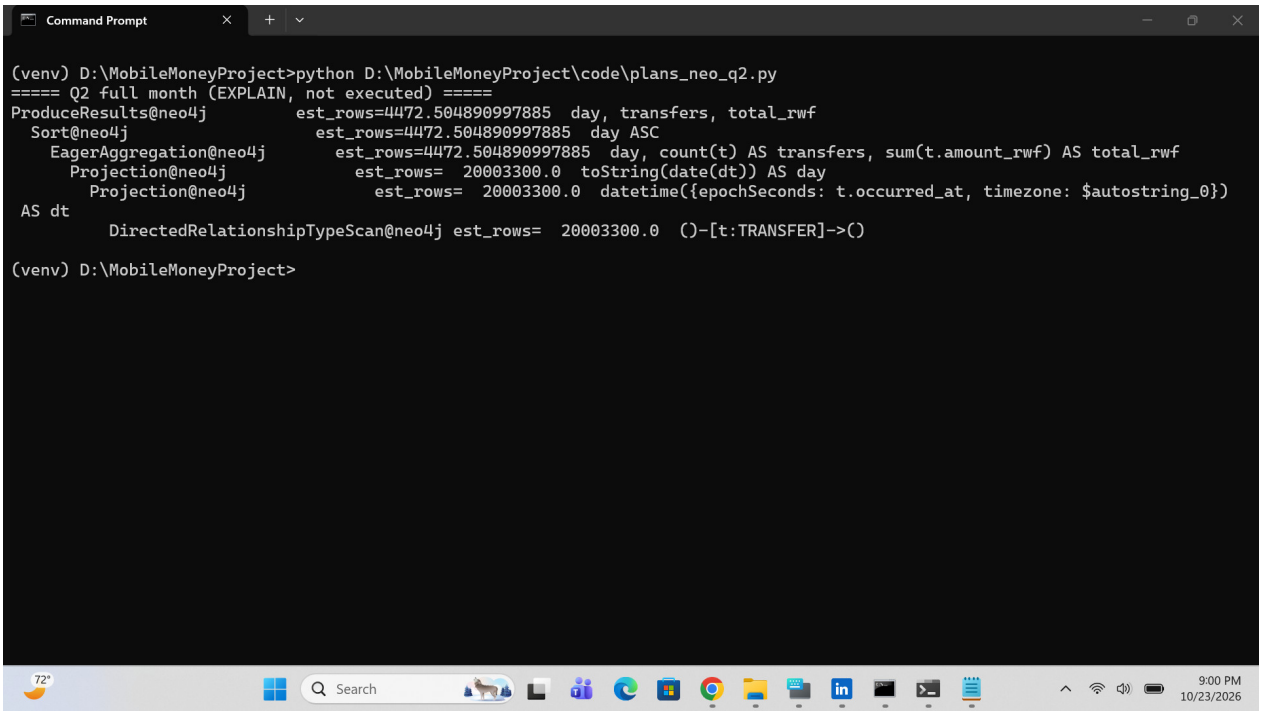


Figure 6.2-7: Neo4j EXPLAIN (not executed) for Q2: a scan of all 20,003,300 TRANSFER relationships, a datetime projection per row and one aggregation

6.3 Try one SQL improvement

What the plan shows. The original Q7 plan (Figure 6.3-1) wastes work in two ways. An account reached again at a later depth is expanded again: the recursive part reads 187,957 rows from the work table in 7 rounds and probes the sender index for each, about 30 distinct person receivers per account, which produces 427,955 rows. And the search continues to the bound after the destination has been reached, because the minimum depth is taken only at the end.

Prediction and change. The number of accounts reached grows roughly by a factor of 30 with every transfer, so two searches of about half the length should meet after far less work than one long search. We did not record a speed-up prediction in writing before the retiming, so none is shown and we do not present one made afterwards. The plan gave the reason for the change: the original search produced 427,955 rows, so a search from both ends, in which each side grows by the same factor per transfer but only over about half the length, should do far less work. The improvement searches from both ends: a forward search from the source along sender-to-receiver transfers and a backward search from the destination along receiver-to-sender transfers, both restricted to person accounts, joined to find the shortest connection (q7_v2.py, SQL_Q7_V2). The original SQL is kept unchanged beside it. In the after plan (Figure 6.3-2) the two recursive unions produce 2,216 and 2,560 rows instead of 427,955.

Plan before and after (set B, bound 6, EXPLAIN ANALYZE)

Measure	Original SQL	Both-ends SQL	Change
Rows produced by the recursive search	427,955	2,216 + 2,560	90 times fewer
Account expansions (hash aggregate loops)	187,957	188 + 210	472 times fewer
Buffer hits	2,775,123	27,670	100 times fewer
Execution time under instrumentation (ms)	58,825.2	276.4	213 times faster
Planning time (ms)	1.3	34.2	26 times slower

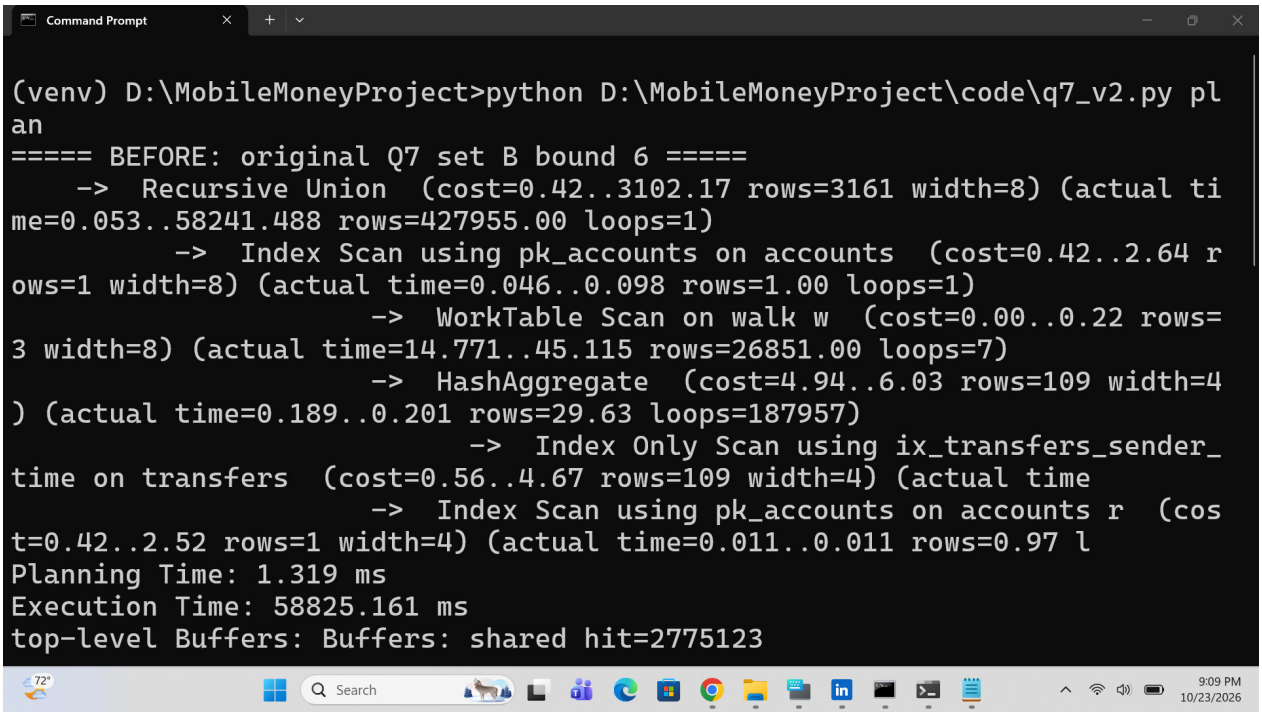


Figure 6.3-1: Q7 set B bound 6, original SQL: the recursive union returns 427,955 rows, the work table is read 187,957 times in total, and the query uses 2,775,123 buffer hits

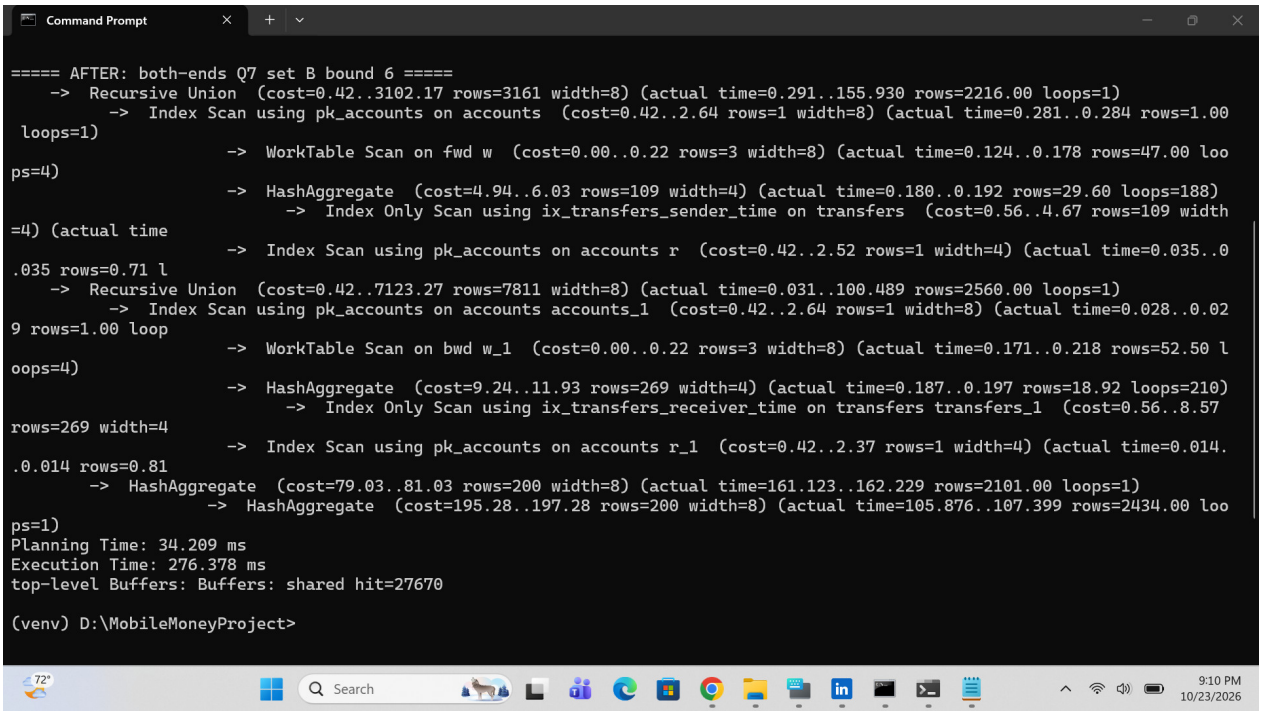


Figure 6.3-2: Q7 set B bound 6, both-ends SQL: a forward and a backward recursive union (2,216 and 2,560 rows), 27,670 buffer hits and 276.4 ms execution time

Correctness. All 12 Q7 cases were rechecked: for every set and both bounds the length from the new SQL equals the original SQL and Neo4j (lengths 3, none, none, 3, none, none at bound 4 and 3, 5, 5, 3, 5, 5 at bound 6), and the program reports that all 12 cases match (Figure 6.3-3).

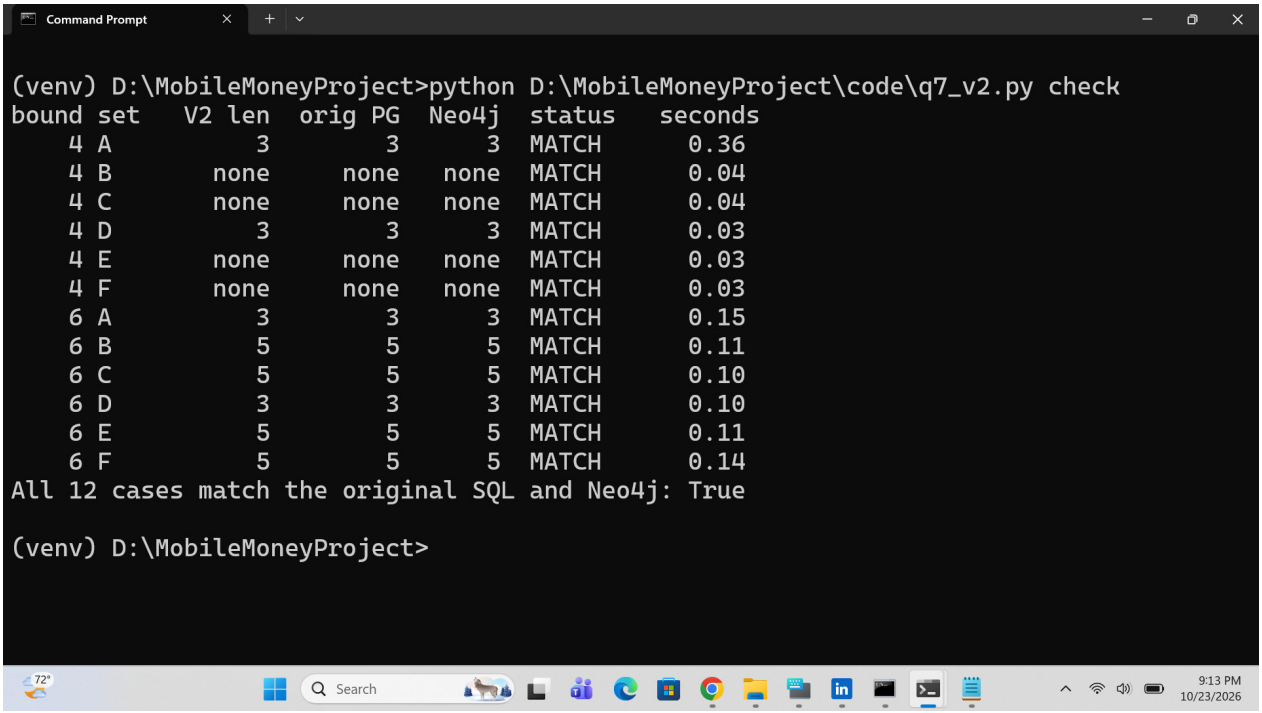


Figure 6.3-3: Recheck of the 12 Q7 cases: both-ends length, original PostgreSQL length and Neo4j length are equal in every case; all 12 MATCH

Timing before and after the improvement (milliseconds, median)

Case	Original SQL	Both-ends SQL	Speed-up	Neo4j	PG / Neo4j before	PG / Neo4j after
B bound 4	375.5	13.3	28.2	42.0	8.9	0.32
B bound 6	23,086.1	68.3	338.0	37.9	609.1	1.80
E bound 4	501.1	13.4	37.4	27.4	18.3	0.49
E bound 6	23,527.8	121.9	193.0	33.2	708.7	3.67

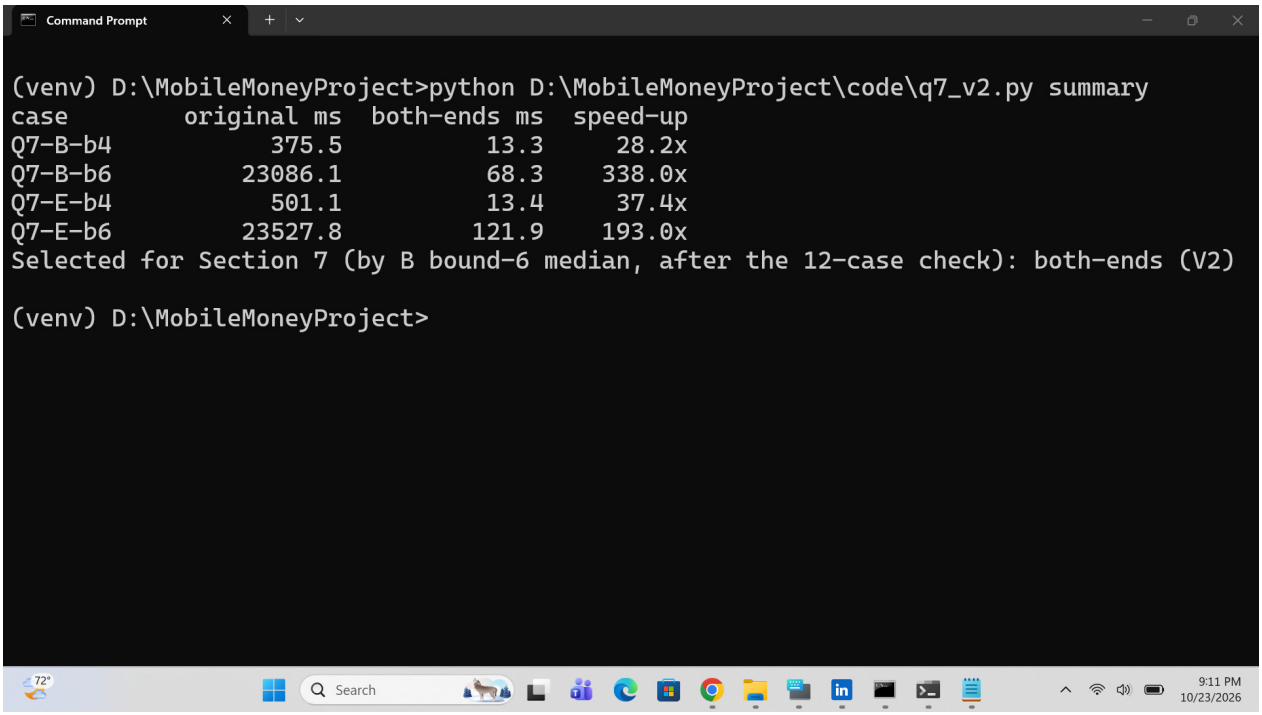


Figure 6.3-4: Q7 median milliseconds for the original and both-ends SQL with the speed-up of each case; both-ends (V2) is selected for Section 7 by the set B bound-6 median

Outcome. The attempt succeeded. The Q7 results did not change, but the work did: the new SQL is 28 to 338 times faster, and most of the earlier gap of about 600 times at bound 6 came from how the SQL was written, not from the relational model. A gap remains at bound 6, where PostgreSQL is now 1.8 times (set B) and 3.7 times (set E) slower than Neo4j, while at bound 4 PostgreSQL is faster (13.3 against 42.0 ms for B and 13.4 against 27.4 ms for E). The cost is a more complex statement: planning rose from 1.3 to 34.2 ms, which is visible on a query that now runs in tens of milliseconds. The both-ends version is selected for Section 7 and the final comparison because it matches in all 12 cases and has the lower set B bound-6 median (68.3 ms against 23,086.1 ms).

7 Test concurrent requests

7.1 Build a small load test

The provider may have several analysts querying at once, so the load test starts 1, 4, 8 or 16 client processes with Python multiprocessing (spawn start method). Each client holds one persistent connection, warms up once with sets A, B and C, and waits at a Barrier. At release a common start time is set and the deadline is 15 s later. Each client then cycles A, B, C, starts new cycles until the deadline, finishes any cycle it has started, and sends its next request as soon as the previous one completes. The 60 s query timeout is kept. Q4 at depth 10 and Q7 at length-only bound 6 are tested separately in each engine, with the both-ends SQL selected in Section 6.3 for Q7, and every engine, workload and client count is repeated three times, one database service at a time.

Every request is logged and its answer is checked after timing. Throughput is the number of correct completions divided by the time from barrier release to the end of the last cycle. The p95 latency is the successful latency at position ceil(0.95 n) of the sorted list, and each summary is the median of the three repetitions. Every one of the 24 repetitions per engine finished with 0 wrong answers, 0 errors and 0 timeouts (Figures 7.1-4 and 7.1-5).

Test conditions. The clients are light processes without numpy or pandas: an earlier version with heavier clients caused memory errors and a PostgreSQL restart in recovery mode on this 8 GB laptop, and the test was then run again with light clients. Neo4j ran with a 1 GB heap and a 1 GB page cache in the load test (Figure 7.2-1), whereas import and timing used 2 GB each; PostgreSQL settings are unchanged (shared_buffers 1 GB). Figure 7.1-1 shows the 16 client processes during a run.

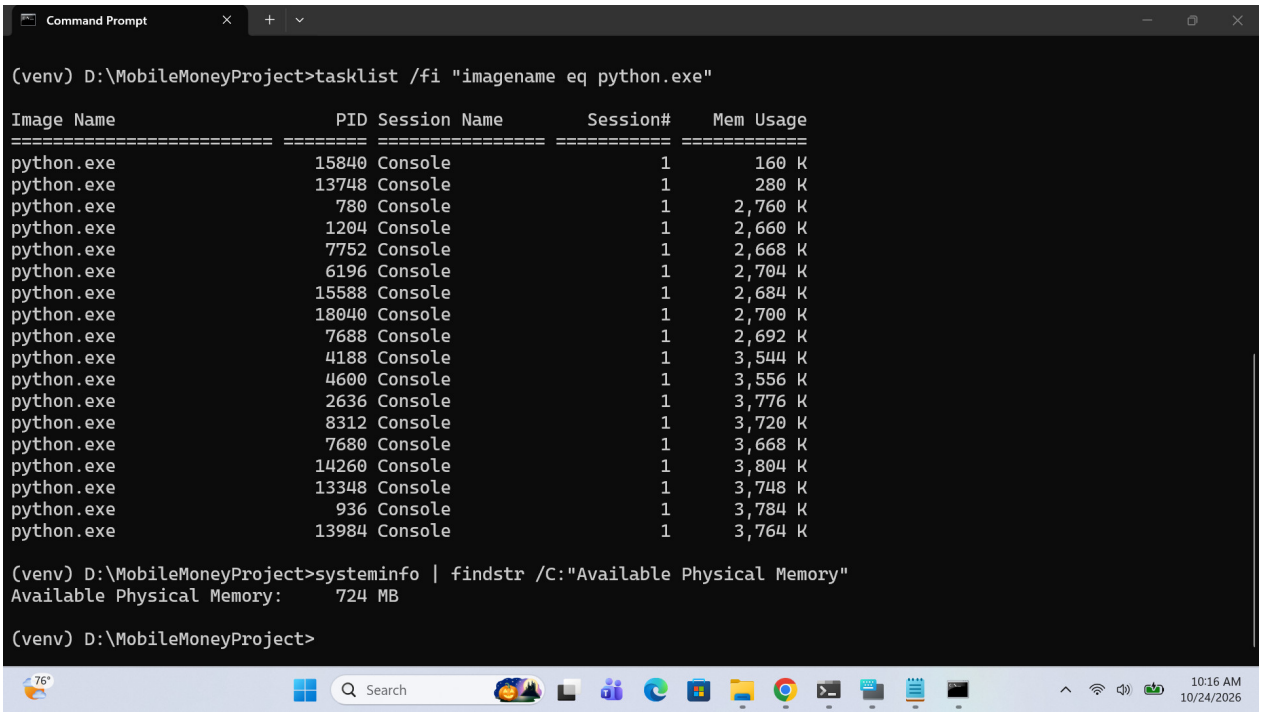


Figure 7.1-1: Active clients: 16 python.exe processes of 2.7 to 3.8 MB each (plus two small ones of 0.2 MB) and 724 MB of available physical memory, 10:16 AM on 24 October

The 16 combinations: median of three repetitions

Workload	Clients	PG req/s	PG median ms	PG p95 ms	Neo4j req/s	Neo4j median ms	Neo4j p95 ms
Q4 depth 10	1	14.98	42.5	176.3	42.23	17.6	51.2
Q4 depth 10	4	50.58	42.8	188.5	154.53	13.1	70.9
Q4 depth 10	8	78.12	61.5	252.3	372.96	10.8	61.0
Q4 depth 10	16	71.12	167.0	617.5	504.97	16.3	110.3
Q7 bound 6	1	8.22	119.9	153.6	23.78	33.6	127.5
Q7 bound 6	4	26.93	135.9	231.1	189.99	20.6	42.4
Q7 bound 6	8	32.14	205.5	503.5	303.67	26.0	61.8
Q7 bound 6	16	32.59	374.5	1,017.7	353.96	44.6	103.8

All 16 combinations ran without errors, timeouts or wrong answers. The repetitions are close except in two places, which is why the median of three is used. Neo4j Q7 with one client ran at 7.2, 23.8 and 37.3 requests per second, the first repetition being the slowest (63.0 ms median latency against 33.6 and 31.6 ms), probably because the page cache was still cold; and PostgreSQL Q4 with 16 clients began at 49.2 requests per second against 71.1 and 72.2. Every repetition is shown in Figures 7.1-4 and 7.1-5.

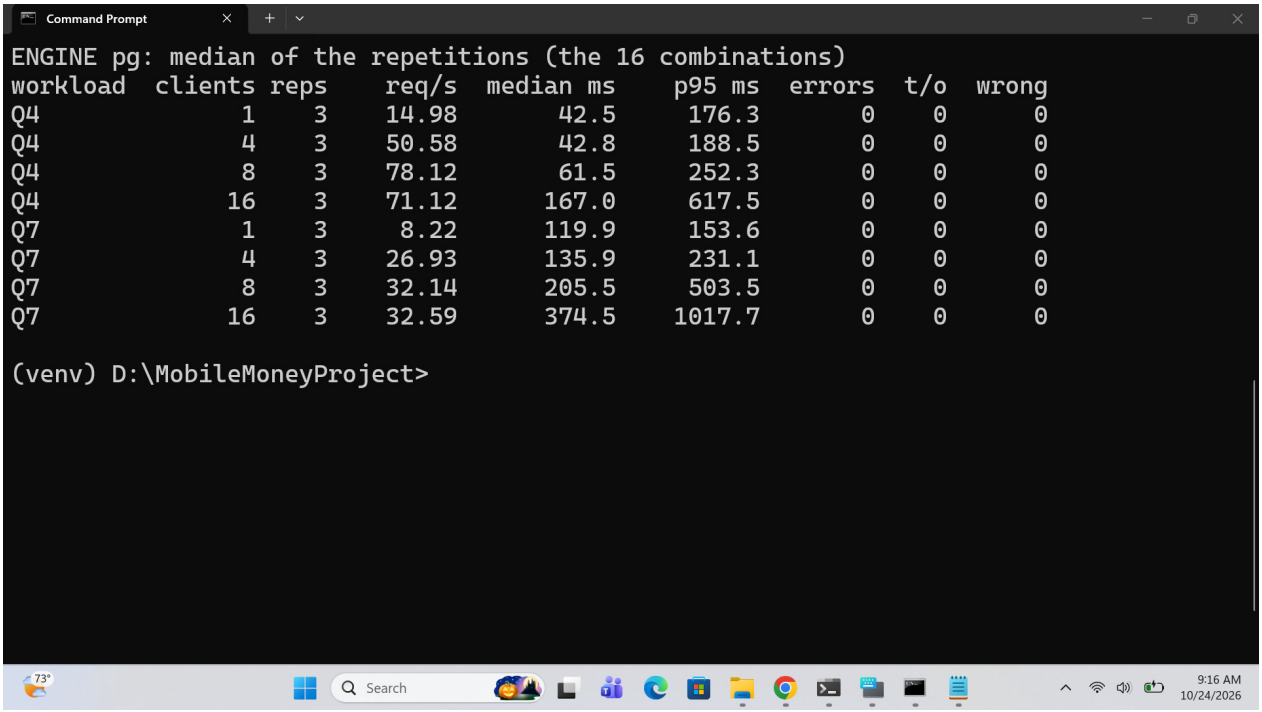


Figure 7.1-2: PostgreSQL, median of the three repetitions for the eight combinations (Q4 and Q7 with 1, 4, 8 and 16 clients): throughput, median latency, p95, errors, timeouts and wrong answers (all 0)

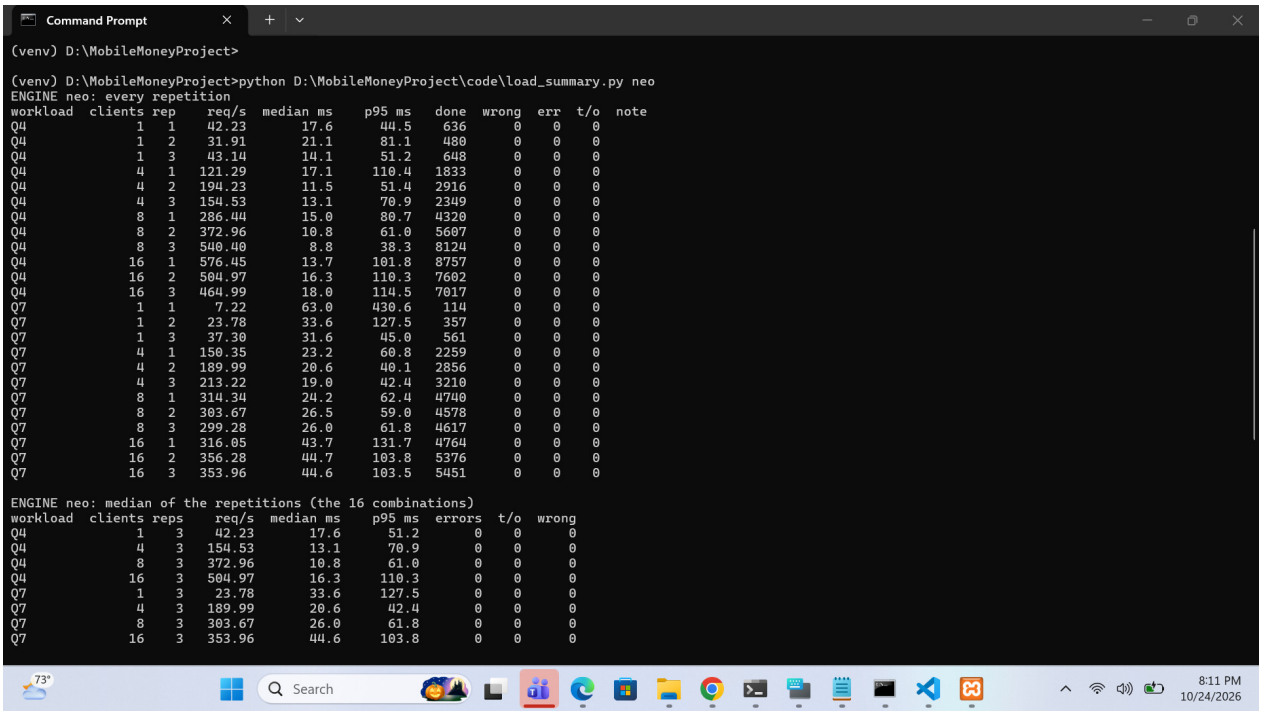


Figure 7.1-3: Neo4j, median of the three repetitions for the eight combinations (shown below the every-repetition list): throughput, median latency, p95, errors, timeouts and wrong answers (all 0)

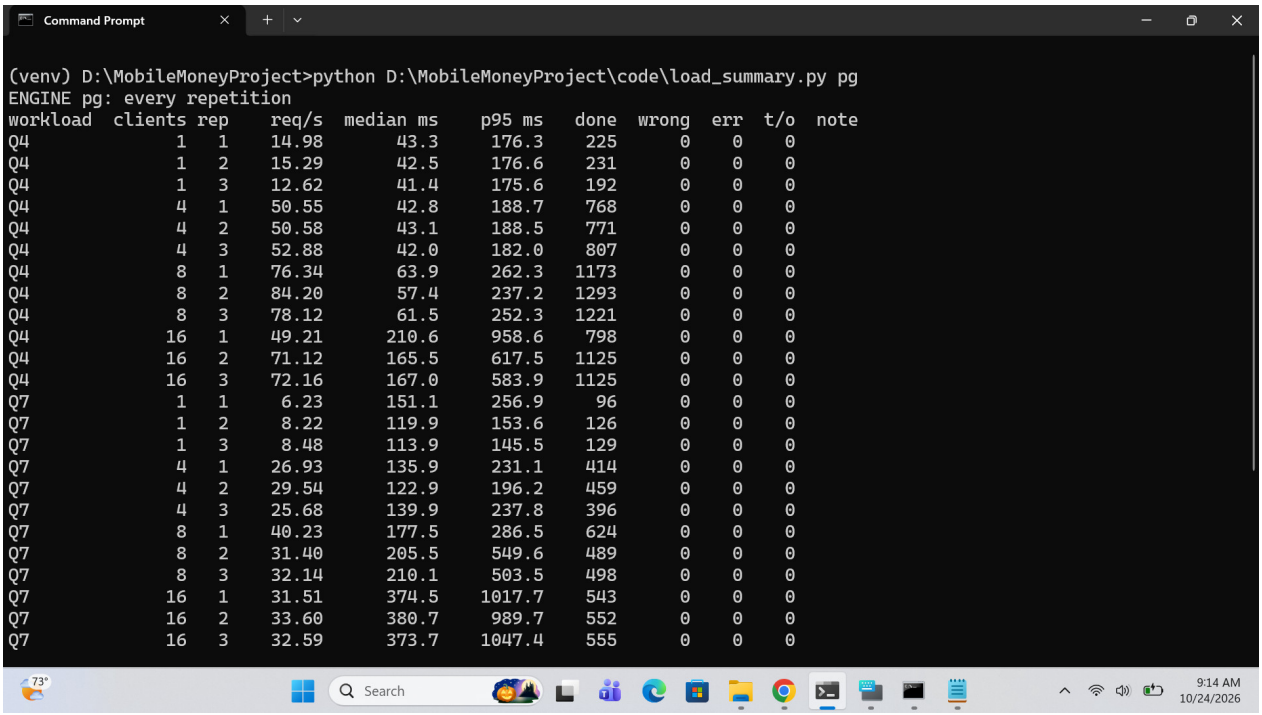


Figure 7.1-4: PostgreSQL, every repetition (24 rows): throughput, median latency, p95 and completed requests; no wrong answers, errors or timeouts

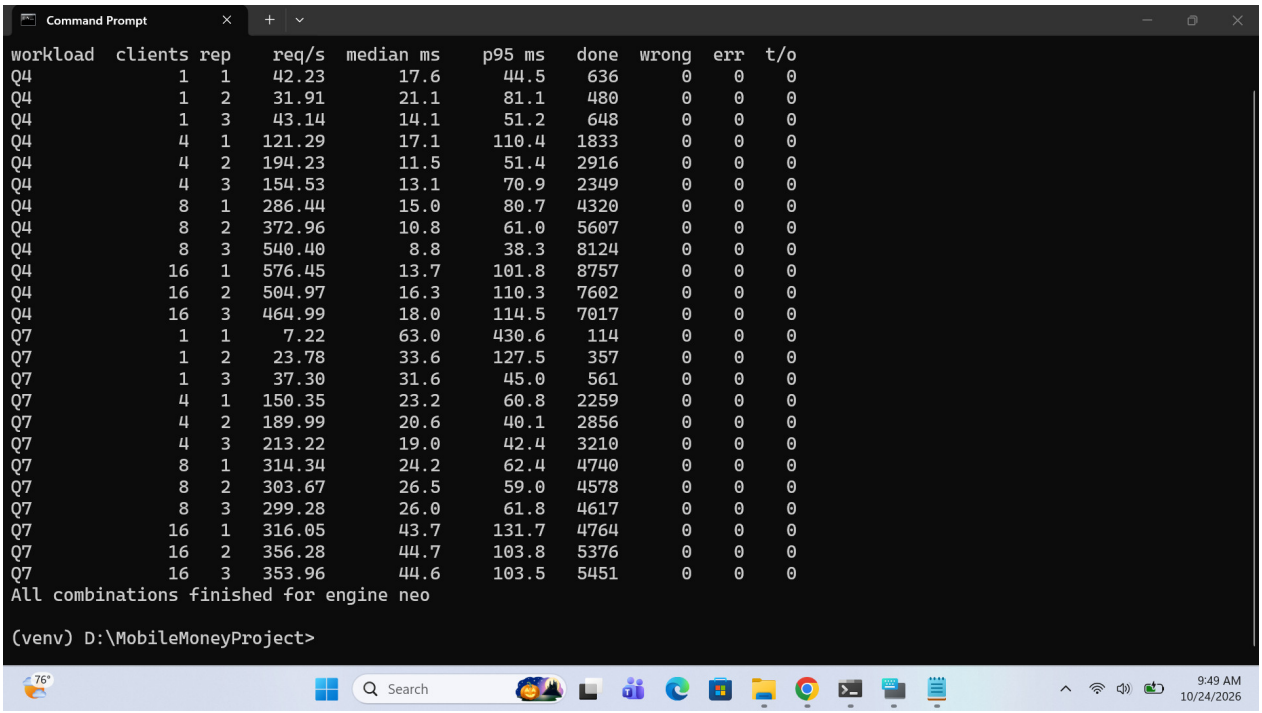


Figure 7.1-5: Neo4j, every repetition (24 rows) and the message that all combinations finished for engine neo; no wrong answers, errors or timeouts

7.2 Explain scaling

Scaling from 1 to 16 clients. PostgreSQL throughput grows 4.7 times for Q4 and 4.0 times for Q7, whereas Neo4j grows 12.0 and 14.9 times. The p95 latency of PostgreSQL rises 3.5 times (Q4) and 6.6 times (Q7), to 617.5 and 1,017.7 ms. For Neo4j it rises 2.2 times for Q4 (to 110.3 ms) and does not rise for Q7 (127.5 ms with one client, whose value is raised by the slow first repetition, and 103.8 ms with 16).

One client against 16 clients (median of three repetitions)

Engine and workload	req/s, 1	req/s, 16	Throughput factor	p95 ms, 1	p95 ms, 16	p95 factor
PostgreSQL Q4 depth 10	14.98	71.12	4.7	176.3	617.5	3.5
PostgreSQL Q7 bound 6	8.22	32.59	4.0	153.6	1,017.7	6.6
Neo4j Q4 depth 10	42.23	504.97	12.0	51.2	110.3	2.2
Neo4j Q7 bound 6	23.78	353.96	14.9	127.5	103.8	0.8

Throughput is not response time. Throughput is how many requests the system completes per second, whereas latency is what one analyst waits. PostgreSQL Q4 peaks at 8 clients (78.12 req/s) and is lower with 16 (71.12 req/s) while its median latency rises from 61.5 to 167.0 ms: the extra clients only wait in a queue. PostgreSQL Q7 is flat from 8 clients (32.14 and 32.59 req/s) while its median latency rises from 205.5 to 374.5 ms. Neo4j still gains throughput with 16 clients (Q4 372.96 to 504.97 req/s, Q7 303.67 to 353.96 req/s), but less than between 4 and 8 clients, and its median latency stays between 10.8 and 44.6 ms.

Resource and plan evidence. The laptop has 4 physical cores (8 logical processors) that the clients, the database service and Windows share. PostgreSQL starts one backend process per client and its requests are long: the improved Q7 statement uses 27,670 buffer hits and 34.2 ms of planning, and a Q4 request uses 14,285 buffer hits (Figures 6.2-5 and 6.3-2), against 19,662 db hits for Q7 and about 3,700 for Q4 in Neo4j (Figure 6.2-6). A few clients therefore occupy the processors and further clients queue, which matches the

flat throughput and the rising p95. Memory was also limited: 724 MB of the 8,035 MB was available with the 16 clients listed (Figure 7.1-1), and a snapshot at 10:17 AM (Figure 7.2-3) shows 603 MB available, 625 MB free, 367 MB of standby cache and 12 MB of modified pages. We did not measure processor use, so these figures do not separate a processor limit from a memory limit and we do not claim which one caused the saturation.

Does the graph advantage survive the SQL improvement? In this experiment, yes. With the improved Q7 SQL, 16 clients complete 354.0 requests per second in Neo4j against 32.6 in PostgreSQL (10.9 times), and Q4 gives 505.0 against 71.1 (7.1 times). The improvement reduced the 600-fold single-request gap of Section 6.2 to 1.8 to 3.7 times at bound 6, but it did not remove the gap under concurrent load, because PostgreSQL stops gaining at 8 clients. The claim is limited to these two workloads, sets A, B and C, one 4-core laptop and Neo4j with a 1 GB heap and page cache. It does not show that a graph database is faster in general, and the full-month report Q2 (Section 6.2) favours PostgreSQL.

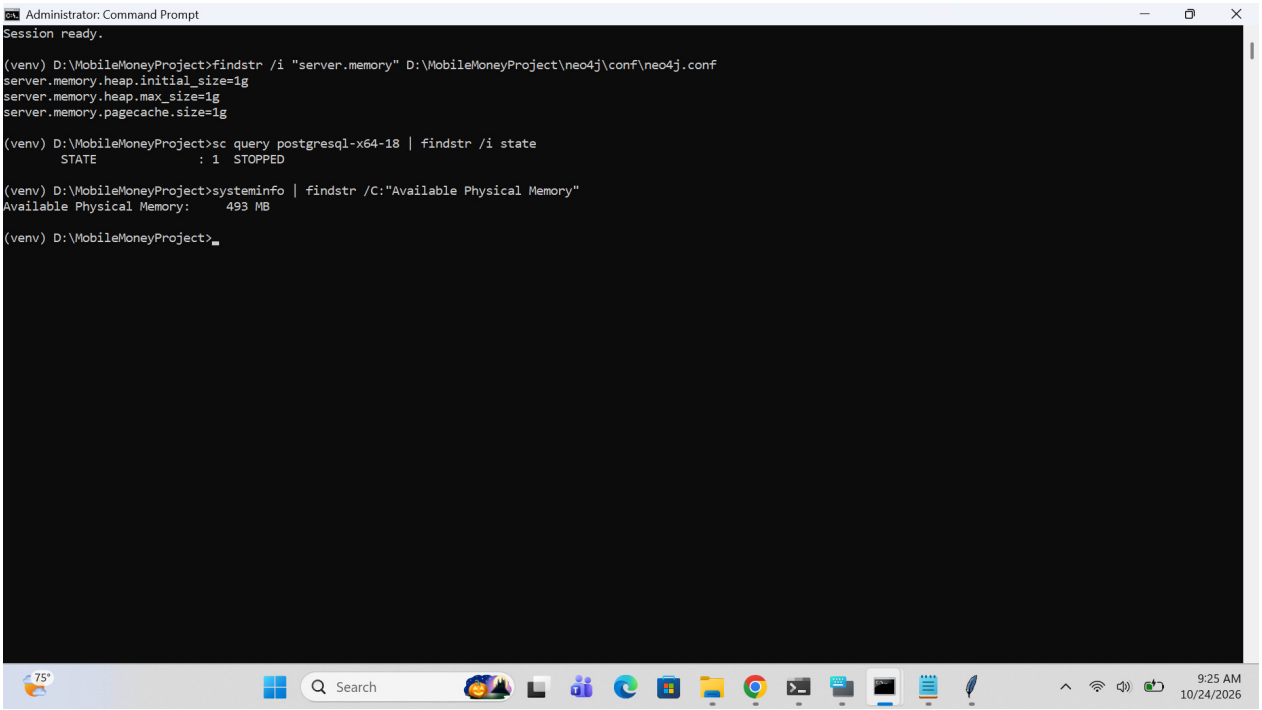


Figure 7.2-1: Neo4j settings for the load runs (heap 1 GB initial and maximum, page cache 1 GB), the PostgreSQL service stopped, and 493 MB of available physical memory, 9:25 AM on 24 October

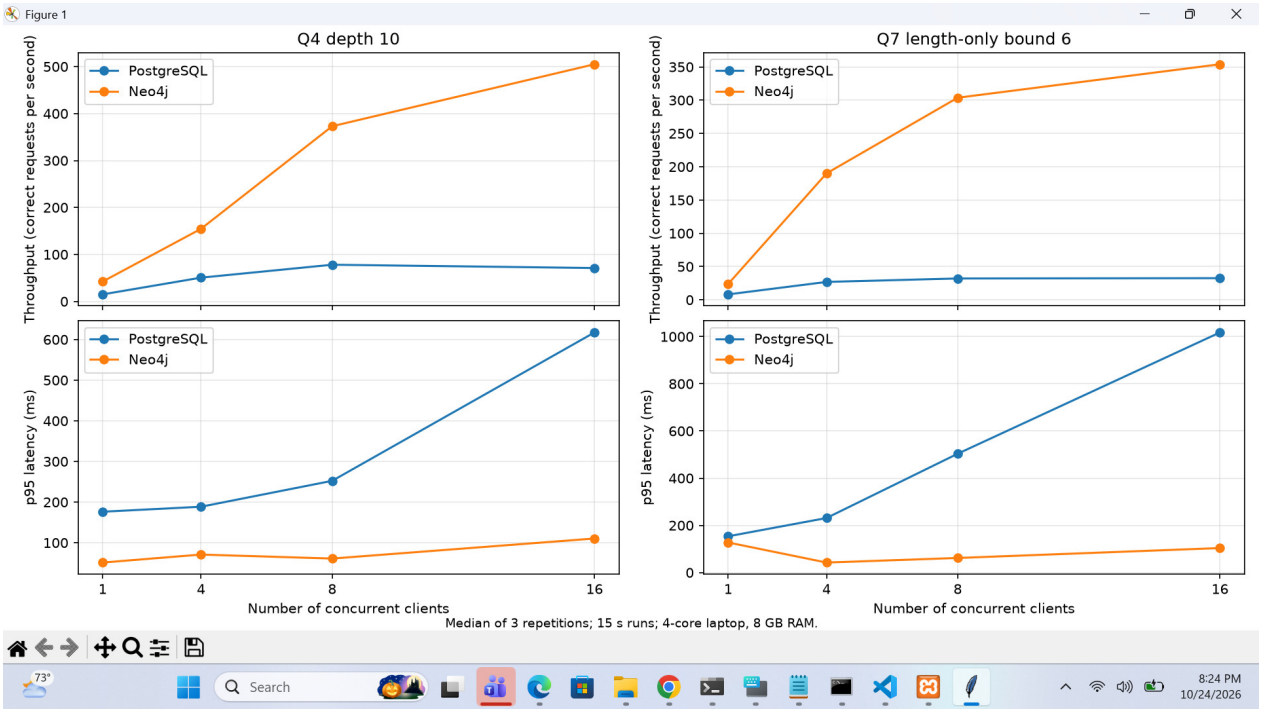


Figure 7.2-2: Throughput and p95 latency against the number of clients for Q4 depth 10 and Q7 bound 6 in PostgreSQL and Neo4j (medians of three repetitions)

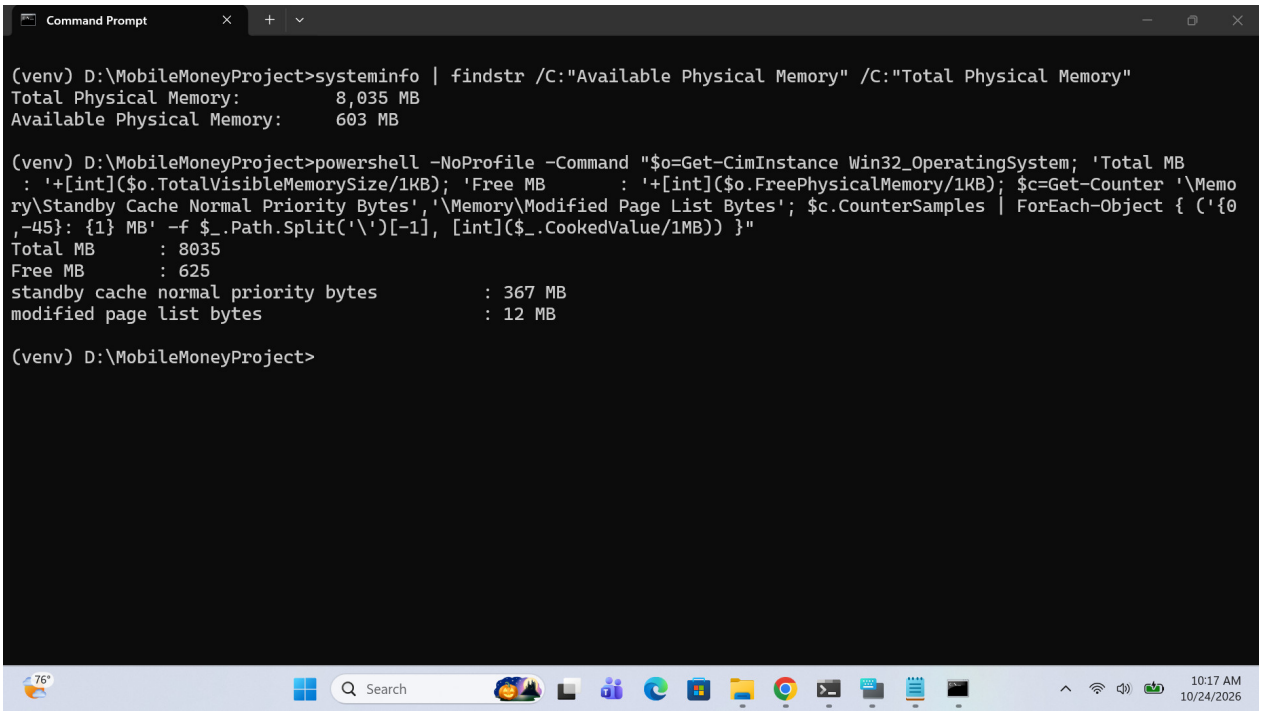


Figure 7.2-3: Memory snapshot at 10:17 AM: 8,035 MB total, 603 MB available, 625 MB free, 367 MB standby cache and 12 MB modified pages

8 Make and defend decisions

8.1 Prioritize investigation

The three pairs are compared with Q3 to Q7 and the Q1 hour value. Each cell shows the first account of the pair, then the second. The rule is fixed in advance: the account with more rapid-onward (Q5) plus circular (Q6) signals is reviewed first, and a tie goes to the larger Q1 hour value. Q3, Q4 and Q7 describe structure and are nearly equal within each pair, so they do not decide the order. This is a screening order for analysts, not a finding of fraud.

Priority comparison of the pairs

Pair	Q3 sharing accounts	Q4 reached, 6 links	Q4 reached, 10 links	Q5 chains	Q6 cycles	Q7 route (b6)	Q1 hour, RWF	Review first
A / D	4 | 4	43 | 44	187 | 192	1 | 0	1 | 0	3 | 3	200,000 | 45,000	A (998)
B / E	9 | 9	178 | 183	228 | 230	1 | 0	0 | 0	5 | 5	1,050,000 | 40,000	B (147228)
C / F	9 | 9	483 | 488	1,433 | 1,426	1 | 0	1 | 0	5 | 5	300,000 | 30,000	C (206978)

A, B and C each have a chain and D, E and F have none; A and C also have a cycle. If one list is needed, the same rule gives C, A, B, because A and C have two signals and C has the larger Q1 value, while B has one signal.

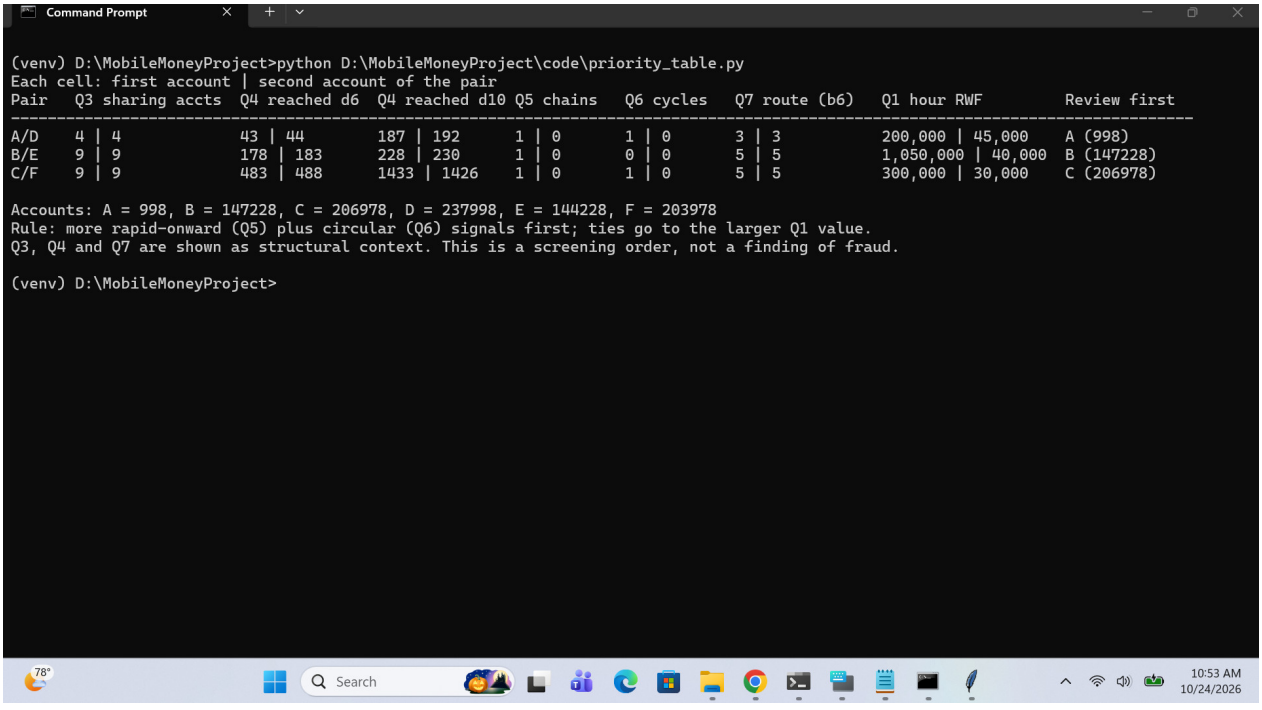


Figure 8.1-1: Priority table for the pairs A/D, B/E and C/F from Q3 to Q7 and Q1, with the rule and the account to review first

Pair A and D in detail. Account 998 (A) sends 200,000 RWF at 10:00:00 CAT on 6 September (transfer 20000001). The next hops follow 40 s apart, and each amount is exactly 98.00 % of the previous one (196,000 and 192,080 RWF); transfer 20000004 then returns 188,238 RWF to 998 at 10:02:00. Accounts 8917, 16836 and 24755 share handset 228001 with 998 for days 0 to 29. Account 237998 (D) sends 45,000

RWF at 10:00:00 on 6, 8 and 10 September, 172,800 s (two days) apart, each at 100.00 % of the previous amount, with no return, and accounts 5917, 13836 and 21755 share handset 228301 with it. A is therefore reviewed first: it combines a fast, shrinking chain with a return to the start, while D looks like a regular payment.

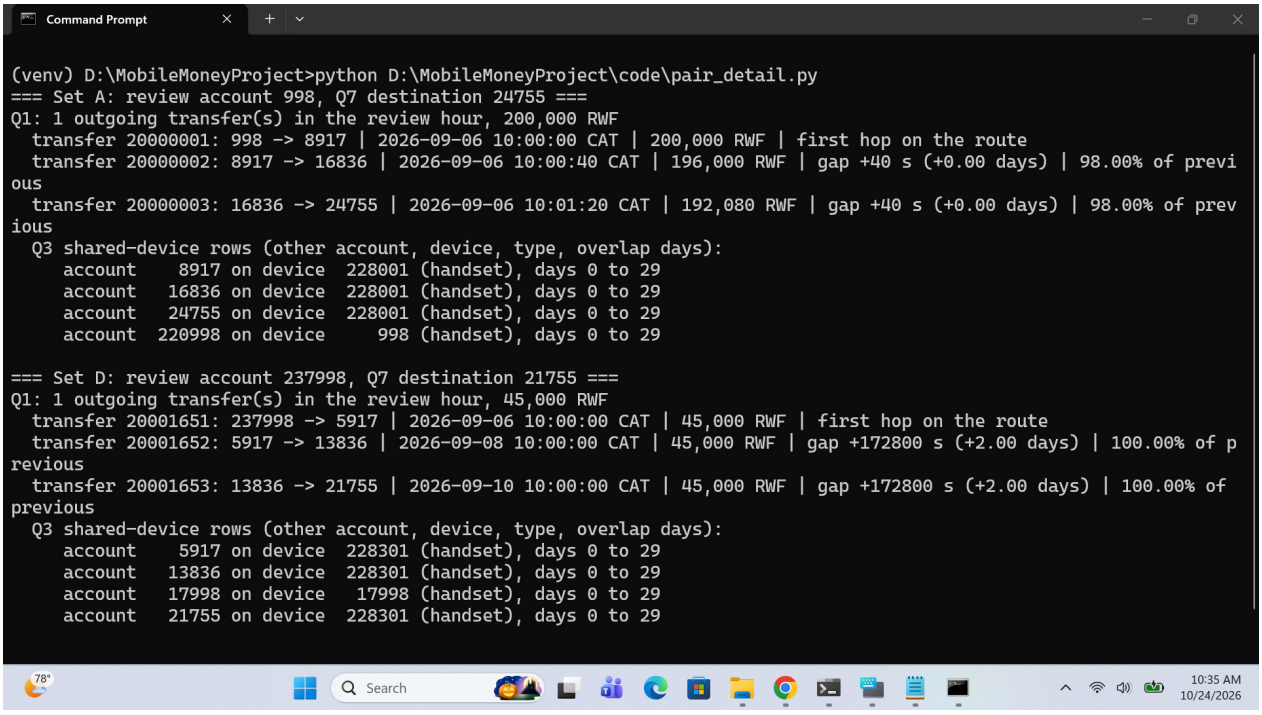


Figure 8.1-2: Pair A and D: the Q1 transfer, the onward hops with CAT times, amounts, gaps and percentage of the previous amount, and the Q3 shared-device rows

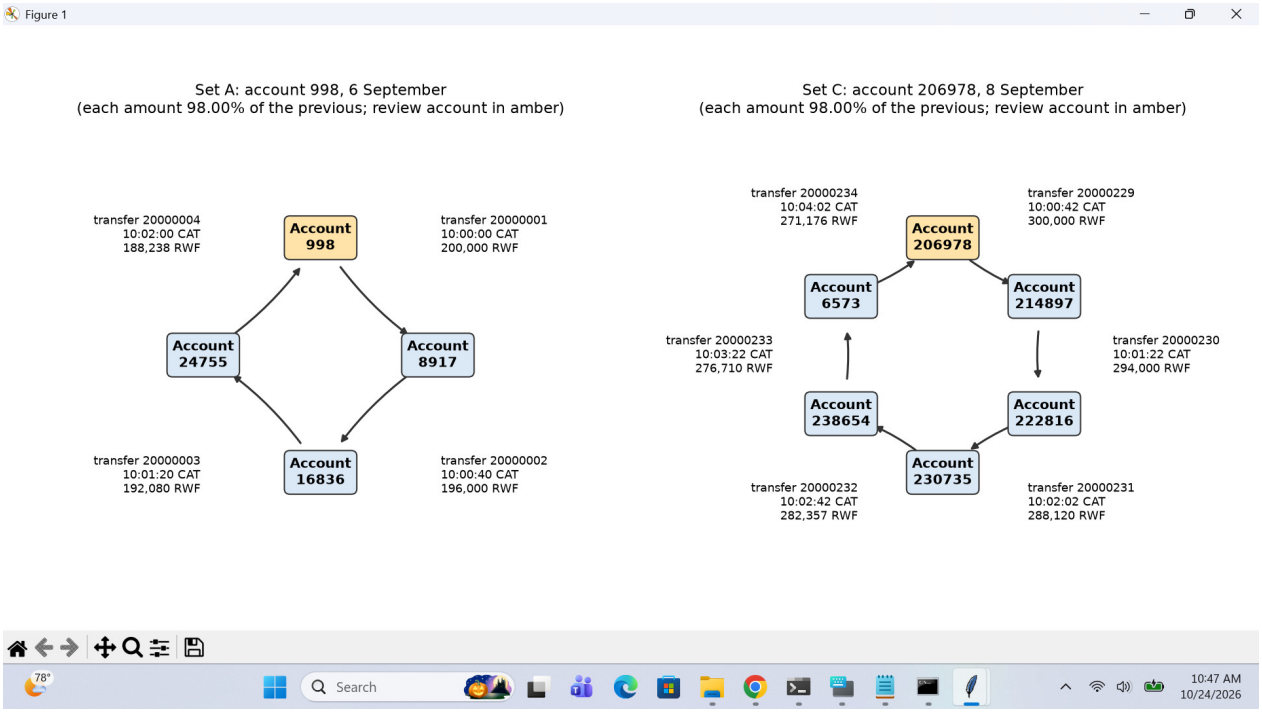


Figure 8.1-3: Transfer cycles of set A (998, four transfers, 10:00:00 to 10:02:00 CAT) and set C (206978, six transfers, 10:00:42 to 10:04:02 CAT), each amount 98.00 % of the previous

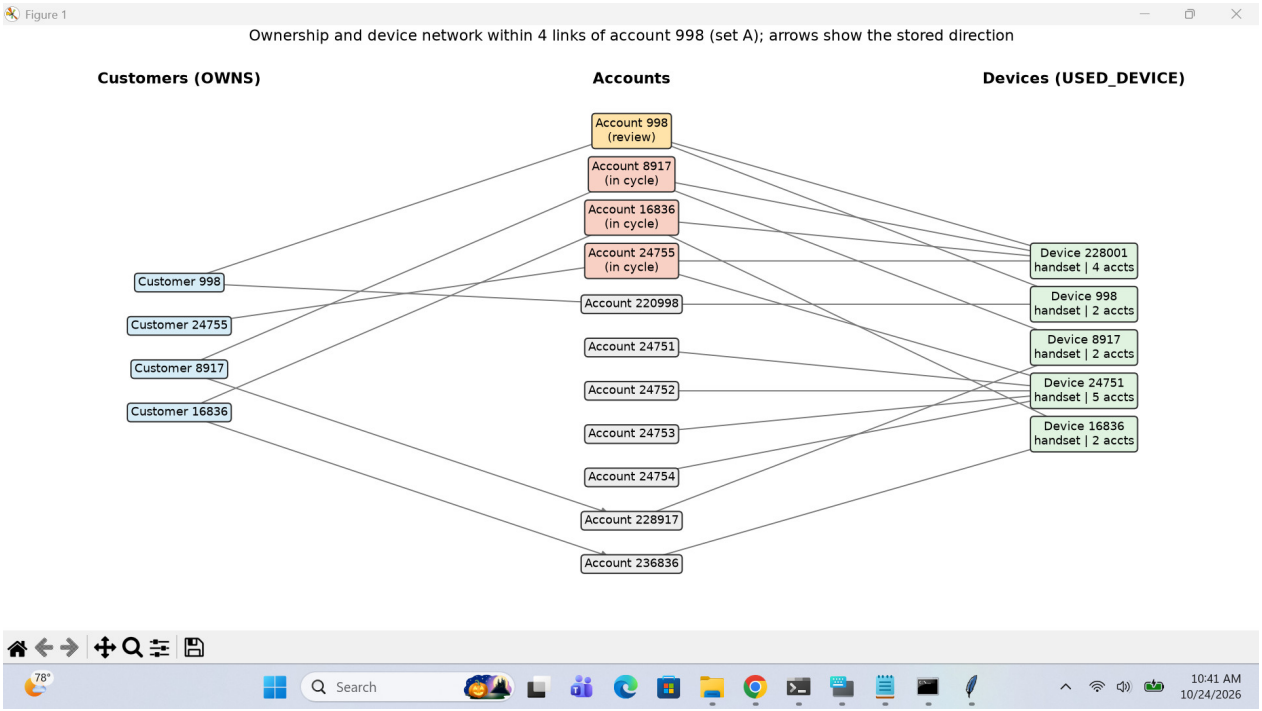


Figure 8.1-4: Ownership and device network within 4 links of account 998: the 10 other accounts of Q4, the owners of the cycle accounts, and handset 228001 shared by four accounts

An ordinary explanation. Accounts 998, 8917, 16836 and 24755 share one handset, which is common for a household or a savings group (ikimina) that passes one phone around. The 2 % lost at each hop could be a service fee, and the return of 188,238 RWF could be one member settling with another. D, with the same amount every two days, looks like a regular contribution.

Missing evidence and how it could change the judgement. We do not have balances, fees, cash-in and cash-out records, which device sent each payment (the observations do not say), whether the accounts belong to one household or group, the holders' own explanation, or a longer history. If the fee schedule shows 2 % and the holders confirm one household, A would fall in priority; if the holders are unrelated, or the money leaves as cash soon after the return, A would rise. Nothing in these records declares fraud.

8.2 Recommend a model

Recommendation. Use PostgreSQL as the system of record and for reporting, and Neo4j as a read-only investigation copy for multi-hop work.

Reporting. Q2 (daily totals for the month) took a median 21.5 s in PostgreSQL, while Neo4j exceeded the 180 s limit because its plan reads all 20,003,300 relationships one by one with no partial aggregation (Figure 6.2-7). Short lookups (Q1, Q3, Q5, Q6) took 0.5 to 4.6 ms in PostgreSQL against 17.7 to 24.9 ms in Neo4j.

Traversal. Q7 at bound 6 took 23.1 s with the first SQL and 37.9 ms in Neo4j. The plans explain it: the recursive union re-expanded accounts (2,775,123 buffer hits) while ShortestPath used 19,662 db hits (Figures 6.2-5 and 6.2-6). Cypher expresses the question as one pattern with a role condition; SQL needs a recursive query that carries its own state.

Concurrency. With 16 clients Neo4j completed 354.0 Q7 and 505.0 Q4 requests per second against 32.6 and 71.1 in PostgreSQL, whose throughput stopped growing at 8 clients (Section 7).

Load and storage. PostgreSQL loaded in 120.6 s plus 135.9 s of indexing and occupies 3.727 GB; Neo4j imported in 1 min 42 s plus about 84 s of setup and occupies 2.049 GB.

Evidence against our choice. Neo4j loads faster, is smaller and wins traversal and concurrency, which argues for using it more widely. We keep PostgreSQL as the record because it was far faster for reports and lookups, and Neo4j failed Q2.

Model or implementation. The graph model made multi-hop questions short to write and cheap to run, but much of the 600-fold Q7 gap was implementation: the both-ends SQL took 68.3 ms, cut the gap to 1.8 times at bound 6, and beat Neo4j at bound 4 (Section 6.3). What remains under load is the part we credit to the model.

Freshness. New transfers should reach the Neo4j copy within 5 minutes, with a nightly comparison of counts and sums against PostgreSQL. If the copy is stale, an analyst could miss a recent transfer and wrongly conclude that no chain, cycle or route exists.

Limits. The results come from one 4-core laptop with 8 GB of RAM, one month of data and a 1 GB Neo4j cache in the load test. Larger data, several servers and write-heavy workloads were not tested.

8.3 Explain individual judgement

Each member writes 150 to 200 words on a different measured finding, its explanation and a check that could overturn it. These notes and the lists below are written by the members themselves.

Member 1: Victor Dusengimana PostgreSQL performed better for the full-month report because it completed the query in about 21.5 seconds, while Neo4j could not finish within the 180-second limit. This is mainly because Neo4j had to read about 20 million TRANSFER relationships one by one, while PostgreSQL could scan and aggregate the data in parallel. However, this result could change if Neo4j were given more memory, a better query, or a more powerful computer. Therefore, for large daily or monthly reports, PostgreSQL appears to be the better choice

Member 2: Umwiza Odile Neo4j was faster to build and smaller on disk than PostgreSQL. PostgreSQL loaded the 20,993,500 records in 120.6 seconds and needed another 135.9 seconds for foreign keys and indexes, and the database occupies 3.727 GB. Neo4j imported the same records in 1 minute 42 seconds, needed about 84 seconds of setup, and its store is 2.049 GB. The main reason is the indexes: the transfers indexes take 2.433 GB, twice the 1.206 GB of table data, because the two covering indexes repeat three further columns beside their search keys. Neo4j holds each transfer as one relationship and only needed three ID constraints and one range index. However, the size depends on our choice of covering indexes. The check that could overturn this is to rebuild PostgreSQL without the INCLUDE columns and compare the two sizes again.

Member 3: Igirukwayo Ildephonse For Q7, Neo4j was much faster than PostgreSQL. The first SQL query took about 23 seconds, while Neo4j took only about 38 milliseconds, creating a difference of almost 600 times. The main reason was that the SQL query repeatedly expanded accounts and caused about 2.7 million buffer hits, while Neo4j's Shortest Path operation was much more efficient. However, a better SQL query could reduce this gap. This shows that we should not judge a database based on only one query, because query design can strongly affect performance.

Member 4: Alain Christian Mukiza During the load test, Neo4j performed better when many clients were connected. At 16 clients, Neo4j achieved about 354 Q7 requests/sec and 505 Q4 requests/sec, compared with only 32.6 and 71.1 requests/sec for PostgreSQL. PostgreSQL stopped gaining performance after about 8 clients, possibly because the laptop had only 4 physical CPU cores that were shared by the database, clients, and Windows. However, CPU usage was not measured, so we cannot say that this was definitely the cause. The next step would be to repeat the test on a computer with more CPU cores and controlled memory allocation to see whether the performance difference remains.

Document contributions

The work was divided by section as listed below. The recommendation (8.2), the interpretation of results and the final review of the report were done together, and every member can explain and present any part of the work if called upon.

No.	Name	Student ID	Individual contribution
1	Victor Dusengimana	101372	Sections 3 and 4. Neo4j model, import and checks (draw_graph_schema.py, prepare_import.py, neo4j_setup.cypher, neo4j_checks.cypher); account trace (trace_account.sql, trace_account.cypher); Q1 to Q3 (q1.py, q2.py, plot_q2.py, q3.py).
2	Umwiza Odile	101365	Sections 1, 2 and 9. Workspace record and connections; capture and preview (capture.py, preview_streams.py, test_connections.py); PostgreSQL schema, bulk load, indexes and checks (schema.sql, load.sql, post_load.sql, checks_a.sql, checks_b.sql, draw_pg_schema.py); README, GitHub repository, ZIP and final checking of the report (check_screenshots.py).
3	Igirukwayo Ildephonse	101404	Sections 5, 6.1 and 8. Q4 to Q7 (q4.py, plot_q4.py, q5.py, q6.py, q7.py, q7_path.py); the 61-case comparison and CSV checks (compare_all.py, csv_checks.py); the priority table, pair detail and diagrams (priority_table.py, pair_detail.py, diagram_cycle.py, diagram_ownership.py).
4	Alain Christian Mukiza	101370	Sections 6.2, 6.3 and 7. The 21-case timing and plans (timing.py, plans_pg.py, plans_neo.py, plans_neo_q2.py); the Q7 improvement (q7_v2.py); the load test, summaries and plots (load_test.py, lt_common.py, load_summary.py, plot_load.py).

LLM and external assistance

Assistance used	What it was used for	How it was checked
Claude (Anthropic LLM assistant)	Drafting sample source codes explaining plans and results; suggesting commands for evidence .	Numbers in the text were compared with the group's screenshots and result files, for example the Neo4j medians against the every-repetition output; the group ran all code and commands itself.

9 Submit reproducible work

9.1 Prepare the report

This report is submitted as one PDF in milestone order. It contains the group names and IDs, the models, summaries, charts, embedded evidence and individual notes. Repeated evidence refers to earlier screenshots, and complete outputs and raw repetitions are kept in the code submission.

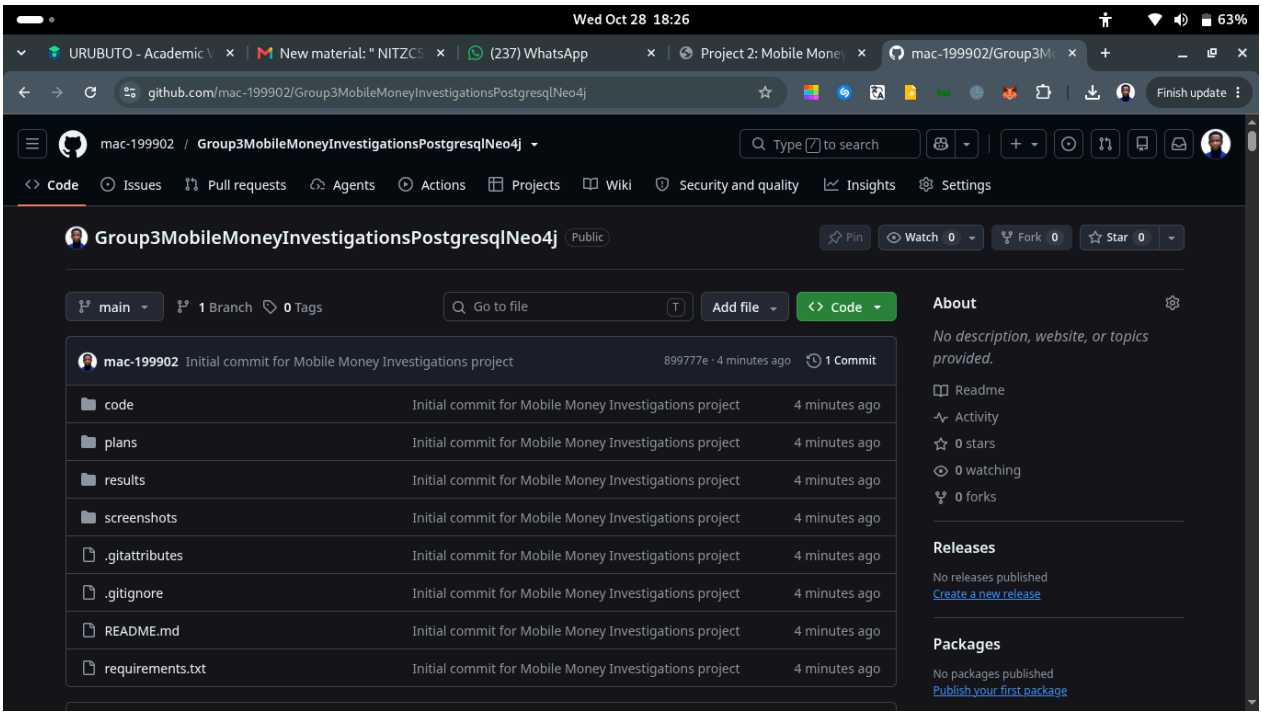
9.2 Provide code and verification

The generator, capture and import programs, definitions, queries, checks, benchmarks, raw results and plans are in the GitHub repository and in the matching ZIP file. The README lists the commands in execution order and uses placeholder secrets. The CSV files and database stores are kept local.

GitHub repository

Repository URL: https://github.com/mac-199902/Group3MobileMoneyInvestigationsPostgresqlNeo4j

Matching ZIP file: MobileMoneyProject



Generator, rerun and stopped services

Generator unchanged. The SHA256 hash of code\mobile_money_generator.py was computed again at submission with certutil (Figure 9.2-1): 2ea0e59d2b7b987d84a3d151bd9d1ad7b3052bfbc25abf1466c8e94467e9455a. The file results\generator_hash.txt, saved at the start of the project (Section 1.1, Figure 1.1-1), holds the same value (Figure 9.2-2), so the generator is unchanged.

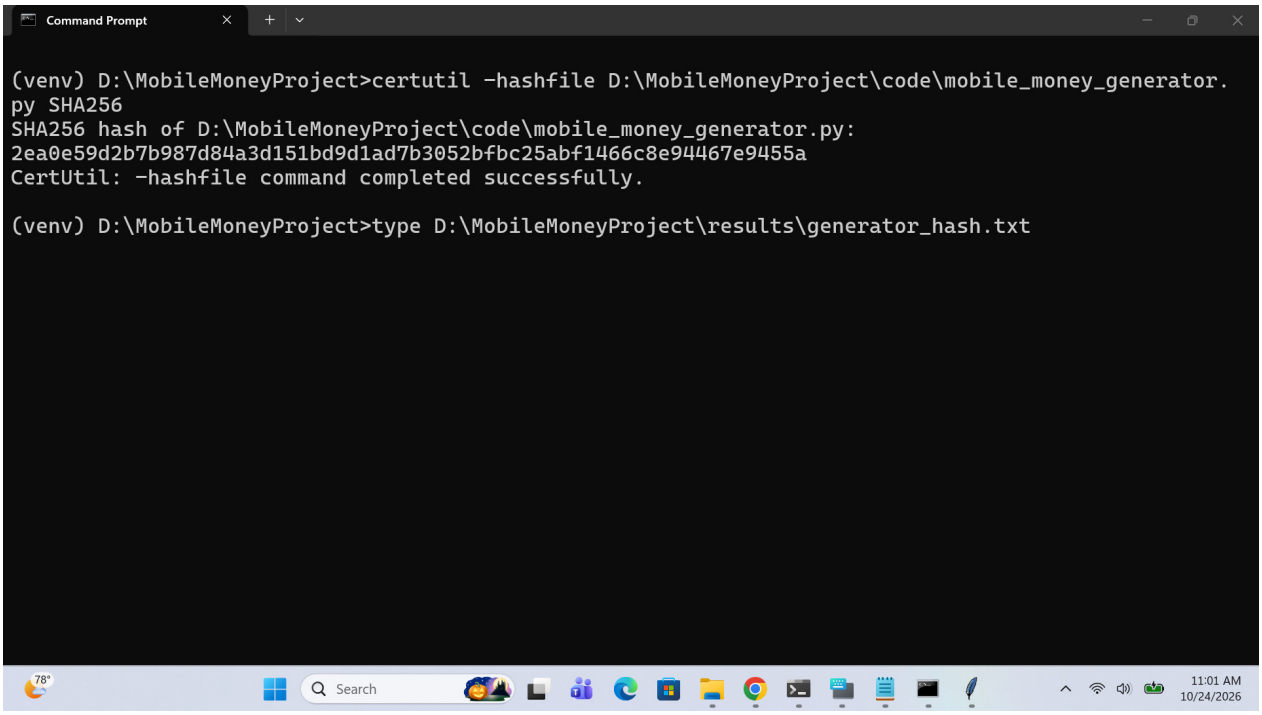


Figure 9.2-1: SHA256 of the project copy of mobile_money_generator.py computed with certutil at submission: 2ea0e59d2b7b987d84a3d151bd9d1ad7b3052bfbc25abf1466c8e94467e9455a

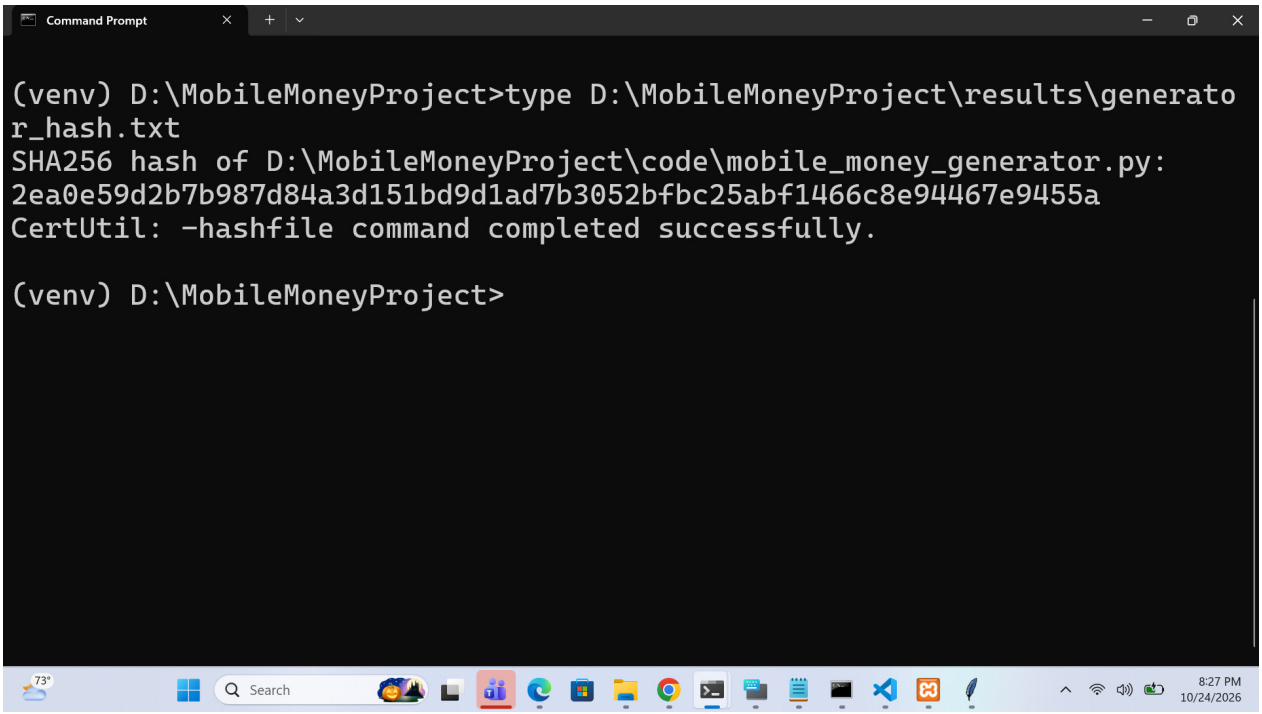


Figure 9.2-2: The saved results\generator_hash.txt holds the same SHA256 value

Rerun of the 61-case comparison. The submitted comparison was run again at the end with Neo4j on the 1 GB heap and 1 GB page cache of Section 7. All 61 cases match and none mismatch (Figure 9.2-3), which agrees with Figure 6.1-2.

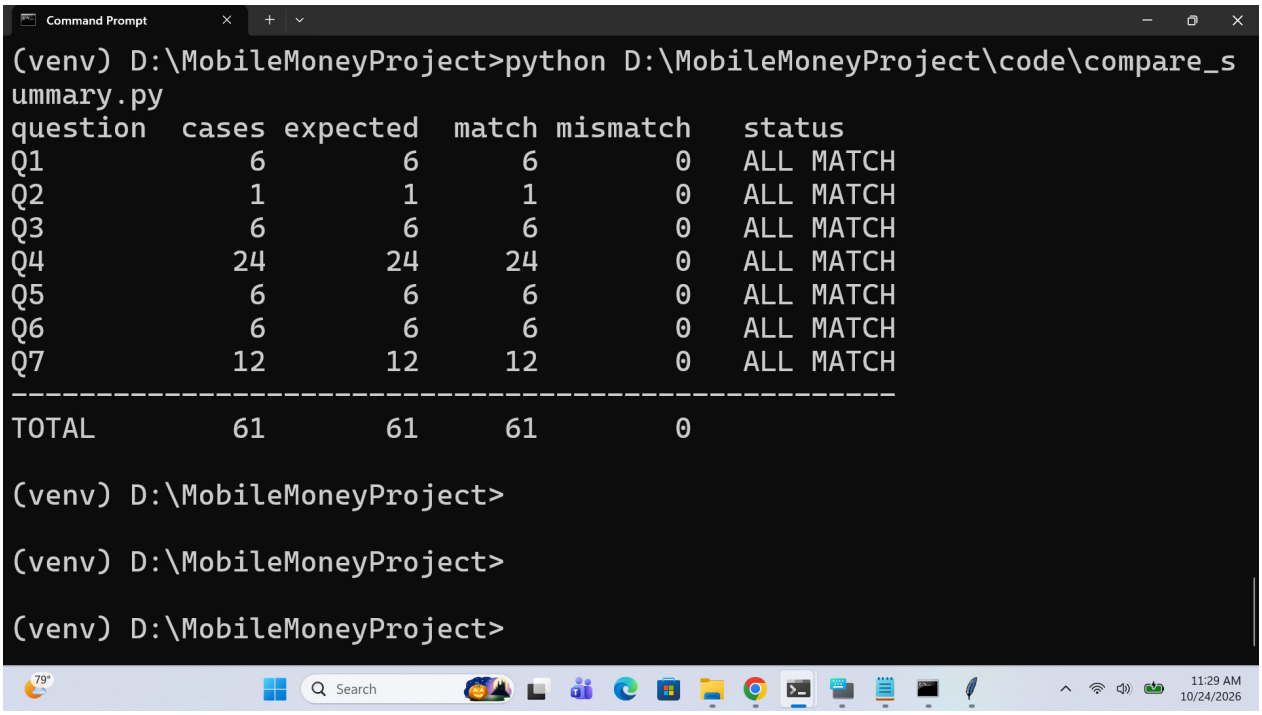


Figure 9.2-3: Rerun of the 61-case comparison: Q1 to Q7 all match, 61 cases, 61 matches, 0 mismatches

Services stopped. Afterwards the PostgreSQL service and Neo4j were stopped: the PostgreSQL service reports STOPPED and nothing is listening on port 7687 (Neo4j) or port 5432 (PostgreSQL) (Figure 9.2-4). The database stores are kept on drive D:. Figure 9.2-5 repeats the check later and also shows that neo4j.bat status could not run from that administrator window: its default Java is older than the Java 21 that Neo4j needs, so it printed a Java version error instead of a status. The empty port checks are therefore the evidence that Neo4j is stopped.

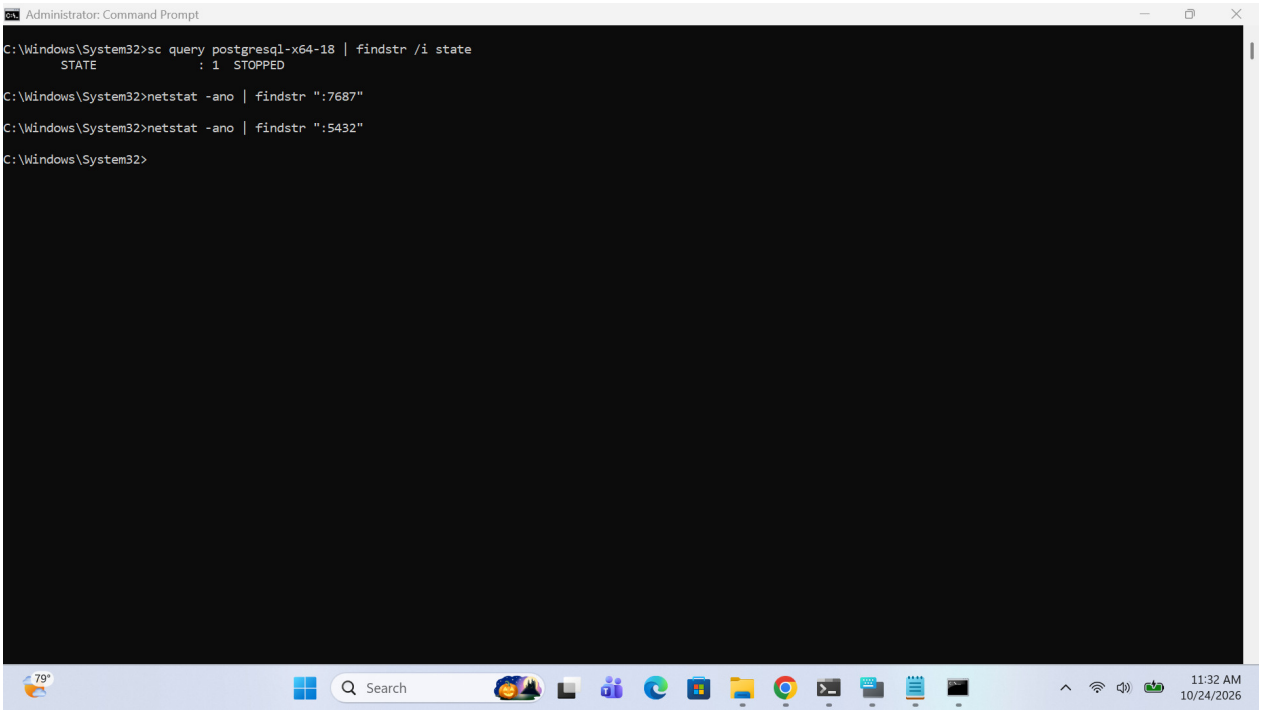


Figure 9.2-4: Stopped services: postgresql-x64-18 is STOPPED and no listener is found on ports 7687 and 5432

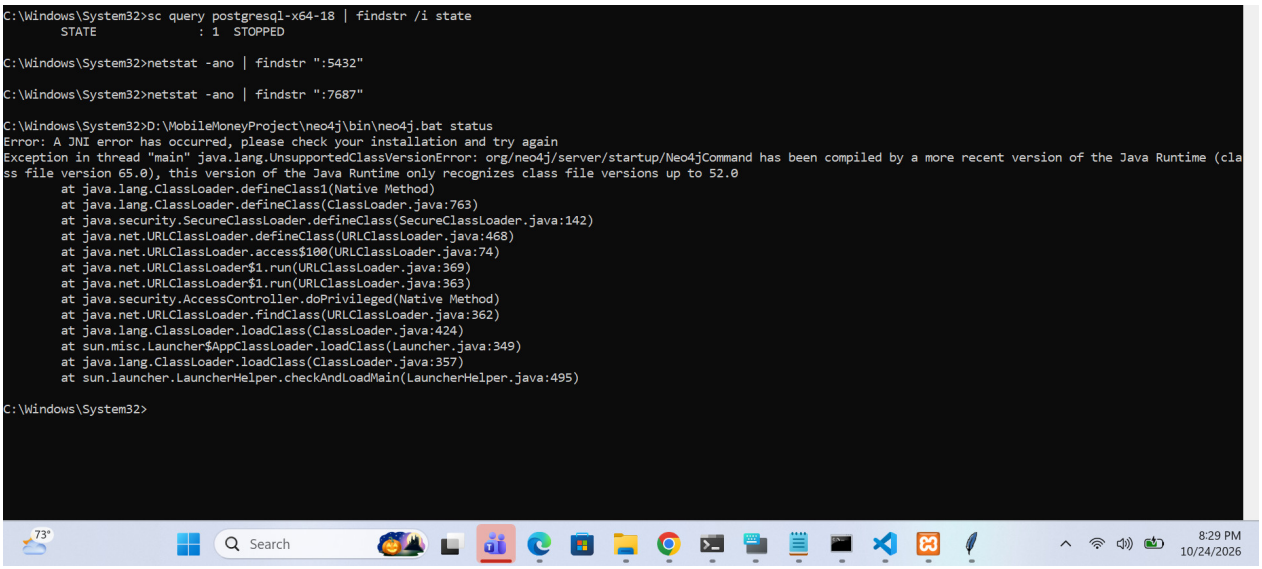


Figure 9.2-5: Second check: PostgreSQL STOPPED and nothing listening on ports 5432 and 7687; the neo4j.bat status command failed with a Java version error in this window